DEVELOPMENT OF AN NLP-DRIVEN CURRICULUM RECOMMENDATION SYSTEM FOR NUC CCMAS 30% LOCALIZED CURRICULUM CUSTOMISATION

BY

ESHIET, OWOABASI PAUL
21/SC/CO/1110

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE,
FACULTY OF COMPUTING, UNIVERSITY OF UYO, UYO,
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
BACHELOR OF SCIENCE (B.Sc.) DEGREE IN COMPUTER SCIENCE

SUPERVISOR: MR. DAN AMBROSE

SEPTEMBER, 2026

DECLARATION

I, Eshiet, Owoabasi Paul, with registration number 21/SC/CO/1110, hereby declare that this project, titled "Development of an NLP-Driven Curriculum Recommendation System for NUC CCMAS 30% Localized Curriculum Customisation", is the result of my own work carried out under the supervision of Mr. Dan Ambrose in the Department of Computer Science, Faculty of Computing, University of Uyo. It has not been submitted, in whole or in part, for the award of any degree or diploma elsewhere. All sources of information used have been duly acknowledged by means of references.

…… ……

Eshiet, Owoabasi Paul (21/SC/CO/1110) Signature / Date

Signature and date

CERTIFICATION

This is to certify that this project, titled "Development of an NLP-Driven Curriculum Recommendation System for NUC CCMAS 30% Localized Curriculum Customisation", was carried out by Eshiet, Owoabasi Paul (21/SC/CO/1110) in the Department of Computer Science, Faculty of Computing, University of Uyo, and meets the requirements for the award of the Bachelor of Science (B.Sc.) degree in Computer Science.

…… ……

Mr. Dan Ambrose Signature / Date

Project Supervisor

…… ……

Prof. Patience Usip Signature / Date

Head of Department

…… ……

[Name of External Examiner] Signature / Date

External Examiner

DEDICATION

This work is dedicated to Almighty God, whose wisdom, grace, and unfailing mercies have been my constant guide throughout this academic journey.

To my family, whose sacrifice, prayers, and unwavering support made this possible; to my supervisor, whose scholarly rigour and guidance shaped this work; and to all who believe that Nigerian universities can deliver curriculum content worthy of the industries their graduates will serve.

ACKNOWLEDGEMENTS

I am grateful to my supervisor, Mr. Dan Ambrose, for his guidance, patience and constructive criticism throughout this project. His comments shaped both the system and this report.

I thank the Head of Department, Prof. Patience Usip, and all the lecturers and staff of the Department of Computer Science, Faculty of Computing, University of Uyo, for the knowledge and support they have given me throughout my studies.

I also thank [names of the annotators, usability-test participants and any other people or organisations who helped].

My sincere appreciation goes to my family, Pst. and Mrs. Paul Eshiet, Ukpono, Udeme, Edidiong and Nsongurua, for their love, prayers and support, and to my friends and course mates, especially members of the Nigerian Association of Computer Science Students (NACOS), University of Uyo chapter, for their encouragement.

I also thank my friends Daniel Awaih, Godsfave Etim, Abasiono Monday and Peter Kingsley for their love and support throughout this journey.

ABSTRACT

The Core Curriculum and Minimum Academic Standards (CCMAS) of the National Universities Commission (NUC) provide about 70% of each Nigerian university programme as a national core and leave the remaining 30% to be designed by each university, but offer no method for deciding what that 30% should contain. In practice, the decision rests largely on the opinions of curriculum committees, with little recorded evidence and a risk of duplicating the core. This project developed SkillBridge, a web-based, NLP-driven curriculum recommendation system that brings evidence from job adverts and policy documents into this decision. The system validates, parses, cleans and segments documents into short extracts, encodes them with Sentence-BERT (all-MiniLM-L6-v2), extracts skills with a rule-based spaCy EntityRuler of 302 patterns, discovers topics with BERTopic (UMAP and HDBSCAN), and compares each topic by cosine similarity with the 86 subject courses of the CCMAS for Computing. Topics are ranked with a transparent score combining employer demand (40%), how often the topic comes up (35%) and how new it is relative to the NUC core (25%), and presented with their evidence to curriculum planners, who accept or reject them, design courses within the 30% credit allowance and generate reports. The system was built incrementally with React, TypeScript, Flask, PostgreSQL with pgvector and Redis, with three server-enforced user roles. Applied to 55 Nigerian computing job adverts and the National Digital Economy Policy and Strategy 2020–2030, it produced 20 ranked suggested topics, led by IT support and help desk operations, threat intelligence and vulnerability management, and testing and Agile practice, none of which was closely matched by an NUC core course (highest similarity 0.735). Its topics were more coherent than those of an LDA baseline (C_v 0.723 against 0.488), repeated runs gave identical results, a warm analysis took 24 seconds on a two-core laptop, and all 522 automated tests that could run on the development machine passed, with 97.1% back-end coverage. Evaluation of skill extraction, overlap detection and usability with human participants is in progress. The study concludes that NLP can make the design of the CCMAS 30% more evidence-based and accountable while leaving the final decisions with people.

Keywords: *curriculum recommendation; CCMAS; natural language processing; BERTopic; Sentence-BERT; job adverts; skills gap; Nigeria*

ABBREVIATIONS

API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BMAS	Benchmark Minimum Academic Standards
CCMAS	Core Curriculum and Minimum Academic Standards
CSRF	Cross-Site Request Forgery
c-TF-IDF	Class-based Term Frequency–Inverse Document Frequency
DFD	Data Flow Diagram
DOCX	Microsoft Word Open XML Document
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
LDA	Latent Dirichlet Allocation
NDEPS	National Digital Economy Policy and Strategy
NER	Named Entity Recognition
NITDA	National Information Technology Development Agency
NLP	Natural Language Processing
NUC	National Universities Commission
PDF	Portable Document Format
REST	Representational State Transfer
RQ	Redis Queue
SBERT	Sentence-BERT
SIWES	Students Industrial Work Experience Scheme
SUS	System Usability Scale
TF-IDF	Term Frequency–Inverse Document Frequency
UML	Unified Modeling Language
UMAP	Uniform Manifold Approximation and Projection

TABLE OF CONTENTS

CHAPTER ONE INTRODUCTION....1
1.1 Background of the Study....1
1.2 Statement of the Problem....2
1.3 Aim and Objectives of the Study....3
1.4 Significance of the Study....3
1.5 Scope of the Study....4
1.6 Limitations of the Study....5
1.7 Definition of Terms....5
1.8 Organisation of the Report....6
CHAPTER TWO LITERATURE REVIEW....7
2.1 Introduction....7
2.2 Conceptual Review....7
2.2.1 Curriculum Development in Nigerian Universities....7
2.2.2 Graduate Employability and the Skills Gap in Nigeria....8
2.2.3 The National Digital Economy Policy and Strategy....9
2.2.4 Natural Language Processing and Text Preprocessing....9
2.2.5 Finding Important Words with TF-IDF....10
2.2.6 Named Entity Recognition and Skill Extraction....10
2.2.7 Representing Meaning with Embeddings....11
2.2.8 Discovering Topics....12
2.2.9 Recommender Systems....13
2.2.10 Measures for Evaluation....13
2.3 Review of Related Works....14
2.4 Summary of Related Works....15
2.5 Research Gap....16
2.6 Summary....17
CHAPTER THREE METHODOLOGY AND SYSTEM DESIGN....18
3.1 Introduction....18
3.2 Research and Development Methodology....18
3.2.1 Choice of Methodology....18
3.2.2 Development Phases....19
3.2.3 Development Tools....20

3.3 Analysis of the Existing System....20
3.4 The Proposed System....21
3.5 Requirements Specification....22
3.5.1 Functional Requirements....22
3.5.2 Non-Functional Requirements....23
3.6 Data Collection....24
3.6.1 Job Adverts and Policy Document....24
3.6.2 The NUC CCMAS Core....24
3.6.3 Skill Patterns and Supporting Lists....25
3.7 System Architecture....25
3.8 Design of the Processing Pipeline....26
3.8.1 Document Ingestion....27
3.8.2 Keywords and Skills....28
3.8.3 Finding, Merging and Naming Topics....28
3.8.4 Comparing Topics with the NUC Core....29
3.8.5 Scoring Model....30
3.8.6 Analysis Algorithm....31
3.9 Behavioural Design....32
3.9.1 Use Case Diagram....32
3.9.2 Activity Diagram....33
3.9.3 Sequence Diagram....34
3.10 Database Design....36
3.11 Security and Access-Control Design....37
3.12 User Interface Design....38
3.13 Evaluation Design....39
3.14 Summary....39
CHAPTER FOUR IMPLEMENTATION, TESTING AND RESULTS....40
4.1 Introduction....40
4.2 Implementation Environment....40
4.2.1 Hardware....40
4.2.2 Software....40
4.3 Implementation of the System Modules....41
4.4 System Interfaces....42
4.4.1 Sign In and Dashboard....42

4.4.2 Documents....44
4.4.3 Running an Analysis....46
4.4.4 Evidence and Suggested Topics....47
4.4.5 Topic Detail....49
4.4.6 Course Design, Proposed Courses and Reports....51
4.4.7 Administration....54
4.5 Testing....58
4.5.1 Testing Strategy....58
4.5.2 Test Results....59
4.6 Results....60
4.6.1 Corpus Processing....60
4.6.2 Skills in Demand....60
4.6.3 Suggested Topics....61
4.6.4 Overlap with the NUC Core....64
4.6.5 Topic Quality Compared with LDA....64
4.6.6 Repeatability....65
4.6.7 Processing Speed....65
4.7 Evaluation with Human Participants....66
4.7.1 Skill Extraction Accuracy....66
4.7.2 Overlap Detection and Threshold Calibration....67
4.7.3 Usability....67
4.8 Discussion of Results....67
4.9 Summary....68
CHAPTER FIVE SUMMARY, CONCLUSION AND RECOMMENDATIONS....70
5.1 Summary....70
5.2 Conclusion....71
5.3 Contributions to Knowledge....71
5.4 Recommendations....72
5.5 Suggestions for Further Work....73
REFERENCES....74
APPENDIX A SYSTEM USABILITY SCALE (SUS) QUESTIONNAIRE AND TASKS....80
APPENDIX B API ENDPOINTS....81
APPENDIX C SOURCE CODE AND DATA....85

LIST OF TABLES

Table 2.1: Summary of related works.....15
Table 3.1: Development phases.....19
Table 3.2: Functional requirements.....22
Table 3.3: Documents in the final analysis.....24
Table 3.4: Topic-finding settings.....29
Table 3.5: Database tables.....37
Table 3.6: What each role may do.....37
Table 3.7: Evaluation design.....39
Table 4.1: Hardware used.....40
Table 4.2: Software components and versions.....40
Table 4.3: Automated test results.....59
Table 4.4: Examples of verified behaviours.....59
Table 4.5: Summary of the final analysis.....60
Table 4.6: Suggested topics of the final analysis (session 13).....61
Table 4.7: Topic coherence of BERTopic and LDA.....64
Table 4.8: Time per stage of the analysis (seconds).....65
Table A.1: SUS items.....80
Table A.2: Usability tasks (on the final analysis, session 13).....80
Table B.1: API endpoints.....81

LIST OF FIGURES

Figure 3.1: Development phases and the cycle followed within each phase.....19
Figure 3.2: Flowchart of the existing process for deciding the 30% component.....20
Figure 3.3: Context diagram (level-0 data flow diagram) of the proposed system.....22
Figure 3.4: System architecture.....25
Figure 3.5: Flowchart of the processing pipeline.....27
Figure 3.6: How a topic's score is calculated, using the top-ranked topic, IT Support and Help Desk Operations, as an example (values rounded).....31
Figure 3.7: Use case diagram.....33
Figure 3.8: Activity diagram of the curriculum planning workflow.....34
Figure 3.9: Sequence diagram for running an analysis.....35
Figure 3.10: Entity-relationship diagram (main tables and fields).....36
Figure 4.1: Sign-in page.....43
Figure 4.2: Dashboard of a Curriculum Planner.....43
Figure 4.3: Read-only dashboard of a Viewer.....44
Figure 4.4: Documents library.....45
Figure 4.5: Upload documents dialog.....45
Figure 4.6: Document detail.....46
Figure 4.7: New analysis, choosing documents, with the Upload documents button..47
Figure 4.8: Analysis in progress.....47
Figure 4.9: Evidence page.....48
Figure 4.10: Suggested topics.....49
Figure 4.11: Topic detail for the top-ranked topic.....50
Figure 4.12: Score breakdown with "Show calculation" open.....50
Figure 4.13: Comparison of a topic with the NUC core.....51
Figure 4.14: Design course form.....52
Figure 4.15: Proposed courses.....52
Figure 4.16: Reports.....53
Figure 4.17: First page of a generated PDF report.....54
Figure 4.18: NUC core curriculum.....55
Figure 4.19: Users and roles.....55
Figure 4.20: Scoring settings.....56
Figure 4.21: Course-style topic names catalogue.....57

Figure 4.22: Activity log.....57
Figure 4.23: No access page shown to a Viewer.....58
Figure 4.24: The ten skills mentioned in the most documents.....61
Figure 4.25: Composition of the scores of the 20 suggested topics.....63
Figure 4.26: Topic coherence of BERTopic and the LDA baseline.....65
Figure 4.27: Time per stage, first run and warm run.....66

CHAPTER ONE
INTRODUCTION

1.1 Background of the Study

Universities exist, among other purposes, to prepare graduates who can contribute meaningfully to the economy and society. In Computer Science and related disciplines, this responsibility is especially demanding because the field changes quickly: tools, platforms and job roles that were uncommon a few years ago, such as cloud infrastructure, DevOps practice and threat intelligence, are now routine requirements in job adverts. A curriculum that is not reviewed against these changes gradually drifts away from what graduates will actually be asked to do.

In Nigeria, the National Universities Commission (NUC) regulates the content of university programmes. For many years this was done through the Benchmark Minimum Academic Standards (BMAS). Beginning in 2018, the Commission restructured the BMAS into the Core Curriculum and Minimum Academic Standards (CCMAS), with the stated aim of reflecting twenty-first-century realities and setting a measurable benchmark of knowledge, skills and competences (Rasheed, n.d.). The CCMAS for Computing covers seven Bachelor of Science programmes, namely Computer Science, Cybersecurity, Data Science, Information and Communication Technology, Information Systems, Information Technology and Software Engineering (National Universities Commission [NUC], 2022). Implementation began in the 2023/2024 academic session (NUC, 2023).

A defining feature of the CCMAS is its division of each programme into two parts. About 70% of the content is a national core written by NUC-appointed experts; the remaining 30% is left to each university, through its Senate, to design "for other innovative courses" that reflect local needs and the institution's strengths (NUC, 2022; NUC, 2023). This 30% is a significant opportunity. It allows a department such as the Department of Computer Science at the University of Uyo to respond to the demands of its own labour market, to national priorities such as the National Digital Economy Policy and Strategy 2020–2030 (Federal Ministry of Communications and Digital Economy [FMCDE], 2019), and to emerging areas of the discipline.

The difficulty lies in how that 30% is decided. In practice, curriculum committees rely largely on the experience and opinions of their members, supplemented by occasional consultation with industry. Such judgement is valuable, but it is difficult to verify, hard to repeat and easily influenced by the particular backgrounds of the people in the room. Studies of Nigerian graduates have repeatedly reported a gap between the skills employers want and the skills graduates bring, with information technology among the

weakest areas (Pitan & Adedeji, 2012; Okolie et al., 2020). Evidence about what employers want already exists in large volumes, in the form of job adverts posted daily on Nigerian recruitment websites, and in policy documents that set national direction. What is missing is a practical way for a curriculum committee to turn that text into evidence it can use.

Natural Language Processing (NLP), the branch of artificial intelligence concerned with enabling computers to process human language, offers such a way. Recent advances, in particular transformer-based language models (Vaswani et al., 2017; Devlin et al., 2019) and the sentence embeddings derived from them (Reimers & Gurevych, 2019), make it possible to represent the meaning of a passage of text as a vector of numbers and to compare passages by meaning rather than by exact words. Combined with techniques for extracting named skills from text and for discovering the recurring topics in a collection of documents (Grootendorst, 2022), these methods can read hundreds of job adverts and policy pages, identify the skill areas they emphasise, and check each area against what the NUC core already teaches.

This project applies those techniques to the specific problem of the CCMAS 30%. It develops SkillBridge, a web-based curriculum recommendation system that ingests job adverts and policy documents, discovers the skill topics they emphasise, compares each topic with every course in the NUC CCMAS core by meaning, ranks the topics with a transparent score, and presents them, with their supporting evidence, to curriculum planners who decide what to accept and design the resulting courses. The system does not change any curriculum by itself; it provides evidence and a structured workflow, while the decisions remain with people.

1.2 Statement of the Problem

The CCMAS gives each Nigerian university the responsibility of designing 30% of its programmes, but it provides no method or tool for deciding what that 30% should contain. In the absence of such a method, the following problems arise:

- Decisions about the 30% are made mainly on the basis of committee members' opinions and experience. The evidence behind a proposed course is rarely recorded, so the decision cannot easily be checked, defended or repeated when the curriculum is next reviewed.
- Employer requirements are available in large volumes of unstructured text, such as job adverts on sites like MyJobMag, but reading and summarising hundreds of adverts by hand is slow, subjective and impractical for a committee.

- A topic that appears to be new may already be covered by one of the 70% core courses under a different name. Detecting this requires comparing the meaning of the proposed topic with the content of well over one hundred CCMAS courses, which is tedious and error-prone when done manually.
- There is no structured link between an identified need, the decision taken on it and the course eventually designed to meet it, and no convenient way to produce a report that documents this trail for Senate or for accreditation.

Consequently, the university-designed 30% risks duplicating the core, missing important skill areas, or reflecting the preferences of individuals rather than the needs of employers and national policy. This contributes to the skills gap that has been reported among Nigerian graduates (Pitan & Adedeji, 2012; McCowan et al., 2016; Okolie et al., 2020).

1.3 Aim and Objectives of the Study

The aim of this study is to develop an NLP-driven curriculum recommendation system that supports the design of the 30% university-determined component of the NUC CCMAS by providing evidence drawn from job adverts and policy documents.

The specific objectives are to:

1. Assemble a corpus of Nigerian computing job adverts and a national policy document, and build a pipeline that validates, parses, cleans, segments and semantically encodes such documents.
2. Extract in-demand skills from the corpus using rule-based named entity recognition, and discover the main skill topics using transformer-based topic modelling.
3. Measure the semantic overlap between each discovered topic and the courses of the NUC CCMAS core, and rank the topics with a transparent score that combines employer demand, topic prevalence and novelty.
4. Implement a secure, role-based web application through which curriculum planners can inspect the evidence behind each suggested topic, record decisions, design proposed courses within the 30% allowance and generate reports.
5. Evaluate the system in terms of topic quality against a baseline, repeatability, processing speed, software correctness and, with human participants, skill-extraction accuracy, overlap detection and usability.

1.4 Significance of the Study

The study is significant to several groups:

- **Curriculum committees and Heads of Department** gain a tool that turns hundreds of documents into a ranked, evidence-backed list of candidate topics within minutes, together with a record of the decisions taken and a report suitable for Senate or accreditation panels.
- **Students and graduates** benefit indirectly from a 30% component that reflects the skills employers are actually advertising, which may improve their employability.
- **Employers and industry** gain a channel through which their stated requirements, as expressed in their own job adverts, can influence what universities teach.
- **The National Universities Commission and policy makers** gain a demonstration of how the flexibility built into the CCMAS can be used in a systematic, auditable way, and of how national policy documents can be brought into curriculum decisions.
- **Researchers** gain a documented, reproducible pipeline and evaluation design for curriculum analytics in a Nigerian context, an area in which, in the sources reviewed for this study, no prior work text-mining Nigerian job adverts for curriculum design was found.

1.5 Scope of the Study

The study covers the design, implementation and evaluation of a web-based system with the following boundaries:

- **Discipline:** Computer Science and the closely related Computing programmes covered by the NUC CCMAS for Computing, with the Department of Computer Science, University of Uyo, as the intended user.
- **Documents:** English-language job adverts, policy documents, institutional documents and academic documents in PDF, DOCX or plain-text format. The corpus used for the results reported here consists of 55 computing job adverts collected between July and September 2026 and the National Digital Economy Policy and Strategy 2020–2030, compared against the CCMAS for Computing.
- **Techniques:** pre-trained models only (spaCy, Sentence-BERT and BERTopic); no language model is trained or fine-tuned in this study.
- **Platform:** a web application accessed through a modern browser, with three user roles: Administrator, Curriculum Planner and Viewer.
- **Decision-making:** the system recommends; it does not approve or modify any curriculum. All decisions are taken and recorded by authorised users.

1.6 Limitations of the Study

The following limitations apply:

- The corpus is modest in size and dominated by job adverts from a single website, MyJobMag, and a single policy document. No institutional or academic documents were included in the final analysis, so the results reflect employer demand more strongly than academic or institutional priorities.
- Job adverts were collected over a short period (mainly September 2026), so the results describe the labour market at one point in time rather than trends over years.
- Scanned PDF documents are not supported because the system does not perform optical character recognition.
- The threshold used to flag a topic as possibly already covered by the NUC core (80% similarity) is a starting value; calibrating it requires human-labelled topic–course pairs, which are part of the pending evaluation.
- Evaluation with human participants (skill annotation, overlap labelling and usability testing) had not been completed at the time of writing; the corresponding results are marked as pending in Chapter Four.
- The system was run and measured on a single laptop with a two-core processor and no GPU; performance under many simultaneous users was not tested.

1.7 Definition of Terms

- **CCMAS (Core Curriculum and Minimum Academic Standards):** the NUC document that specifies the national core (about 70%) of each programme in Nigerian universities and leaves about 30% to each university.
- **30% component:** the portion of a programme that each university designs for itself under the CCMAS.
- **Natural Language Processing (NLP):** the field of computing concerned with enabling computers to analyse and generate human language.
- **Corpus:** a collection of documents gathered for analysis; here, job adverts and a policy document.
- **Extract (passage):** a short piece of text of usually three to five sentences and up to 200 words into which each document is divided for analysis.
- **Embedding:** a list of numbers (here, 384 numbers) that represents the meaning of a piece of text, so that texts with similar meanings have similar embeddings.

- **Cosine similarity:** a measure of how similar two embeddings are, ranging from −1 to 1, where values near 1 indicate very similar meaning.
- **Named Entity Recognition (NER):** the task of finding and labelling named items in text; here, skills, tools, programming languages and certifications.
- **Topic (theme):** a group of extracts that discuss the same subject, discovered automatically by topic modelling.
- **Topic modelling:** a family of methods that discover the main subjects in a collection of documents without being told what they are.
- **Suggested topic (recommendation):** one of the top-ranked topics presented to curriculum planners as a candidate for the 30% component.
- **Employer demand, how often it comes up, how new it is:** the three parts of each topic's score, referring respectively to skill demand, topic prevalence and novelty relative to the NUC core.
- **Curriculum Planner:** a user who reviews suggested topics, records decisions and designs courses.

1.8 Organisation of the Report

This report is organised in five chapters. Chapter One has introduced the background, problem, aim and objectives, significance, scope and limitations of the study. Chapter Two reviews the literature on the Nigerian curriculum framework, graduate employability, the NLP techniques used, and related works that analyse job adverts to inform curricula, and identifies the gap this study addresses. Chapter Three presents the methodology, the analysis of the existing process, the requirements, and the design of the proposed system, including its architecture, processing pipeline, scoring model, database and user interface. Chapter Four describes the implementation, testing and results, including the evaluation of the system. Chapter Five summarises the work, draws conclusions, states the contributions of the study and makes recommendations for practice and further work.

CHAPTER TWO
LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature that underpins the study. It begins with the Nigerian context: the curriculum framework within which the 30% component exists, the evidence on graduate employability and the skills gap, and the national digital economy policy that forms part of the study's corpus. It then explains, in simple terms, the natural language processing ideas on which the system is built: preparing text, finding important words, recognising skills, representing meaning with numbers, and discovering topics. It also covers recommender systems and the measures used to evaluate the system. The chapter closes with a review of related works that have used job adverts to analyse or inform curricula, a summary table of those works, and a statement of the gap that this study addresses.

2.2 Conceptual Review

2.2.1 Curriculum Development in Nigerian Universities

The National Universities Commission (NUC) is the statutory body responsible for setting and monitoring academic standards in Nigerian universities. Its principal instrument for this was historically the Benchmark Minimum Academic Standards (BMAS). In 2016 the Commission set up a group of experts to develop a BMAS for Computing, and from 2018 it began restructuring the BMAS across disciplines into a new framework, the Core Curriculum and Minimum Academic Standards (CCMAS) (NUC, 2022; Rasheed, n.d.). According to the Executive Secretary's preface, the CCMAS was introduced to reflect twenty-first-century realities and to provide a "measurable benchmark of knowledge, 21st century skills and competences" (Rasheed, n.d.).

The CCMAS for Computing covers seven Bachelor of Science programmes: Computer Science, Cybersecurity, Data Science, Information and Communication Technology, Information Systems, Information Technology and Software Engineering (NUC, 2022). The Federal Government confirmed that implementation of the CCMAS would begin in the 2023/2024 session, following a review process that lasted more than four years (NUC, 2023).

The most important structural feature of the CCMAS for this study is the split between national and local content. The CCMAS provides about 70% of the courses in each programme as a national core, "while allowing universities to utilise the remaining 30%

for other innovative courses" (NUC, 2022). The 70% is written by NUC-appointed experts, while the 30% is determined by each university's Senate (NUC, 2023). Ogunode et al. (2024) describe the same 70/30 structure across seventeen disciplines and argue that artificial intelligence tools can support the implementation of the CCMAS. Studies of CCMAS delivery have so far concentrated on the core, for example on platforms for teaching CCMAS content online (James et al., 2026), rather than on how the university-designed 30% should be chosen. The framework therefore creates a responsibility, the design of the 30%, without prescribing how that responsibility should be discharged.

2.2.2 Graduate Employability and the Skills Gap in Nigeria

A recurring concern in Nigerian higher education is the mismatch between the skills of graduates and the needs of employers. In a survey of 600 managers in 300 organisations across the six geopolitical zones, Pitan and Adedeji (2012) found a skills mismatch of 60.6% and a negative correlation between the skills demanded and those supplied, with information technology, communication and critical thinking among the weakest areas. Omoniwa and Adedapo (2017) reported that although most employers in their Ibadan sample considered graduates employable, over a third stated that the candidates they interviewed lacked the required skills. A British Council study of Ghana, Kenya, Nigeria and South Africa concluded that more than half of graduates in the region were inadequately prepared for employment (McCowan et al., 2016).

The reasons offered for this gap point directly at the curriculum. Okolie et al. (2020), in interviews with students, industry executives and professors in Nigeria, identified lecturers without industry experience, lecture-heavy teaching and curricula poorly aligned to the labour market as barriers to employability. An earlier Nigerian discussion paper attributed the gap between Computer Science curricula and industrial IT requirements to static curricula and weak links with industry (Ayofe & Ajetola, 2009). Pitan and Muller (2023), surveying 865 students in Nigerian and South African universities, found gaps in students' exposure to experiential learning and career guidance oriented towards the changing workplace. More recently, Onifade et al. (2026) assessed the relevance of the undergraduate Computer Science curriculum in South-West Nigeria from the perspective of graduates and their managers; managers rated graduates above average but recommended the addition of project work, self-directed learning and AI ethics. That study, like most in this area, is based on perception surveys rather than on analysis of labour-market data.

Headline unemployment statistics should be interpreted with care. Under the revised International Labour Organization methodology adopted by the National Bureau of

Statistics, the headline unemployment rate in the second quarter of 2024 was 4.3%, with 4.8% among people with post-secondary education and a labour underutilisation rate of 13.0% (National Bureau of Statistics [NBS], 2024). These figures are much lower than those produced by earlier methods, but the employer studies above show that the quality of the match between graduates and jobs, rather than the headline rate alone, remains a concern.

2.2.3 The National Digital Economy Policy and Strategy

The National Digital Economy Policy and Strategy (NDEPS) 2020–2030 was unveiled in November 2019 by the Federal Ministry of Communications and Digital Economy (FMCDE, 2019). It is organised around eight pillars, the second of which, Digital Literacy and Skills, sets a target of 95% digital literacy within ten years and calls for digital skills to be integrated into curricula at all levels. A critical review of the policy describes it as largely aspirational, with weak dissemination and governance, and reports that over 60% of graduates lack basic digital skills (Chime, 2022). The NDEPS is relevant to this study in two ways: it states a national expectation that curricula should build digital skills, and it is itself one of the documents analysed by the system, representing the policy perspective alongside the employer perspective of job adverts.

2.2.4 Natural Language Processing and Text Preprocessing

Natural Language Processing (NLP) is the part of computing that deals with teaching computers to work with human language, such as English (Jurafsky & Martin, 2026). A computer cannot understand a page of text the way a person does, so most NLP systems first prepare the text in a series of steps known as preprocessing:

- **Extracting and cleaning the text:** taking the text out of a file such as a PDF and removing things that are not real content, such as page numbers, repeated headers and web addresses.
- **Tokenisation and sentence splitting:** dividing the text into words (called tokens) and into sentences.
- **Lemmatisation:** reducing each word to its dictionary form, so that "managing", "managed" and "manages" are all treated as "manage".
- **Stop-word removal:** ignoring very common words such as "the" and "and", as well as words that are common in a particular kind of document but say little, such as "apply" or "candidate" in job adverts.

The spaCy library provides ready-made, well-tested tools for these steps, together with pre-trained models of English (Honnibal et al., 2020). The right preparation depends on the method that will use the text. Modern meaning-based models (Section 2.2.7) work

best on natural, complete sentences, while word-counting methods such as TF-IDF (Section 2.2.5) work best on simplified text with stop words removed. For this reason the system described in Chapter Three keeps two versions of every extract: a lightly cleaned version for meaning, and a simplified version for counting words.

2.2.5 Finding Important Words with TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) is a long-established way of measuring how important a word is to a document within a collection (Salton & Buckley, 1988). The idea is simple. A word is important to a document if it appears often in that document (term frequency) but rarely in the other documents (inverse document frequency). A word like "the" appears everywhere, so it scores low. A word like "Kubernetes" appears often in a few adverts and nowhere else, so it scores high in those adverts. In one common form:

$$tfidf(t, d) = tf(t, d) \times log(N / df(t)) \quad (2.1)$$

where $tf(t, d)$ is how often word t appears in document d, N is the number of documents, and $df(t)$ is the number of documents that contain t. For example, in a collection of 100 documents, a word that appears 5 times in one document but in only 2 documents overall gets a much higher weight than a word that appears 5 times but in 90 documents, because log(100/2) is much larger than log(100/90). Software libraries such as scikit-learn provide ready implementations with small refinements (Pedregosa et al., 2011). TF-IDF is easy to understand and explain, and a variant of it is used by BERTopic to choose the words that describe each topic (Grootendorst, 2022).

2.2.6 Named Entity Recognition and Skill Extraction

Named Entity Recognition (NER) means finding and labelling names of things in text, such as people, organisations and places (Jurafsky & Martin, 2026). In job adverts, the "things" of interest are skills, tools, programming languages and certifications, for example "Python", "Microsoft Azure" or "CompTIA Security+". There are two main ways to find them:

- **Learning from examples.** A model is trained on many sentences in which people have already marked the skills. Zhang et al. (2022), for example, created SkillSpan, a collection of 14,500 sentences from English job adverts with 12,500 skills marked by hand, and showed that models trained on job-advert text find skills better than general-purpose models.
- **Matching against a list.** The system is given a carefully prepared list of skill names and patterns, and finds every place where they appear. This is called a rule-based approach.

A survey by Senger et al. (2024) describes the many methods and datasets in use, and notes that there is still no agreed standard list of skills. For this study, the rule-based approach was chosen because no marked-up collection of Nigerian job adverts exists to learn from. Its other advantages are that it is transparent (it is always clear why a skill was found), new skills can be added simply by adding patterns, and every match is linked to one standard skill name. Vogt et al. (2023) took the same route, combining an ontology with a hand-checked list of 206 terms to study data science skills in more than 5,000 job adverts, and confirmed the results with a panel of experts. The system in this study uses spaCy's EntityRuler, a component that finds matches for a list of patterns and gives each match a label and a standard name (Honnibal et al., 2020).

2.2.7 Representing Meaning with Embeddings

To compare texts by meaning rather than by exact words, NLP systems turn each piece of text into an embedding: a list of numbers chosen so that texts with similar meanings get similar numbers. A helpful way to picture this is as a map: each text is a point on the map, and texts about the same subject sit close together, even if they use different words. "Manage cloud servers" and "administer AWS infrastructure" share almost no words, but their embeddings are close.

Early methods such as word2vec learned embeddings for single words from large amounts of text (Mikolov et al., 2013). A major step forward came with the transformer, a type of neural network that looks at how every word in a sentence relates to every other word (Vaswani et al., 2017). BERT, a transformer trained on very large amounts of text, learns the meaning of a word from the words on both sides of it (Devlin et al., 2019).

BERT on its own is slow for comparing many sentences, because it must process every pair of sentences together. Sentence-BERT (SBERT) solves this by producing one embedding for each sentence or short passage, which can then be compared quickly with any other (Reimers & Gurevych, 2019). This study uses all-MiniLM-L6-v2, a small and fast SBERT model based on MiniLM, a method for shrinking large transformer models while keeping most of their ability (Wang et al., 2020). It turns each extract into 384 numbers and runs well on an ordinary laptop without a graphics card. SBERT has already been used successfully on job texts, for example to match job seekers with vacancies (Lavi et al., 2021).

The similarity between two embeddings a and b is usually measured with cosine similarity:

$$\cos(a, b) = (a \cdot b) / (\|a\| \times \|b\|) \quad (2.2)$$

In simple terms, cosine similarity measures the angle between two arrows. A value near 1 means the two texts mean almost the same thing; a value near 0 means they are unrelated. In this study, a value of 0.80 is treated as "very similar". When embeddings are scaled to a length of 1, as they are here, cosine similarity is simply the sum of the products of their matching numbers.

2.2.8 Discovering Topics

Topic modelling means finding the main subjects in a collection of documents automatically, without being told in advance what they are. The best-known older method is Latent Dirichlet Allocation (LDA). LDA assumes that each document is a mixture of topics, and that each topic is a set of words that tend to appear together (Blei et al., 2003). Because LDA works only with word counts, it does not understand that different words can mean the same thing, and it performs poorly on short texts such as job adverts.

BERTopic is a newer method that works with meaning (Grootendorst, 2022). It follows four steps:

1. **Embed:** each extract is turned into an embedding with a sentence model such as SBERT.
2. **Simplify:** UMAP (Uniform Manifold Approximation and Projection) reduces each embedding from hundreds of numbers to a handful, while keeping texts that were close still close. This makes the next step faster and more reliable (McInnes et al., 2018).
3. **Group:** HDBSCAN finds groups of extracts that sit closely together. Unlike older methods, it does not need to be told how many groups to find, and it leaves out extracts that do not clearly belong to any group instead of forcing them into one (Campello et al., 2013; McInnes et al., 2017).
4. **Describe:** a version of TF-IDF, called class-based TF-IDF, finds the words that best distinguish each group from the others, and these words describe the topic (Grootendorst, 2022).

To judge how good the topics are, researchers use coherence scores, which check whether the main words of each topic really tend to appear together in the documents. Röder et al. (2015) compared many coherence measures and found that one called C_v agreed best with human judgement. Another common measure is UMass. A higher C_v, and a UMass value closer to zero, mean more coherent topics.

2.2.9 Recommender Systems

A recommender system suggests items that are likely to be useful to a person (Ricci et al., 2022). Familiar examples suggest films, products or songs. Some recommend items liked by similar people (collaborative filtering); others recommend items similar to those a person liked before (content-based); others reason from stated needs and expert knowledge (knowledge-based).

Designing a curriculum is different from recommending films. There are no ratings from thousands of users, and the "items" (possible new topics) do not exist in a catalogue; they must be discovered from evidence. The system in this study is therefore best described as a decision-support recommender. It discovers candidate topics from documents, ranks them with a score whose parts and weights are visible and can be changed, and leaves the final decision to a person, whose decision is recorded. This is different from course recommenders for students, such as the one proposed by Soni et al. (2025), which suggest existing courses to individual learners rather than helping a department design new ones.

2.2.10 Measures for Evaluation

Precision, recall and F1. When a system finds items, such as skills in a text, three standard measures describe how well it does:

- **Precision** answers "of the items the system found, how many were correct?"
- **Recall** answers "of all the correct items, how many did the system find?"
- **F1** combines the two into one number (their harmonic mean).

For example, if a text contains 10 real skills and the system finds 8 items of which 6 are real skills, precision is 6/8 = 0.75 and recall is 6/10 = 0.60.

Cohen's kappa. When people mark the correct answers by hand, their agreement with each other is measured with Cohen's kappa, which adjusts for the agreement that would happen by chance (Cohen, 1960). Landis and Koch (1977) proposed widely used bands for reading kappa, from "slight" to "almost perfect" agreement.

System Usability Scale (SUS). Usability is often measured with the SUS, a short questionnaire of ten statements that gives a score from 0 to 100 (Brooke, 1996). Bangor et al. (2009) linked SUS scores to everyday words: an average score of about 71 corresponds to "good" and about 86 to "excellent", and the overall average across many studies is about 70. A score of 70 or more is therefore usually taken as acceptable.

2.3 Review of Related Works

A growing body of work analyses job adverts to understand the skills demanded in computing and related fields, and some of it compares those demands with curricula.

Early and descriptive text mining of job adverts. Debortoli et al. (2014) applied latent semantic analysis to job adverts from monster.com to compare business intelligence and big data skills, finding that business knowledge was as important as technical skills. De Mauro et al. (2018) combined text mining of more than 2,700 job postings with expert judgement to classify big data professions into four job families and nine skill groups. Pejic-Bach et al. (2020) mined LinkedIn adverts for Industry 4.0 roles using phrase and topic extraction and identified two job-profile clusters. Gurcan and Cagiltay (2019) used LDA on job adverts to map the knowledge domains and skill sets of big data software engineering, proposing the resulting map as a guide for curriculum updates. These studies established that job adverts are a rich source of skill information, but they were descriptive: none compared the discovered skills with the actual courses of a curriculum.

Neural topic modelling of job adverts. More recent studies use BERTopic. Gurcan et al. (2025) applied n-gram analysis and BERTopic to 3,156 big data analytics postings from 2024 and identified 26 skill topics grouped into analytical, technical, developer and soft skills. Li et al. (2025) combined BERTopic with the TOPSIS multi-criteria method to analyse 5,349 postings and 63 big data programmes in China, ranking skills by frequency and salary; they found that 52% of programmes claimed to develop data-application ability but only 11% stated concrete graduation requirements for it. This is the closest published method to the multi-criteria scoring used in the present study, although it uses salary as a proxy for importance and has no step in which people accept or reject the results.

Comparing curricula with job adverts. Several studies compare what is taught with what is sought. Kauttonen et al. (2024) analysed 18,254 ICT thesis abstracts and 107,335 Finnish job adverts from 2010 to 2022 using subject indexing, TF-IDF and clustering, found 16 overlapping clusters, and showed that job-advert topics led thesis topics by up to four years. Li and Jia (2024) used LDA on 10,050 logistics postings and the syllabi of 29 Chinese universities, finding that some content areas were over-supplied and others under-supplied. Ahadi et al. (2022) compared 2,747 university subjects with 144,000 job adverts using skills tagged by a commercial parser, and Yaprak et al. (2025) compared 66 course descriptions with 107 adverts in communication programmes using TF-IDF and LDA. Ojan et al. (2026) mapped communication curricula and 2.7 million Spanish postings to the European ESCO skills taxonomy and found that 28–52% of the demanded skills had no clear place in the

curriculum. Turaev et al. (2026), in a preprint, used a large language model to extract competencies from 1,922 learning outcomes of five computing programmes and from over 5,000 jobs, reporting that the shared core covered about one third of demand.

Embedding-based course–job alignment. Kverne et al. (2025) embedded 210 computing courses and 95,376 job postings with several sentence-embedding models, including SBERT, and measured course-to-job cosine similarity, finding that core courses aligned better with jobs than electives. This is methodologically close to the overlap measurement in the present study, but it ranks existing courses rather than discovering new topics that a curriculum lacks.

African studies. The closest African work is that of Nyale et al. (2026), who scraped 46,514 job postings in Nairobi, Kenya, extracted 9,077 skills and built a decision-tree-based curriculum alignment model with a Digital Skills Index, identifying gaps in cloud computing, artificial intelligence and security. Samwi et al. (2026) applied association rule mining to 15,653 ICT job adverts from several portals to uncover co-occurring skill sets, finding, for example, that Java and Python each appeared in 18.18% of adverts. Neither study discovers topics with transformer-based models, incorporates national policy documents or maps findings to a national core curriculum. In the sources reviewed for this study, no peer-reviewed work was found that text-mines Nigerian job adverts to inform the design of the CCMAS 30% component.

Framing studies. Aljohani et al. (2022) conducted a bibliometric analysis of 10,214 papers on the gap between university curricula and job-market requirements, confirming the breadth of interest in data-driven approaches to the problem. Senger et al. (2024) and Zhang et al. (2022), discussed in Section 2.2.6, provide the methodological background for skill extraction from job postings.

2.4 Summary of Related Works

Table 2.1 summarises the most relevant related works, their data, methods, main findings and the limitation most relevant to this study.

Table 2.1: Summary of related works

Author(s) and year	Data	Method	Key finding	Limitation relative to this study
Debortoli et al. (2014)	monster.com job adverts	Latent semantic analysis	Business knowledge as important as technical skills	Descriptive; no curriculum comparison
De Mauro et al. (2018)	2,700+ big data postings	Text mining and expert judgement	Four job families, nine skill groups	No mapping to courses

Author(s) and year	Data	Method	Key finding	Limitation relative to this study
Gurcan & Cagiltay (2019)	Big data job adverts	LDA topic modelling	Competency map for curriculum updates	LDA only; no course mapping
Pejic-Bach et al. (2020)	LinkedIn Industry 4.0 adverts	Phrase and topic extraction	Two job-profile clusters	Dictionary-based; no curriculum link
Ahadi et al. (2022)	2,747 subjects; 144,000 adverts	Commercial skill parser, comparative advantage	Gaps between skills taught and sought	Proprietary parser; poster paper
Vogt et al. (2023)	5,000+ Jooble adverts	Ontology NER with whitelist, expert validation	Data science competency profile	Snapshot; no course mapping
Kauttonen et al. (2024)	18,254 theses; 107,335 Finnish adverts	TF-IDF, SVD, K-means, lagged correlation	Adverts lead theses by up to 4 years	Retrospective; no recommendations
Li & Jia (2024)	10,050 postings; 29 syllabi	LDA	Over- and under-supplied content areas	Bag-of-words; no semantic overlap
Gurcan et al. (2025)	3,156 Indeed postings	N-grams and BERTopic	26 skill topics	No curriculum comparison
Li et al. (2025)	5,349 postings; 63 programmes	BERTopic and TOPSIS	Stated abilities rarely backed by requirements	Salary as proxy; no human decision step
Kverne et al. (2025)	210 courses; 95,376 postings	SBERT and other embeddings, cosine similarity	Core courses align better than electives	Ranks existing courses; no new topics
Nyale et al. (2026)	46,514 Nairobi postings	Scraping, NLP, decision tree	Gaps in cloud, AI, security	No topic discovery; no policy corpus
Samwi et al. (2026)	15,653 ICT adverts	Association rule mining	Co-occurring skill sets	No curriculum mapping
Ojan et al. (2026)	2.7 million Spanish postings	ESCO taxonomy mapping	28–52% of demanded skills not covered	Taxonomy-bound; no emerging topics
Onifade et al. (2026)	Nigerian CS graduates and managers	Mixed-methods survey	Graduates rated above average	Perception-based, not data-driven

2.5 Research Gap

The review shows that job adverts have been used extensively to describe skill demand and, increasingly, to compare that demand with curricula, using methods that have progressed from word counts and LDA to sentence embeddings and BERTopic. However, the following gaps remain, and this study addresses them:

1. **No Nigerian, CCMAS-specific work.** No study found in this review text-mines Nigerian job adverts, or uses Nigerian policy documents, to inform the design of

the CCMAS 30% component. Nigerian studies of the skills gap rely on surveys of perception rather than analysis of labour-market text.

2. **Description rather than decision support.** Most studies stop at describing skill demand or measuring alignment. Few generate candidate topics, check them against every course of a national core by meaning, rank them transparently, and support the subsequent human decision.
3. **No human-in-the-loop workflow or audit trail.** None of the reviewed systems records which suggestions were accepted or rejected, links accepted topics to designed courses within a credit allowance, and produces a report documenting the evidence for each decision.
4. **Limited transparency of scoring.** Where ranking is used, the criteria are often fixed or implicit. Curriculum committees need to see why a topic is ranked highly and to adjust the weight given to each criterion.

This study addresses these gaps by developing a system that combines rule-based skill extraction, SBERT embeddings and BERTopic topic discovery on a Nigerian corpus, measures each topic's semantic overlap with the 86 subject courses of the NUC CCMAS for Computing, ranks topics with an explicit and adjustable three-part score, and embeds the results in a role-based workflow for review, course design and reporting.

2.6 Summary

This chapter has reviewed the Nigerian curriculum framework, which leaves 30% of each programme to universities without providing a method for designing it, and the evidence of a persistent skills gap among Nigerian graduates. It has explained the NLP techniques relevant to the study, namely text preprocessing, TF-IDF, rule-based named entity recognition, sentence embeddings, BERTopic and coherence measures, together with recommender systems and evaluation measures. The review of related works showed rapid methodological progress internationally but no work applying these methods to Nigerian job adverts and the CCMAS, and little support for the human decisions that follow the analysis. The next chapter describes the methodology and design of the system developed to fill this gap.

CHAPTER THREE
METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

This chapter explains how the study was carried out and how the system was designed. It first describes the method used to build the software. It then looks at how the 30% component is decided today, and why that process needs support. After that, it introduces the proposed system, lists what the system must do, and describes the data it uses. The rest of the chapter explains the design in detail: the overall architecture, the steps the system follows to turn documents into suggestions, the way each suggestion is scored, the diagrams that describe how people use the system, the database, security, the user interface and the plan for evaluating the system.

3.2 Research and Development Methodology

3.2.1 Choice of Methodology

This is a design-and-build study: the main output is a working software system, which is then tested against the objectives stated in Chapter One. To build the system, an incremental (phase-by-phase) methodology was used. In incremental development, the system is built in a series of small, complete parts called increments. Each increment adds new features to what already works, and each is planned, built and tested before the next one begins (Sommerville, 2016). The approach follows the ideas of the Agile Manifesto, which values working software, frequent delivery and readiness to change (Beck et al., 2001).

This method suited the project for three reasons:

- **The NLP parts were uncertain.** It was not possible to know in advance whether the topic model would find stable, meaningful topics in a small, real set of Nigerian documents. The only way to find out was to build it and test it early.
- **Later parts depended on earlier ones.** For example, the analysis could not be built until document upload and user accounts worked. Building in order meant each part could be tested on its own before the next was added.
- **Real results led to improvements.** Running the system on real documents revealed problems that a fixed, up-front design would have missed. For example, one long policy document was drowning out the job adverts, so the scoring was changed to count every document equally; and lecturers found keyword titles such as "project, english, problem" confusing, so course-style names were added.

3.2.2 Development Phases

The work was organised into seven phases. Each phase was built on its own branch of a Git repository (a separate working copy of the code) and checked with automated tests. Phases 0 to 5 were merged into the main development branch when they were complete; phase 6, the final release work, was still on its own branch at the time of writing. Within every phase the same cycle was followed: plan, design, implement, test and review. Figure 3.1 shows the phases and this cycle, and Table 3.1 lists what each phase delivered.

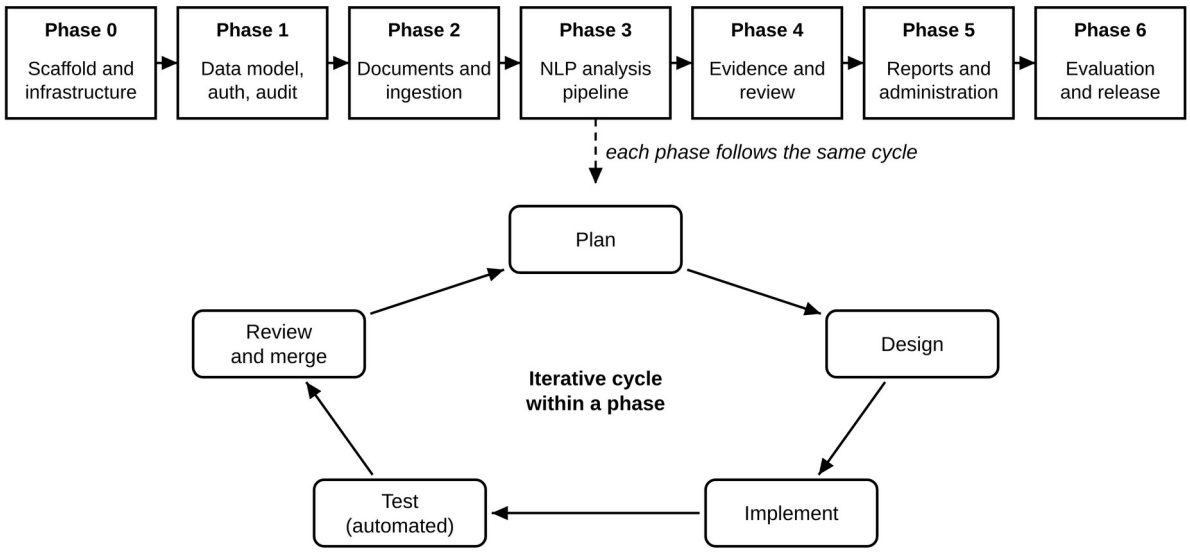


Figure 3.1: Development phases and the cycle followed within each phase

Table 3.1: Development phases

Phase	Focus	Main deliverables
0	Scaffold and infrastructure	Basic Flask server, health check, background worker, React shell, Docker set-up (PostgreSQL with pgvector, Redis, Nginx)
1	Data model, sign-in, roles and audit	Database tables, starter data, secure sign-in, role-based access control, activity log, user management
2	Documents and ingestion	Upload checks, reading PDF/DOCX/TXT files, cleaning, extracts, SBERT embeddings, NUC core upload
3	Analysis pipeline	Keywords, skill extraction, topic discovery, NUC comparison, scoring, background analysis with progress
4	Evidence and review	Evidence pages, decisions on topics, course design, proposed curriculum
5	Reports, dashboard and administration	Word and PDF reports, dashboard, settings, skill patterns, stop words, activity log

Phase	Focus	Main deliverables
6	Evaluation, real data and release	Tuning on real documents, advert collector, evaluation scripts, topic merging and naming, plain-language wording, SkillBridge redesign

3.2.3 Development Tools

The back end was written in Python and the front end in TypeScript. Git was used for version control, Docker Desktop to run the database and job queue, and PowerShell on Windows 11 to run commands. An AI coding assistant (Claude Code, by Anthropic) was used under the author's direction to generate and revise code, tests and documentation. All requirements, design decisions, data collection, evaluation choices and the acceptance of each increment were made by the author, and every increment was checked with the automated tests described in Chapter Four.

3.3 Analysis of the Existing System

Today, the 30% component of a CCMAS programme is decided through the university's normal curriculum process, shown in Figure 3.2. The department sets up a curriculum committee, usually made up of senior lecturers. Members propose courses based mainly on their own knowledge, their contacts in industry and any reports they happen to know about. The proposals are discussed at the departmental board and the faculty board and are finally approved by Senate. If Senate does not approve, the proposals go back to the committee to be revised.

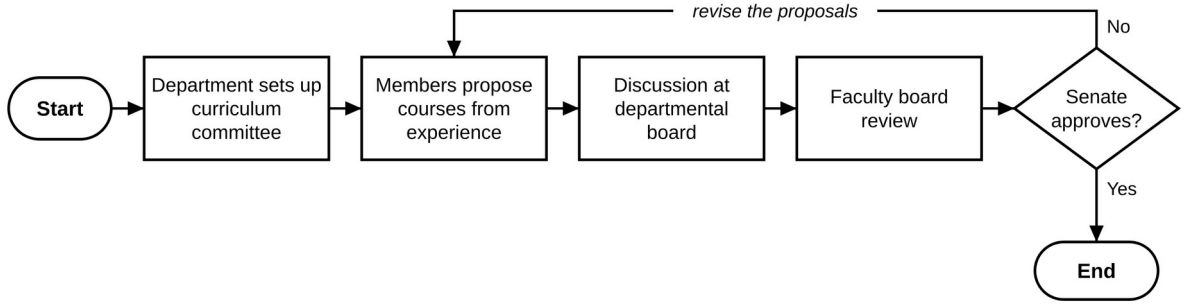


Figure 3.2: Flowchart of the existing process for deciding the 30% component

This process has four main weaknesses:

- **Little evidence.** Proposals rest mostly on opinion. No committee can read hundreds of job adverts, so what employers want is known only from anecdotes.
- **Risk of repeating the core.** To check whether a proposed course is already covered by one of the 118 courses in the CCMAS for Computing, someone would

have to search a 228-page document. Overlaps are easily missed, especially when the same subject has a different name.

- **No record of reasons.** The reasons for including or leaving out a topic are rarely written down in a way that can be checked at the next review or shown to an accreditation panel.
- **Slow and hard to repeat.** Because everything is done by hand, the process happens rarely and cannot easily be repeated when new evidence appears.

3.4 The Proposed System

The proposed system, named SkillBridge, is a web-based tool that brings evidence into the existing process. It does not replace the committee; it gives the committee better information. It works in five simple steps:

1. Authorised users upload job adverts and policy, institutional or academic documents.
2. The system breaks each document into short passages ("extracts") and records what each one means.
3. When a planner starts an analysis, the system finds the skills mentioned, groups the extracts into topics, compares each topic with every subject course in the NUC core, and ranks the topics with a clear score.
4. The planner looks at the top-ranked topics and their evidence, and accepts, rejects or defers each one. For accepted topics, the planner designs a course within the credit units available for the 30% component.
5. The planner downloads a report that records the evidence and the decisions.

Figure 3.3 shows the system as a whole, and the information that flows between it and the people and sources around it. This kind of diagram is called a context diagram, or level-0 data flow diagram: the circle is the system, the rectangles are the people and sources outside it, and each arrow is a flow of information.

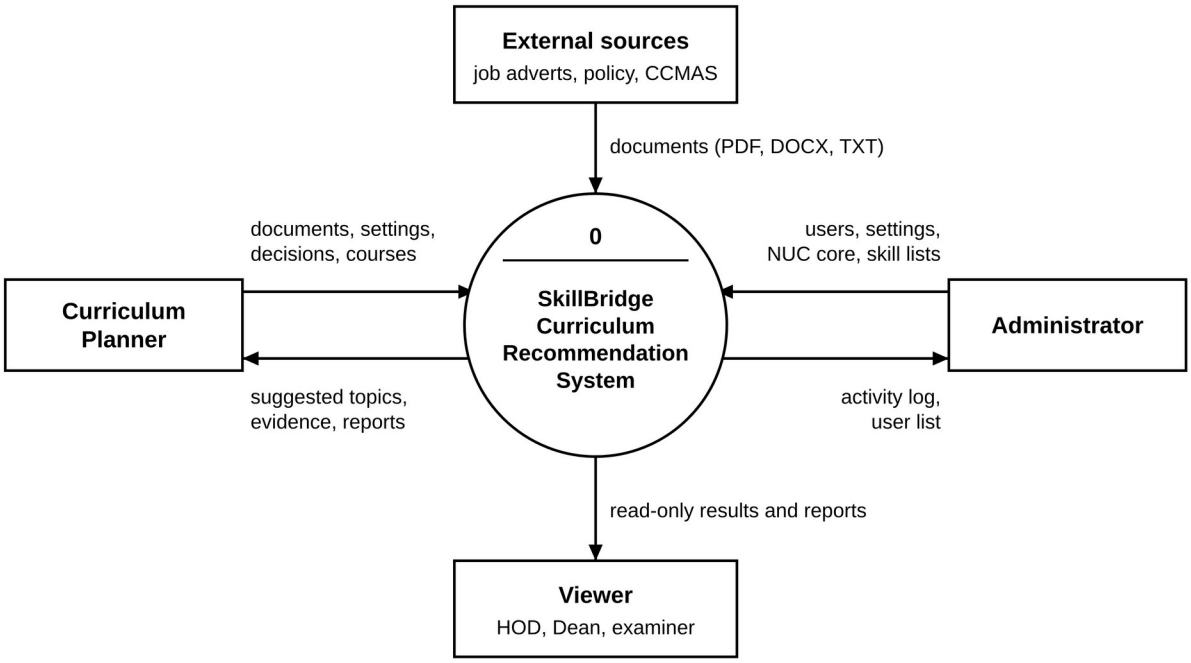


Figure 3.3: Context diagram (level-0 data flow diagram) of the proposed system

Compared with the existing process, the proposed system:

- summarises many documents quickly and in the same way every time, so the same documents and settings always give the same results;
- compares every candidate topic with every NUC core subject course by meaning, not just by title;
- shows the evidence behind every suggestion, including example extracts and where they came from;
- records every decision, course design and administrative action, with who did it and when; and
- keeps people in control, because the system only suggests and never changes the curriculum itself.

3.5 Requirements Specification

3.5.1 Functional Requirements

Functional requirements describe what the system must do. They are listed in Table 3.2.

Table 3.2: Functional requirements

ID	Requirement
FR1	Users shall be able to sign in and out, and each function shall be limited to the right role: Administrator, Curriculum Planner or Viewer.

ID	Requirement
FR2	The system shall accept PDF, DOCX and TXT files of up to 25 MB, check that each file is what it claims to be, and reject exact duplicates.
FR3	The system shall extract and clean the text of each document, divide it into extracts of three to five sentences, and store a meaning vector (embedding) for each extract.
FR4	An Administrator shall be able to upload versions of the NUC CCMAS core; the system shall find its courses and allow general courses to be left out of the comparison.
FR5	A planner shall be able to create an analysis from chosen documents, adjust the scoring weights, and run it in the background while watching its progress.
FR6	The system shall find keywords, extract skills, discover and name topics, compare each topic with the NUC core and rank the topics.
FR7	The system shall show each suggested topic with its score out of 100, the three parts of the score, keywords, a plain description, example evidence and the closest NUC course.
FR8	Planners shall be able to accept, reject or defer each topic with notes, and design a course (code, title, 1–3 credit units, prerequisites, learning outcomes) for an accepted topic.
FR9	The system shall show the proposed courses and their total credit units against the 30% allowance, and produce reports in Word (DOCX) and PDF format.
FR10	An Administrator shall be able to manage users, settings, skill patterns, stop words and the list of course-style names, and view and export the activity log.

3.5.2 Non-Functional Requirements

Non-functional requirements describe how well the system must work:

- **Security:** passwords are stored only as bcrypt hashes (scrambled forms that cannot be reversed); session cookies cannot be read by page scripts; every request that changes data must carry an anti-forgery (CSRF) token; role checks are made on the server; and uploaded files are stored under random names and never made public.
- **Performance:** an analysis of about 20 documents should finish within 60 seconds on ordinary hardware once the system is warmed up, and long tasks must run in the background so the screen does not freeze.
- **Usability:** plain language on screen, scores out of 100, technical detail only on request, and a target System Usability Scale score of at least 70.
- **Transparency and repeatability:** fixed random seeds, stored settings for every analysis, and visible evidence for every suggestion.
- **Maintainability:** separate modules, exact software versions, managed database changes, and automated tests covering at least 85% of the back-end code.

- **Portability:** the whole system can run in Docker containers; a lighter mode runs the application directly with only the database and queue in Docker.

3.6 Data Collection

3.6.1 Job Adverts and Policy Document

The final analysis (session 13) used 56 documents, summarised in Table 3.3. Fifty-four job adverts were collected from MyJobMag, a Nigerian recruitment website, using a small collection program written for the project, and one advert was copied by hand from LinkedIn. The collection program was designed to be polite to the website: it follows the site's robots.txt rules (the site's instructions to automated programs), waits three to five seconds between requests, makes no more than 150 requests per run, and never collects from Jobberman or LinkedIn. The policy document is the National Digital Economy Policy and Strategy 2020–2030 (FMCDE, 2019).

Table 3.3: Documents in the final analysis

Measure	Value
Documents	56 (55 job adverts, 1 policy document)
Sources	MyJobMag 54; NITDA 1; not recorded 1 (advert copied by hand from LinkedIn)
File types	55 TXT, 1 PDF
Publication dates recorded	7 July 2026 to 25 September 2026 (51 of the 54 dated documents from September 2026)
Total words	30,822
Extracts	417 (301 from job adverts, 116 from the policy document)
Policy document	41 pages, 10,239 words, 116 extracts

3.6.2 The NUC CCMAS Core

Every topic is compared with the CCMAS for Computing (NUC, 2022), uploaded to the system as "NUC CCMAS Computing 2023". The system read its 228 pages (74,717 words) and found 118 courses from their headings (for example, "CSC 301: … (3 Units)"). Thirty-two of these were left out of the comparison because they describe work experience or general skills rather than subject content: four general studies (GST) courses, 11 SIWES courses, 14 final year project courses and three industrial work experience courses. The other 86 subject courses were compared. This rule was added after an early test showed that SIWES and project courses, which mention many subjects in general terms, were wrongly appearing as the "closest course" for five of the top 20 topics.

3.6.3 Skill Patterns and Supporting Lists

To recognise skills, the system uses a list of 302 patterns (33 for programming languages, 112 for tools, 120 for skills and 37 for certifications) that together cover 172 different skills. It also uses a list of 116 "stop words" (words that are common in the documents but carry little meaning, and are therefore ignored) and a list of 120 course-style topic names taken from the ACM/IEEE Computer Science Curricula 2023 (Joint Task Force on Computer Science Curricula, 2024) and common NUC course titles. An Administrator can edit all three lists.

3.7 System Architecture

The system uses a three-tier architecture, which means it is split into three layers, each with its own job. A background worker is added to handle long tasks. Figure 3.4 shows the layers and how the parts talk to each other.

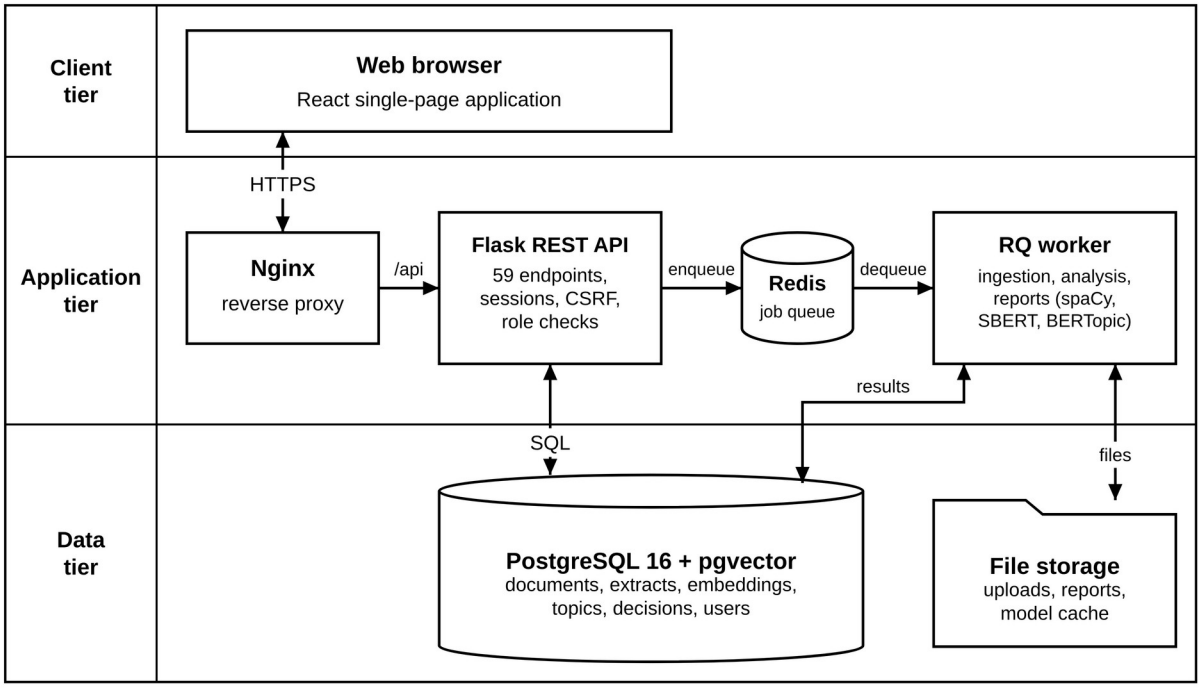


Figure 3.4: System architecture

- **Client tier (what the user sees).** The user opens SkillBridge in a web browser. The screens are a single-page application written in React and TypeScript, which means the page updates itself without reloading. It fetches data from the server and, while a job is running, checks its progress every two to three seconds.
- **Application tier (the logic).** Nginx is the single entry point: it passes every request that starts with /api to the Flask API and serves the screens for everything else. The Flask API offers 59 endpoints (addresses the screens can call). It checks who the user is and what their role allows, records important actions, and places

long tasks on a queue held in Redis. A separate RQ worker takes tasks from the queue and does the heavy work: reading documents, running analyses and producing reports.

- **Data tier (where information is kept).** PostgreSQL stores all the data, including the meaning vectors (embeddings) of every extract and NUC course; the pgvector extension lets PostgreSQL store these vectors. Uploaded files, reports and the language models are kept in file storage.

Keeping the heavy work in a separate worker means that an analysis lasting several minutes does not slow down other users, and the screen can show exactly which step the analysis has reached.

3.8 Design of the Processing Pipeline

The pipeline is the sequence of steps the system follows to turn documents into suggestions. It has three parts, shown as a flowchart in Figure 3.5: (A) document ingestion, which runs once for each uploaded document; (B) the analysis, which runs when a planner starts it; and (C) human review by the planner. The flowchart uses standard symbols: a parallelogram for input, rectangles for processing steps, a cylinder for the database, a wavy-bottomed shape for a document produced, a trapezoid for a manual (human) step, diamonds for decisions, and rounded shapes for start and end.

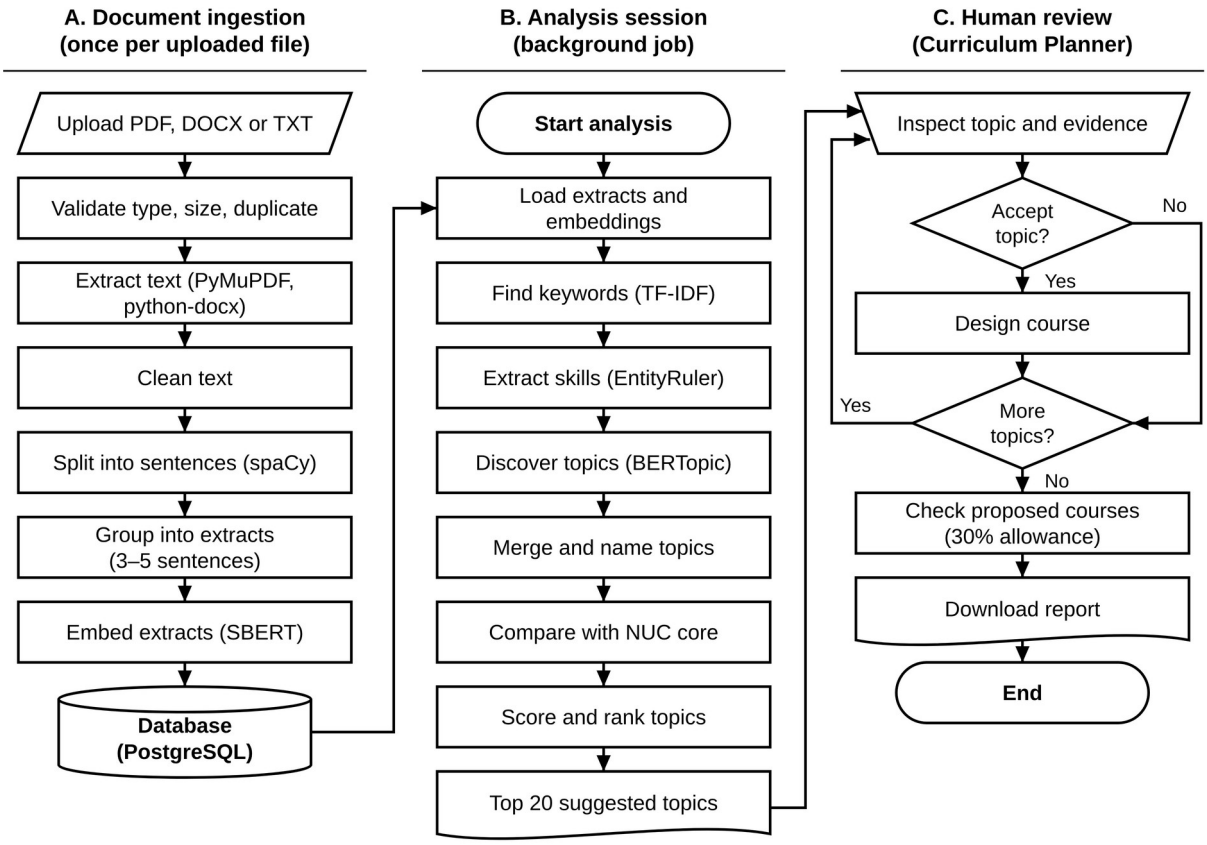


Figure 3.5: Flowchart of the processing pipeline

3.8.1 Document Ingestion

Each uploaded document goes through seven steps:

1. **Check the file.** The file must be PDF, DOCX or TXT and no larger than 25 MB. Its contents are checked to confirm it really is that type of file, and a digital fingerprint (a SHA-256 hash) is used to reject exact copies of documents already uploaded.
2. **Extract the text.** PDFs are read page by page with PyMuPDF. Password-protected PDFs, and scanned PDFs that contain pictures of text rather than real text, are rejected. Word files are read with python-docx, including their tables.
3. **Clean the text.** The system removes things that are not real content: repeated page headers and footers, page numbers, dotted lines in tables of contents, web addresses and e-mail addresses. It rejoins words split across lines and turns bullet points into sentences. Otherwise the wording is kept as it is, because the meaning model works best on natural sentences.
4. **Split into sentences.** The spaCy language model finds where each sentence starts and ends.

5. **Group into extracts.** Sentences are grouped into extracts of usually three to five sentences and up to 200 words. This size is long enough to carry a clear meaning but short enough for the meaning model, which can read only about 256 word-pieces at a time.
6. **Make a simplified copy.** A second, simplified copy of each extract is made for keyword counting: words are reduced to their dictionary form (for example, "managing" becomes "manage"), and stop words, punctuation and numbers are removed.
7. **Record the meaning.** Each extract is passed through Sentence-BERT (the all-MiniLM-L6-v2 model), which turns it into a list of 384 numbers (an embedding) that represents its meaning. Extracts with similar meanings get similar numbers. The embeddings are stored in the database.

For the NUC core document, the system also finds each course by looking for headings of the form "CODE NNN: Title (N Units)". The text of each course (up to 500 words) is cut into 200-word pieces, and the course's embedding is the average of the embeddings of its pieces.

3.8.2 Keywords and Skills

Keywords. The system uses TF-IDF (Section 2.2.5) to find the words and two-word phrases that best characterise the documents. It keeps the 30 most important terms overall and for each type of document.

Skills. The system then looks for named skills using spaCy's EntityRuler, a component that matches text against the list of skill patterns described in Section 3.6.3, ignoring capital letters. For example, "linux", "Linux" and "LINUX" are all recognised as the same skill. Only matches from the skill list are counted, so names of companies or places are ignored. For each skill, the system records how many times it is mentioned and in how many documents it appears.

3.8.3 Finding, Merging and Naming Topics

Finding topics. Topics are found with BERTopic (Section 2.2.8), which groups together extracts whose embeddings are close, in other words extracts that talk about the same thing. It works in two stages: UMAP first simplifies the 384 numbers of each embedding into 5, and HDBSCAN then finds dense groups of extracts. Extracts that do not clearly belong to any group are set aside as "outliers" rather than forced into a topic. Table 3.4 lists the settings used. The random seed is fixed at 42, so the same documents always give the same topics.

Table 3.4: Topic-finding settings

Component	Setting	What it means
UMAP	15 neighbours, 5 dimensions, cosine distance, seed 42	How the meaning vectors are simplified before grouping
HDBSCAN	smallest group 5 extracts, min_samples 3	A topic needs at least five extracts; min_samples controls how cautious the grouping is
Topic words	Top 10 one- and two-word terms	The words used to describe each topic
Outliers	Extracts in no group are set aside	Unclear extracts do not distort the topics
Merging	Topics more than 83% similar are merged	Removes near-duplicate topics
Naming	A catalogue name is used if at least 50% similar	Gives topics names lecturers recognise

The value min_samples = 3 was chosen after testing. With the default value, the number of topics changed wildly from run to run (between 2 and 23 depending on the random seed), and one topic sometimes swallowed 70% of all extracts. With min_samples = 3, every seed tested gave a steady 26 to 33 topics.

Merging topics. The centre of a topic is the average of its extracts' embeddings. Early runs sometimes found the same subject twice, so any two topics whose centres are more than 83% similar are merged, starting with the most similar pair. The eight extracts closest to each topic's centre are kept as its evidence.

Naming topics. The words that describe a topic can be confusing on their own (one early topic was described as "project, english, problem"). So each topic is compared with the list of 120 course-style names, and gets the closest name if it is at least 50% similar. The similarity looks at the topic in two ways, its centre and its main keywords, and takes the average:

$$sim(t, n) = [\ cos(\text{topic centre}, n) + cos(\text{topic keywords}, n)\ ] / 2 \qquad (3.1)$$

Here, cos( ) is the cosine similarity of Section 2.2.7. Each name can be used only once. A topic that finds no suitable name keeps a title made from its keywords. The system also writes a plain description for each topic, for example: "Why this is recommended: 16 of the 56 documents mention this area, mostly job adverts for roles such as IT Specialist … Employers ask for skills in IT Support, Information Security, Linux…".

3.8.4 Comparing Topics with the NUC Core

To decide whether a topic is already taught, the system compares the topic's centre with the embedding of each of the 86 compared NUC courses and keeps the highest similarity:

$$max_sim = highest\ cos(topic\ centre, course)\ over\ the\ 86\ courses \quad (3.2)$$

$$how\ new = 1 - max_sim \ \ (kept\ between\ 0\ and\ 1) \quad (3.3)$$

If max_sim is above 0.80 (80% similar), the topic is labelled "May already be in NUC core". Otherwise it is labelled "Not in NUC core". A topic that is very similar to an existing course therefore gets a low "how new it is" value, and a topic unlike any existing course gets a high one. The closest course is shown to the planner so that they can judge for themselves.

3.8.5 Scoring Model

Every topic receives a score out of 100 made of three parts. Each part is first turned into a value between 0 (lowest among the topics) and 1 (highest among the topics). This is called min–max scaling:

$$scaled\ value = (value - lowest) / (highest - lowest) \quad (3.4)$$

Part 1: How often it comes up. This measures how much of the evidence is about the topic. To stop one long document from dominating, every document counts equally: a document with 100 extracts gives each of its extracts a weight of 1/100, and a document with 4 extracts gives each a weight of 1/4. The topic's share of the total weight is multiplied by how firmly its extracts belong to the topic (their average HDBSCAN membership probability), and the square root is taken so that the single largest topic does not overshadow all the others:

$$strength = \sqrt{(\ topic's\ share\ of\ the\ weight \times average\ membership\ )} \quad (3.5)$$

Part 2: Employer demand. This measures how strongly the documents ask for the topic's skills. For each of the topic's ten most frequent skills, the system multiplies the number of documents that mention the skill by the share of that skill's mentions that fall inside this topic. The second factor stops very common skills, such as Agile or SQL, from raising every topic equally. The results are added up and a logarithm is taken to reduce extreme values:

$$demand = ln(\ 1 + \Sigma\ documents\ with\ skill \times topic's\ share\ ) \quad (3.6)$$

Part 3: How new it is. This is the value from Equation 3.3.

The final score combines the three parts with weights that add up to 1:

$$score = 0.40 \times demand + 0.35 \times how\ often + 0.25 \times how\ new \quad (3.7)$$

Figure 3.6 shows this with the top-ranked topic of the final analysis. Its three parts were 84, 100 and 42 out of 100, which give 33.4 + 35.0 + 10.6 = 79 points. Employer demand has the largest weight because the purpose of the 30% component is to respond to what

employers and the nation need; how often a topic comes up is next, because a topic mentioned by many documents is more reliable; how new it is has the smallest weight because a topic can still be useful even if the core touches on it. These weights are defaults: planners can change them for each analysis.

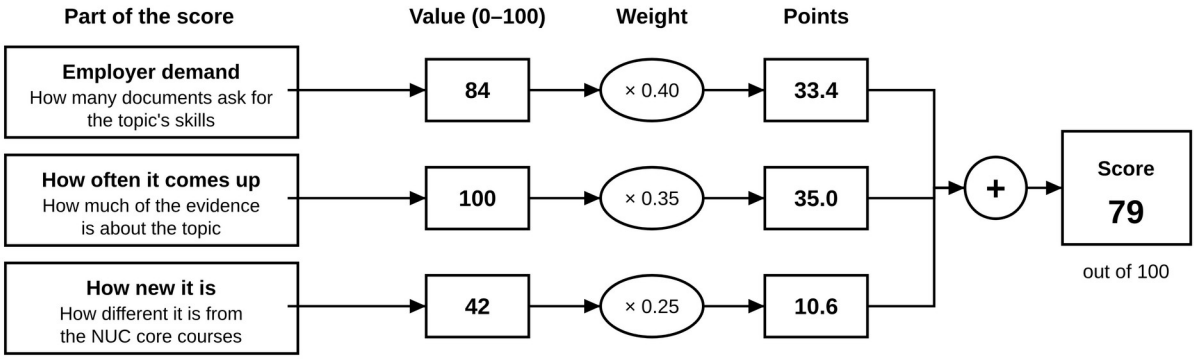


Figure 3.6: How a topic's score is calculated, using the top-ranked topic, IT Support and Help Desk Operations, as an example (values rounded)

Topics are ranked by score (larger topics come first if two scores are equal), and the top 20 become the suggested topics. On screen, scores are shown as whole numbers out of 100 and labelled High (70 and above), Medium (40–69) or Low (below 40). The exact calculation is available under "Show calculation".

3.8.6 Analysis Algorithm

The analysis steps can be written as the following algorithm.

```
Algorithm 3.1: Analysis session
Input:  chosen documents, compared NUC courses, settings (weights,
threshold, number of topics N)
Output: ranked suggested topics with evidence
1   load the extracts of the chosen documents with their embeddings
2   find keywords with TF-IDF
3   find skills with the EntityRuler (mentions and number of documents)
4   group extracts into topics with BERTopic (UMAP + HDBSCAN, seed 42)
5   merge topics whose centres are more than 83% similar
6   for each topic: give it a catalogue name if at least 50% similar;
write its description
7   for each topic: max_sim = highest similarity to any NUC course;
how_new = 1 − max_sim
8       label "May already be in NUC core" if max_sim > threshold, else
"Not in NUC core"
9   calculate demand and how-often values; scale all three parts between 0
and 1
10  score = w1 × demand + w2 × how_often + w3 × how_new
11  rank topics by score; keep the top N with their 8 closest extracts as
evidence
12  save the results and mark the analysis as completed
```

3.9 Behavioural Design

Behavioural diagrams show how people use the system and how its parts work together. They are drawn with the Unified Modeling Language (UML), the standard notation for software design.

3.9.1 Use Case Diagram

A use case diagram shows who uses the system (the actors, drawn as stick figures) and what they can do with it (the use cases, drawn as ovals inside the system boundary). Figure 3.7 shows the three actors. A Viewer, such as a Head of Department, Dean or external examiner, can look at results and download existing reports. A Curriculum Planner can do everything a Viewer can, and can also upload documents, run analyses, decide on topics, design courses and generate reports. An Administrator can do everything a Planner can, and can also manage users, the NUC core, settings, skill lists and the list of names, and view the activity log. The arrows with hollow triangles show this "can do everything the other role can" relationship, which UML calls generalisation.



Figure 3.7: Use case diagram

3.9.2 Activity Diagram

An activity diagram shows the steps of a workflow in order. Figure 3.8 shows the main workflow, divided into two columns ("swimlanes"): what the Curriculum Planner does, and what the system does. The filled circle is the start, rounded rectangles are actions, diamonds are decisions (with the conditions written in square brackets) and the circled dot is the end.

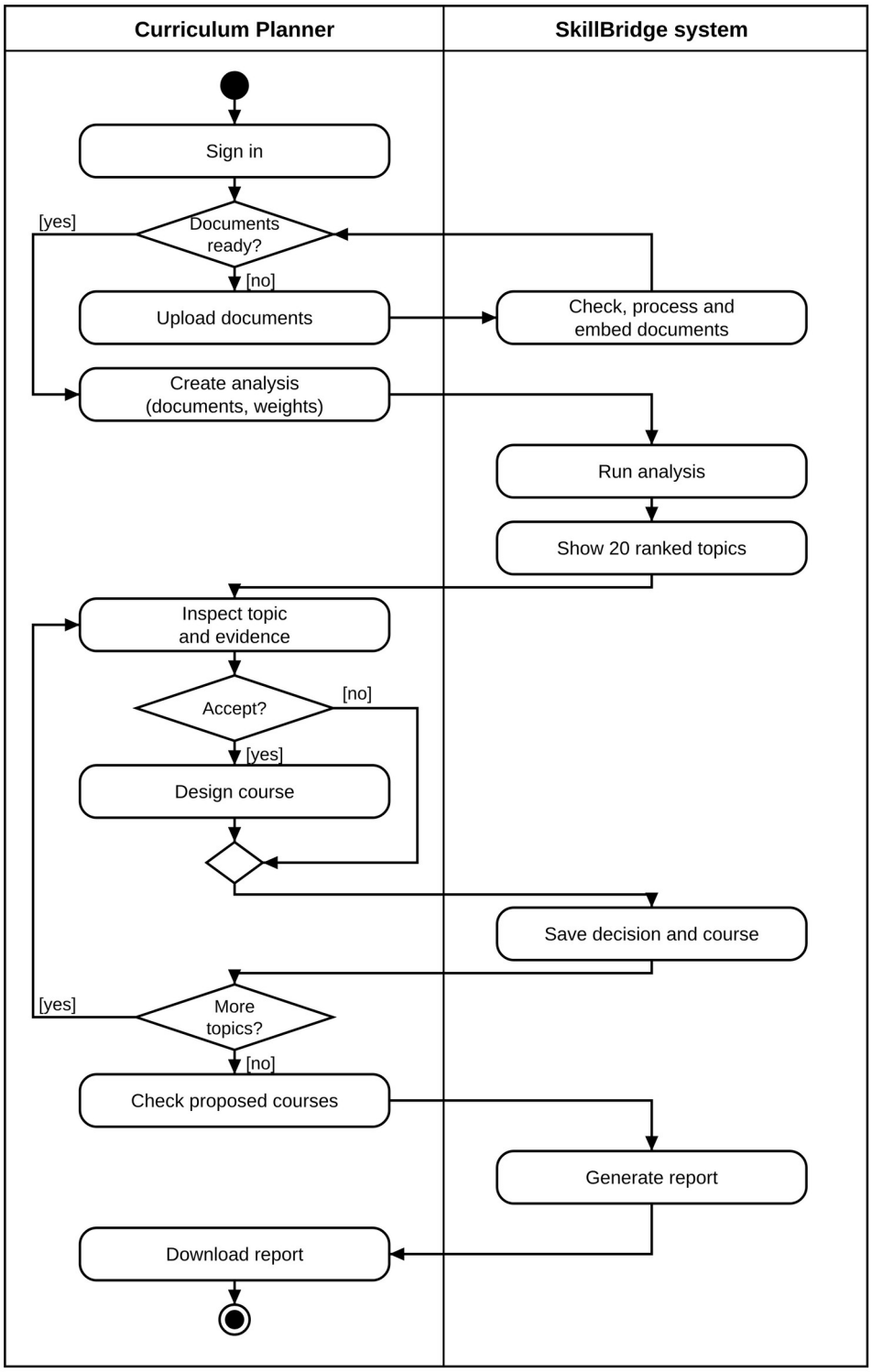


Figure 3.8: Activity diagram of the curriculum planning workflow

3.9.3 Sequence Diagram

A sequence diagram shows the messages passed between parts of the system over time, read from top to bottom. Figure 3.9 shows what happens when a planner runs an analysis. The interface sends the request to the API, which checks the user's role, records the new analysis in the database, places a job on the Redis queue and replies at

once. The worker then carries out the eight stages, updating the progress in the database after each one. Meanwhile, the interface asks for the progress every two to three seconds and shows it to the planner. When the analysis is finished, the interface fetches and displays the 20 suggested topics. Solid arrows are requests, dashed arrows are replies, and the boxes marked "loop" show steps that repeat.

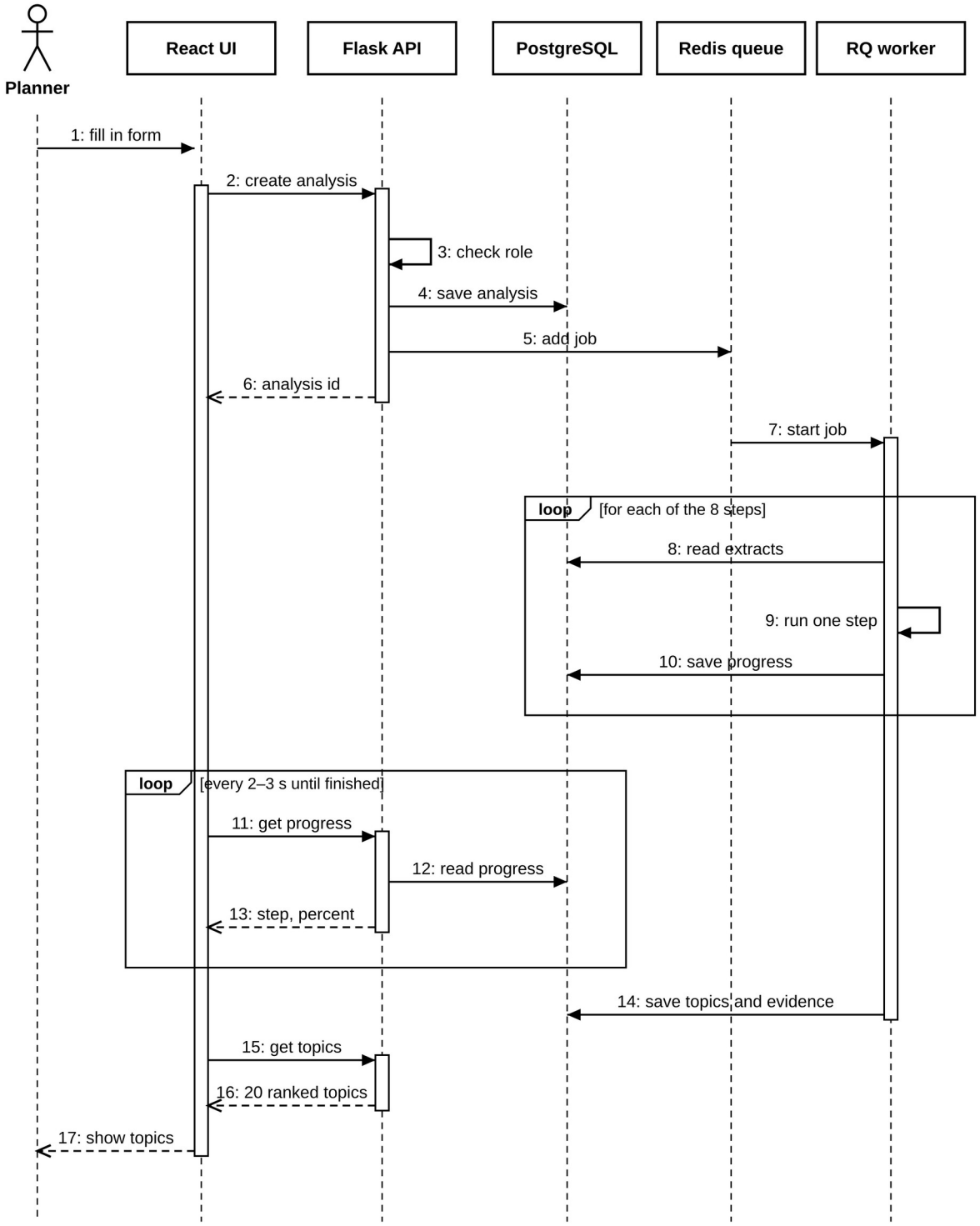


Figure 3.9: Sequence diagram for running an analysis

3.10 Database Design

The database is built in PostgreSQL 16 with the pgvector extension. It has 16 tables for the application, plus one table that keeps track of database updates; the tables are created and updated with Alembic migrations. Figure 3.10 shows the main tables and how they are related, using crow's-foot notation: the symbols at the end of each line show how many records on one side can be linked to a record on the other side (the key in the figure explains each symbol). For example, one document is split into one or many passages (extracts), and one suggested topic becomes at most one designed course. Table 3.5 describes every table.

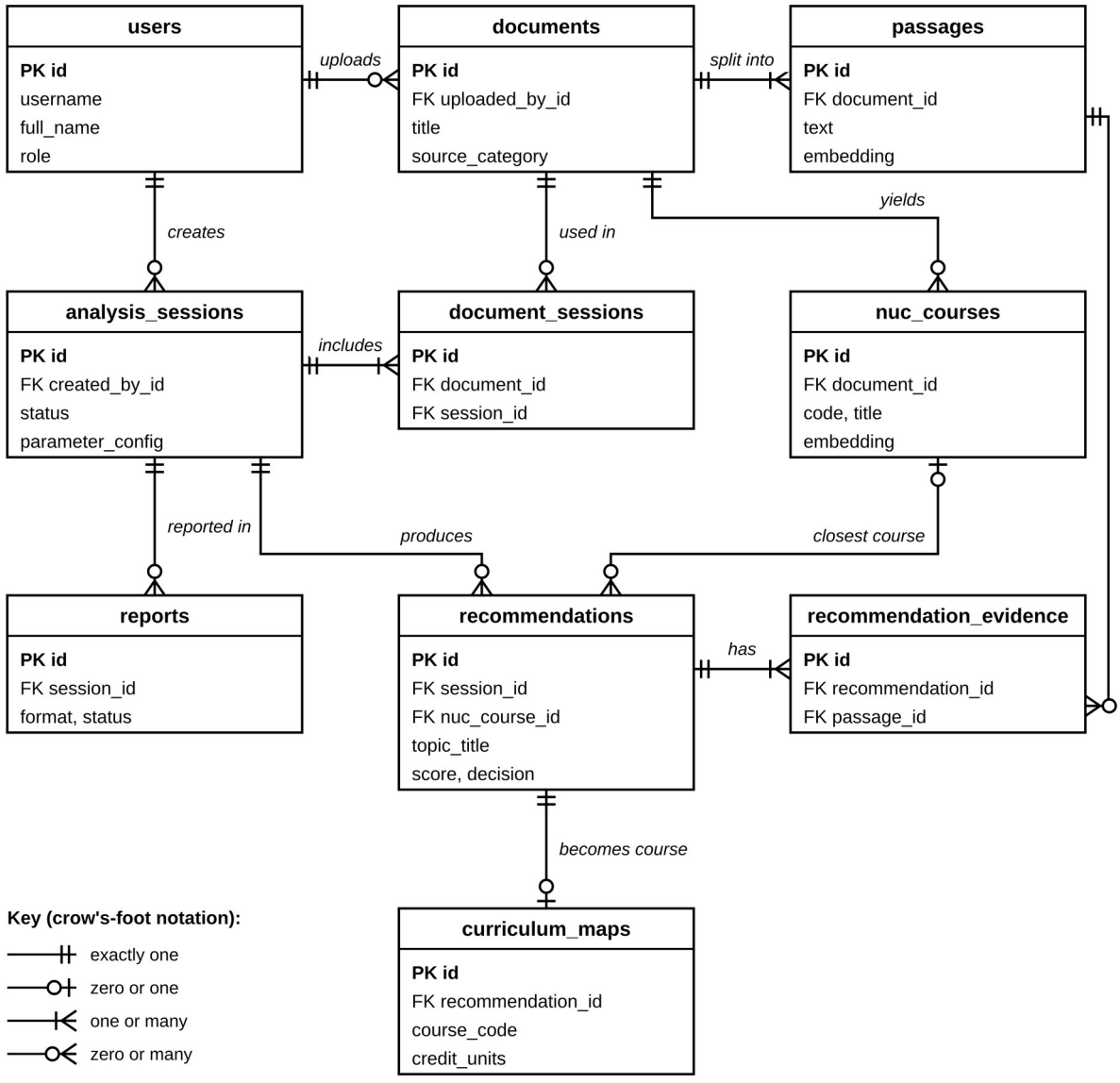


Figure 3.10: Entity-relationship diagram (main tables and fields)

Table 3.5: Database tables

Table	What it stores
users	User accounts with name, role (admin, planner or viewer), password hash and session token
documents	Uploaded documents with title, type, source, publication date, status and fingerprint (SHA-256)
passages	The extracts of each document, with their text, simplified text and 384-number embedding
nuc_core_versions	Versions of the NUC core document, with one marked as active
nuc_courses	Courses found in the NUC core, with code, title, units, text and embedding
analysis_sessions	Analyses, with status, current step, progress, settings and step timings
document_sessions	Which documents each analysis uses
nlp_results	The full results of each analysis (keywords, skills, topics, similarities)
recommendations	Suggested topics with title, description, keywords, skills, the three score parts, total score, closest NUC course and decision
recommendation_evidence	The evidence extracts of each suggested topic
curriculum_maps	Courses designed for accepted topics (code, title, 1–3 units, prerequisites, learning outcomes)
reports	Generated reports with format, sections and status
settings	System settings
skill_patterns	The skill patterns and the skill each one stands for
stop_words	Words to ignore
audit_logs	The activity log: who did what, and when

The database itself protects the data from mistakes. Links between tables are enforced (a passage cannot exist without its document), some values must be unique (one designed course per suggested topic), and some values are restricted (credit units must be 1, 2 or 3, and every score must be between 0 and 1).

3.11 Security and Access-Control Design

Every request is checked on the server to make sure the user's role allows it. The screens also hide buttons that a user cannot use, but the real protection is on the server. Table 3.6 shows what each role may do.

Table 3.6: What each role may do

Action	Administrator	Planner	Viewer
Sign in, view own profile, change own password	Yes	Yes	Yes

Action	Administrator	Planner	Viewer
View dashboard, documents, analyses, evidence, suggested topics, proposed courses	Yes	Yes	Yes
Download existing reports and the topics spreadsheet (CSV)	Yes	Yes	Yes
Upload, edit, archive or delete documents	Yes	Yes	No
Create, run, retry or delete an analysis	Yes	Yes	No
Accept, reject or defer topics; design courses	Yes	Yes	No
Generate or delete reports	Yes	Yes	No
Upload NUC core versions	Yes	No	No
Manage users, settings, skill patterns, stop words and names	Yes	No	No
View and export the activity log	Yes	No	No

Other security measures are:

- passwords are stored only as bcrypt hashes, never as plain text;
- each user has a secret session token that changes when their password is changed or their account is deactivated, so any other signed-in sessions are ended;
- the same error message is shown for an unknown username and a wrong password, so that an attacker cannot discover which usernames exist;
- every request that changes data must carry an anti-forgery (CSRF) token;
- uploaded files are stored under random names; and
- 31 kinds of action, including every sign-in, decision and change of settings, are recorded in the activity log.

3.12 User Interface Design

The screens were designed for lecturers and administrators who are not NLP specialists. Three principles guided the design:

- **Plain language.** Technical words are replaced on screen: "topic" instead of "theme", "extract" instead of "passage", and "employer demand", "how often it comes up" and "how new it is" instead of skill demand, theme strength and novelty. Technical detail is kept in a "Show calculation" panel and in advanced settings for administrators.
- **Evidence first.** Every suggested topic shows why it is recommended, an example quotation with its source, and the documents behind it.

- **Guided workflow.** The screens always show the next step (review topics, design courses, download report) and how much of the review is done.

The final visual design, branded SkillBridge, has a split-screen sign-in page, a sidebar that shows administration links only to Administrators, and a consistent set of colours, fonts and components across all screens. The screens are presented in Chapter Four.

3.13 Evaluation Design

The system is evaluated against its objectives using the methods in Table 3.7.

Table 3.7: Evaluation design

Aspect	Method	Measure	Target
Topic quality	Compare the system's topics with an LDA baseline on the same extracts (28 topics, 3 random seeds)	C_v and UMass coherence (Röder et al., 2015)	Higher than LDA
Repeatability	Run the final analysis again with the same documents and settings	Same topics, ranks and scores	Identical results
Speed	Time each step of real and test analyses	Seconds per step and in total	≤ 60 s for about 20 documents (warm)
Software correctness	Automated back-end and front-end tests; code-quality checks	Tests passed; code coverage	All pass; back-end coverage ≥ 85%
Skill extraction	Two people mark the skills in 60 sample extracts	Precision, recall, F1; Cohen's kappa	P > 0.80, R > 0.75, F1 > 0.75
Overlap detection	People label 140 topic–course pairs as covered or not	AUC-ROC; precision, recall and F1 at 0.80; best threshold	Calibrated threshold
Usability	5–8 lecturers and curriculum officers do 5 tasks and fill in the SUS	SUS score (average, spread, 95% range); tasks completed	SUS ≥ 70

3.14 Summary

This chapter has explained the incremental method used to build the system and the weaknesses of the current manual process. It has introduced SkillBridge, listed its requirements and described the documents it analyses. It has explained the design in detail: the three-tier architecture with a background worker, the pipeline that turns documents into ranked topics, the three-part scoring model, the UML and database diagrams, and the security, interface and evaluation designs. The next chapter shows how this design was built and tested, and presents the results.

CHAPTER FOUR
IMPLEMENTATION, TESTING AND RESULTS

4.1 Introduction

This chapter describes the implementation of the system designed in Chapter Three, presents its user interface, reports how it was tested, and presents the results of applying it to the real corpus of Nigerian job adverts and the National Digital Economy Policy and Strategy. It then reports the evaluation of the system against the objectives of the study, including the parts of the evaluation that depend on human participants, and discusses the findings.

4.2 Implementation Environment

4.2.1 Hardware

The system was developed, run and measured on the machine described in Table 4.1. No graphics processing unit (GPU) was used; all language models ran on the CPU.

Table 4.1: Hardware used

Item	Specification
Computer	HP EliteBook 840 G4 laptop
Processor	Intel Core i5-7200U, 2.50 GHz (2 cores, 4 threads)
Memory	7.9 GB RAM
Operating system	Windows 11 Home (10.0.26100)
Services	PostgreSQL and Redis in Docker Desktop (WSL 2); API and worker run natively

4.2.2 Software

Table 4.2 lists the main software components and the exact versions used. The Python and npm dependencies are pinned to exact versions in the project's dependency files.

Table 4.2: Software components and versions

Layer	Component	Version
Front end	React / TypeScript / Vite	19.3.0 / 6.0.3 / 8.3.0
Front end	Tailwind CSS / shadcn / TanStack Query	4.3.3 / 4.21.0 / 5.103.2
Front end	React Router / React Hook Form / Zod / Recharts	8.4.0 / 7.88.0 / 4.6.5 / 3.10.1
Back end	Python (Docker / development laptop)	3.11 / 3.13.15
Back end	Flask / Flask-SQLAlchemy / SQLAlchemy / Alembic	3.1.3 / 3.1.1 / 2.0.54 / 1.20.0

Layer	Component	Version
Back end	Flask-Login / Flask-WTF (CSRF) / bcrypt / Gunicorn	0.6.3 / 1.3.0 / 5.0.0 / 26.2.0
Data	PostgreSQL / pgvector / Redis / RQ	16.15 / 0.8.6 / 7 / 2.12.0
NLP	spaCy / en_core_web_sm	3.8.16 / 3.8.0
NLP	sentence-transformers / all-MiniLM-L6-v2 / PyTorch (CPU)	6.1.0 / 384-d / 2.14.0
NLP	BERTopic / umap-learn / hdbscan / scikit-learn	0.17.4 / 0.5.12 / 0.8.44 / 1.9.1
Documents	PyMuPDF / python-docx / WeasyPrint	1.28.2 / 1.2.0 / 70.0
Testing	pytest / pytest-cov / Vitest / Testing Library	9.1.1 / 7.1.0 / 5.0.1 / 16.3.3
Quality	ruff / black / ESLint / Prettier	0.16.8 / 26.5.1 / 10.11.0 / 3.9.9

4.3 Implementation of the System Modules

The back end (server side) is written in Python with the Flask framework and is divided into separate modules for settings, database models, API routes, processing services and background tasks. The main modules work as follows.

- **Sign-in and access control.** Users sign in with a username or e-mail address and a password. The password is checked against its stored bcrypt hash. The signed-in session is kept in a secure cookie that expires after eight hours and contains a secret token for that user, so that changing a password signs the user out everywhere else. Every request that changes data must carry an anti-forgery (CSRF) token. Each API route is marked with the roles allowed to use it: a user who is not signed in receives an "unauthorised" reply (HTTP 401), and a signed-in user without the right role receives a "forbidden" reply (HTTP 403).
- **Documents.** When a file is uploaded, it is checked, saved under a random name and placed on the job queue. The worker then reads, cleans and splits it, records the meaning of each extract and saves the results, while the document's status changes from Queued to Processing to Ready (or Failed, with the reason). Planners can correct a document's title, source, web link and date, and the corrected details are then used everywhere the document is cited.
- **NUC core.** An Administrator uploads a CCMAS document with a version name. The worker finds its courses, records their meaning and makes this version the active one. A setting lists which course codes and title words (such as GST, SIWES and Final Year Project) should be left out of the comparison.
- **Analysis.** When a planner creates an analysis, the system saves its name, its documents and a copy of all its settings, and places it on the queue. The worker

then carries out the eight steps described in Section 3.8, recording how long each step takes and updating the progress shown on screen. The results are saved as suggested topics with their evidence, and also in full for the evidence pages.

- **Review and course design.** Planners record a decision on each suggested topic (accepted, rejected or discuss later), with optional notes, and design one course for each accepted topic. The database allows only one course per topic and only 1, 2 or 3 credit units, and the application requires course codes to be unique within an analysis.
- **Reports.** A report job gathers the analysis summary, the documents analysed, the findings, the suggested topics with their scores and evidence, the decisions and the proposed courses, and writes them into a Word document (using python-docx) or a PDF (using WeasyPrint).
- **Administration.** Administrators manage users, settings, skill patterns, stop words and the list of course-style names, and can search and export the activity log as a spreadsheet file (CSV).

The front end (what the user sees) is a React application organised by feature: sign-in, dashboard, documents, analyses, review, reports and administration. If a user opens a page that their role does not allow, they are taken to a "No access" page. Forms are checked before they are sent, and data from the server is fetched and kept up to date with the TanStack Query library.

4.4 System Interfaces

This section presents the main screens of the implemented system, captured from the running application with the real data of the final analysis (session 13).

4.4.1 Sign In and Dashboard

The sign-in page (Figure 4.1) uses a split-screen layout with the SkillBridge logo, a short explanation of the system and the three steps of the workflow. After signing in, a Curriculum Planner sees the dashboard (Figure 4.2), which shows counts of documents, analyses and topics awaiting review, and a prominent next step. A Viewer sees a read-only summary instead (Figure 4.3).

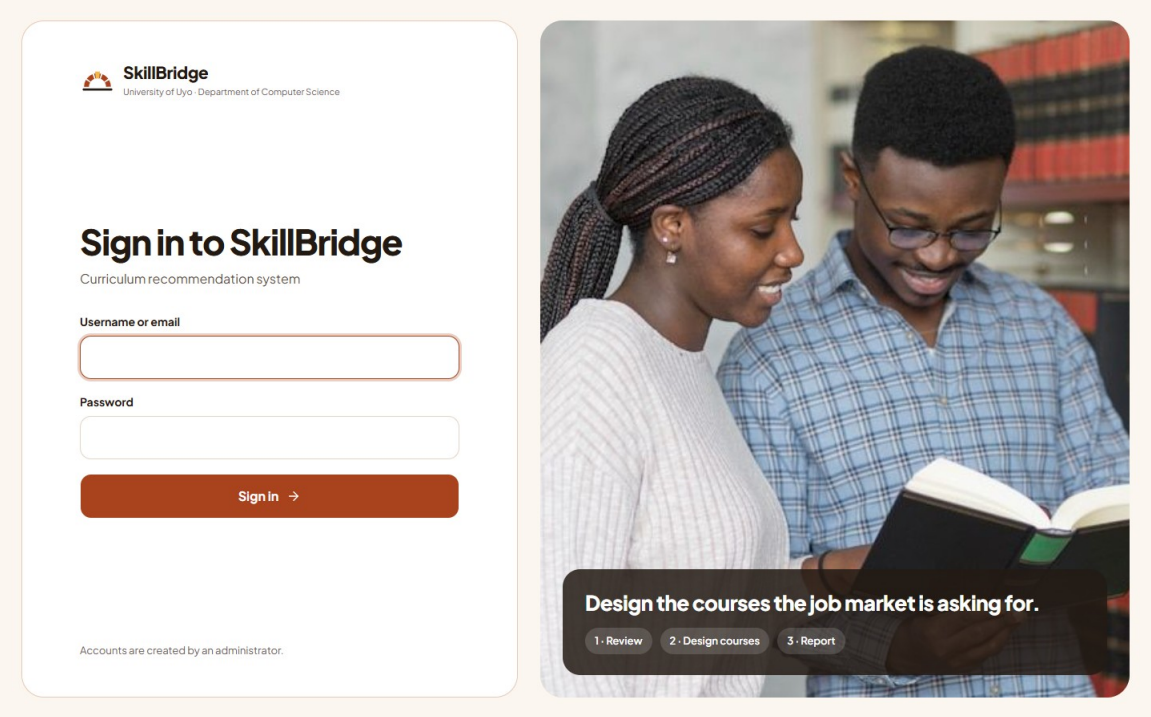


Figure 4.1: Sign-in page

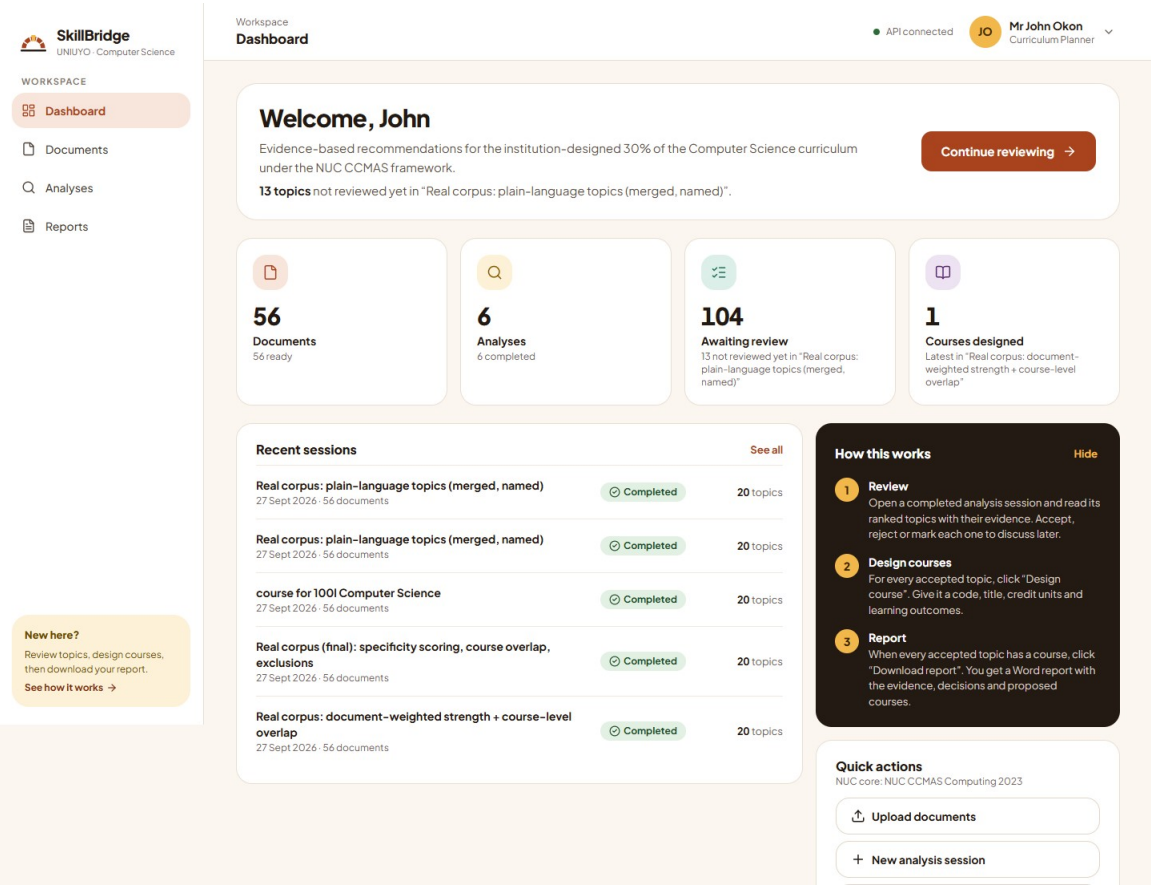


Figure 4.2: Dashboard of a Curriculum Planner

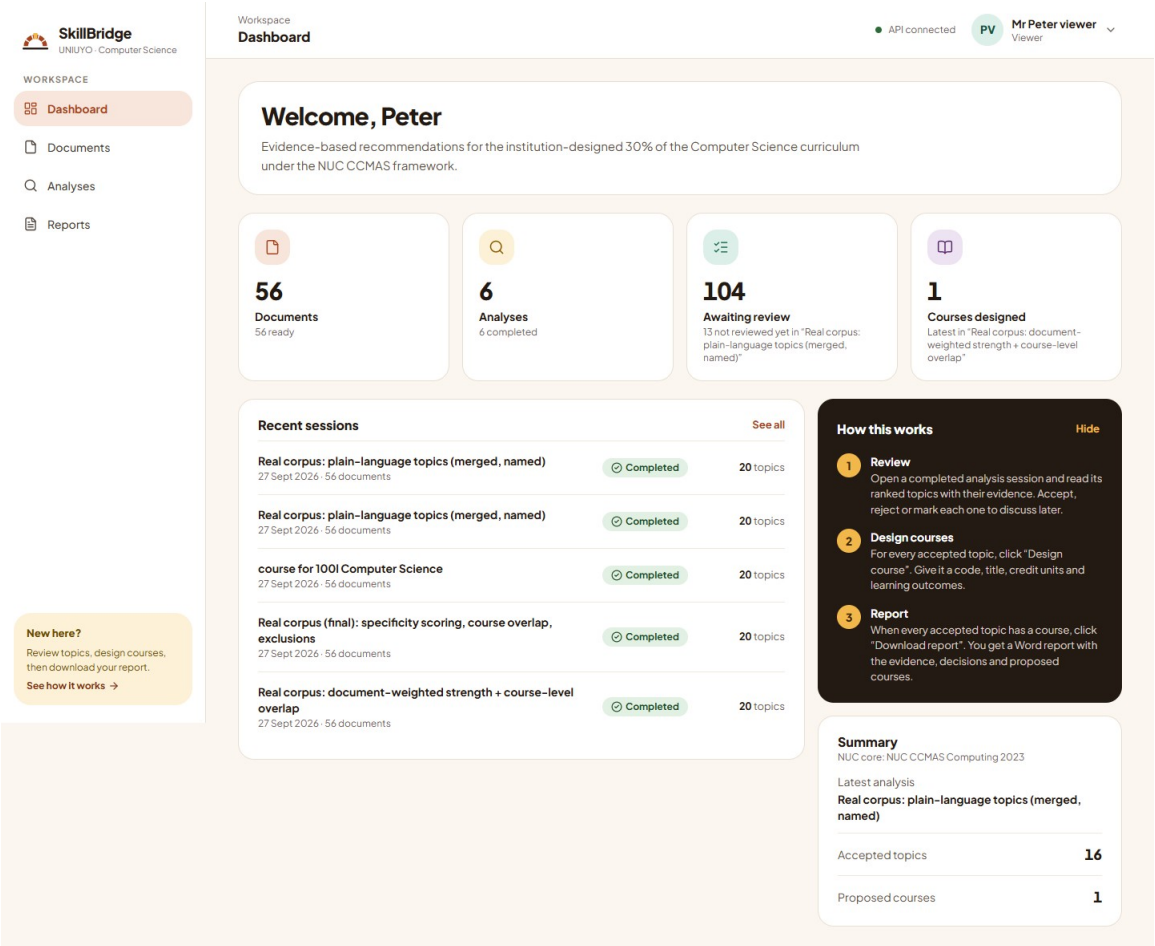


Figure 4.3: Read-only dashboard of a Viewer

4.4.2 Documents

The documents library (Figure 4.4) lists all documents with their type, source, date and processing status, with filters by type. Planners add documents through an upload dialog (Figure 4.5) in which they choose a category for each file. The document detail page (Figure 4.6) shows the document's details and the extracts produced from it.

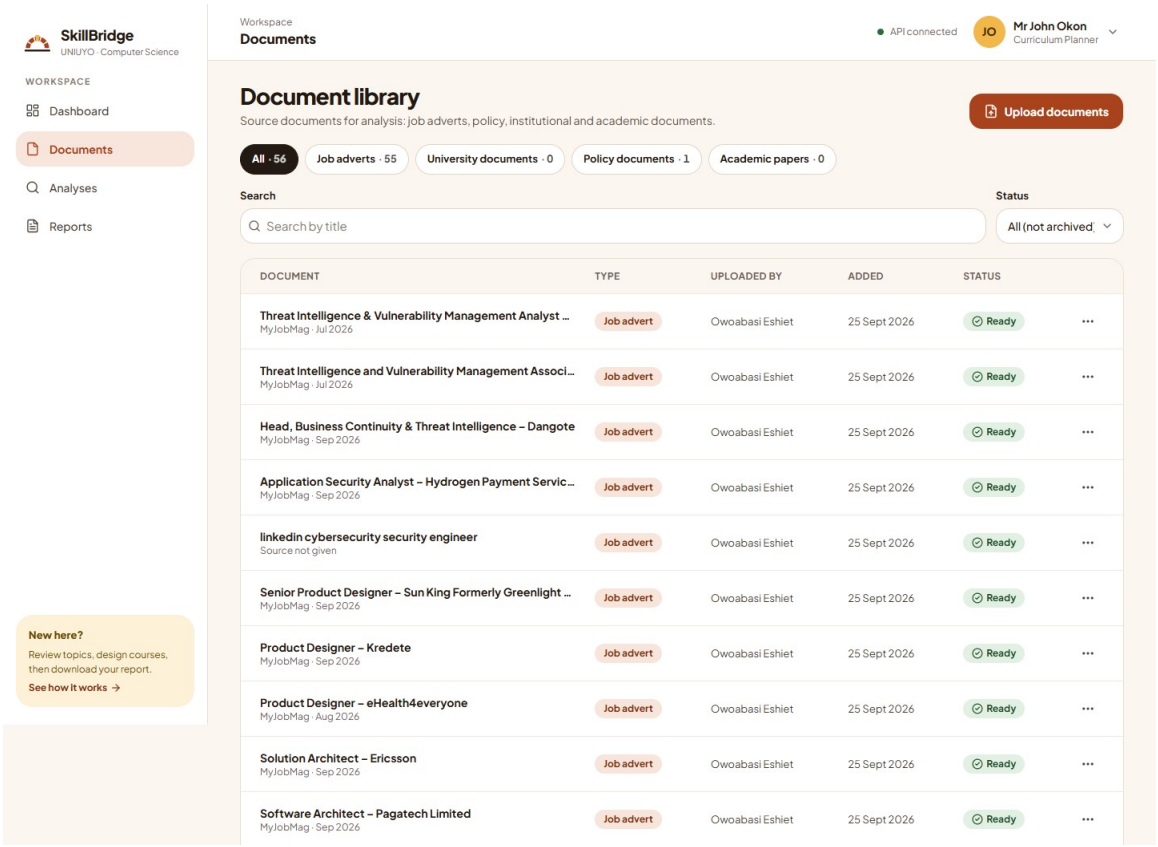


Figure 4.4: Documents library

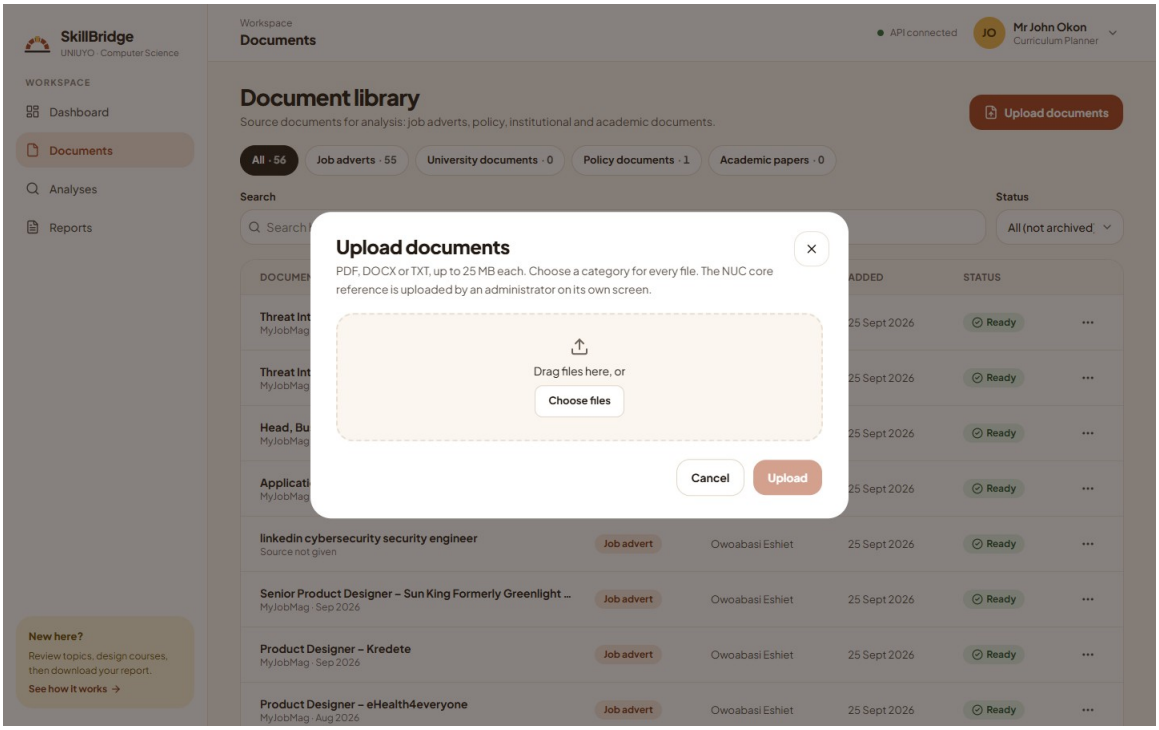


Figure 4.5: Upload documents dialog

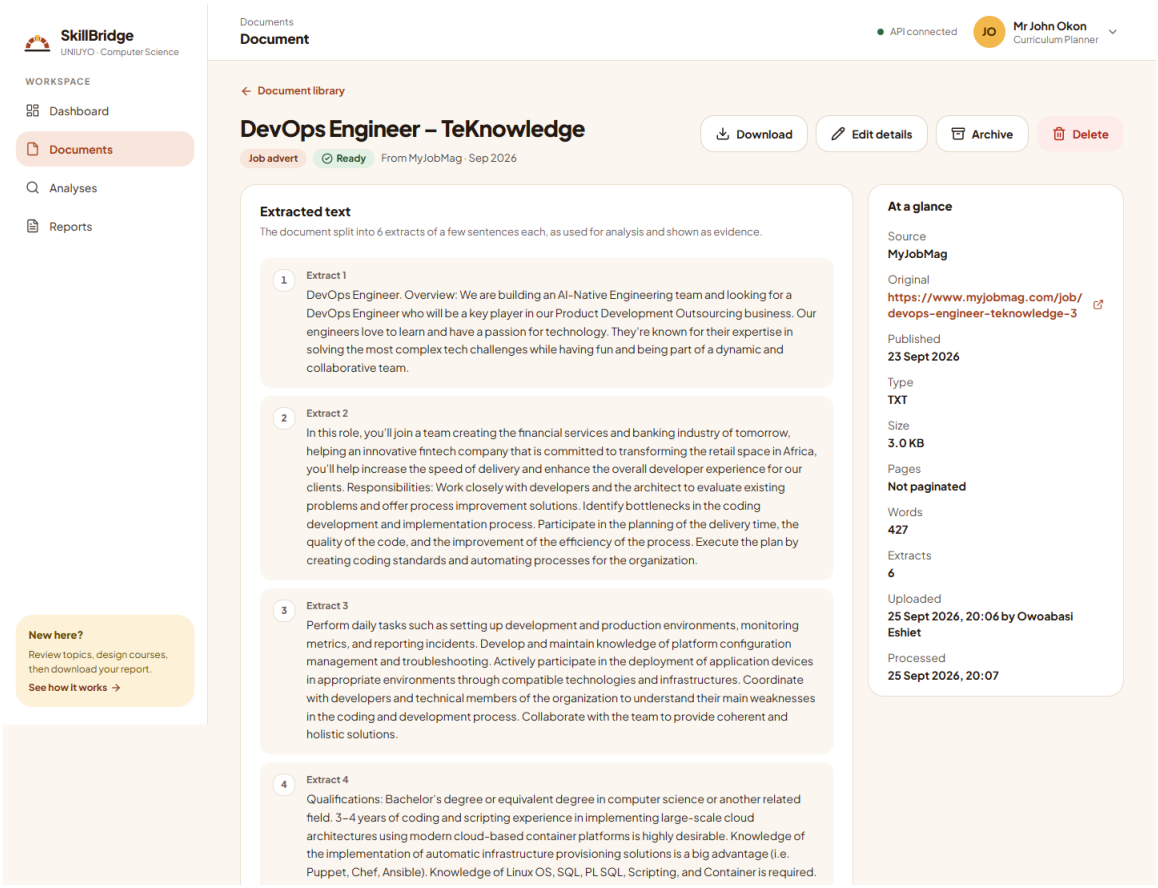


Figure 4.6: Document detail

4.4.3 Running an Analysis

A new analysis is created in three steps: naming it, choosing the documents (Figure 4.7) and choosing the scoring settings. If a needed document is not yet in the library, the planner can upload it directly from the documents step with the **Upload documents** button; the new document is processed in the background and ticked automatically as soon as it is ready. While it runs, the progress page (Figure 4.8) shows the current stage, the percentage completed and a plain explanation of what is happening. The screenshot was taken during a real run on the 56 documents of the final analysis.

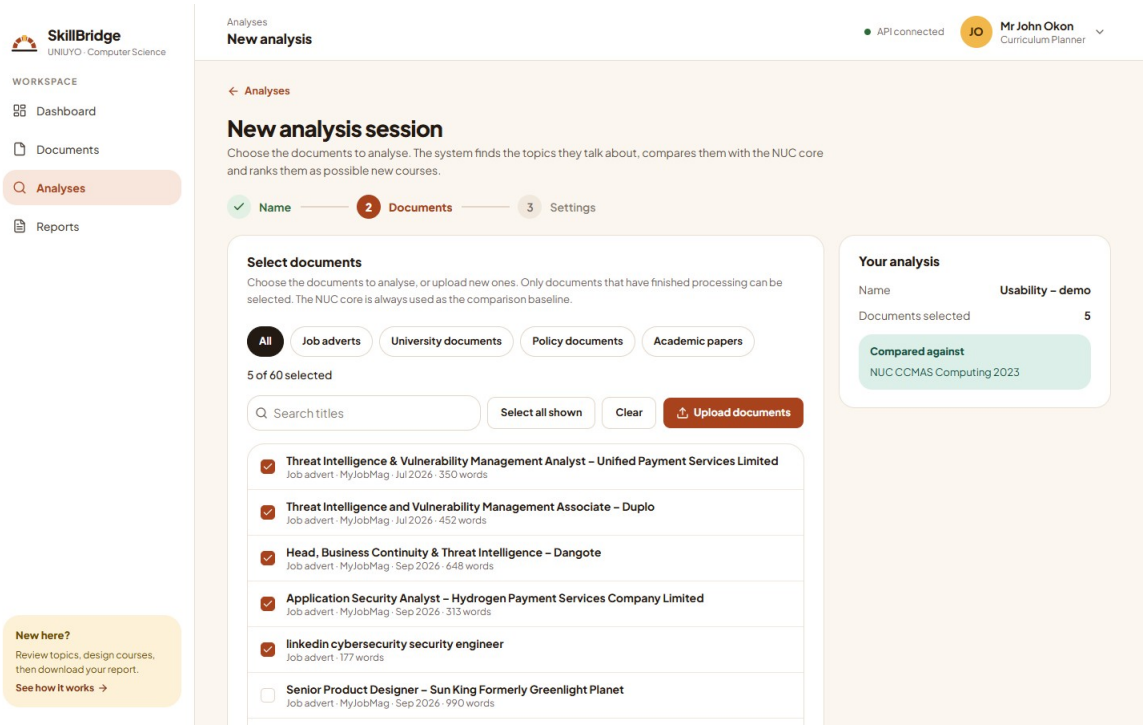


Figure 4.7: New analysis, choosing documents, with the Upload documents button

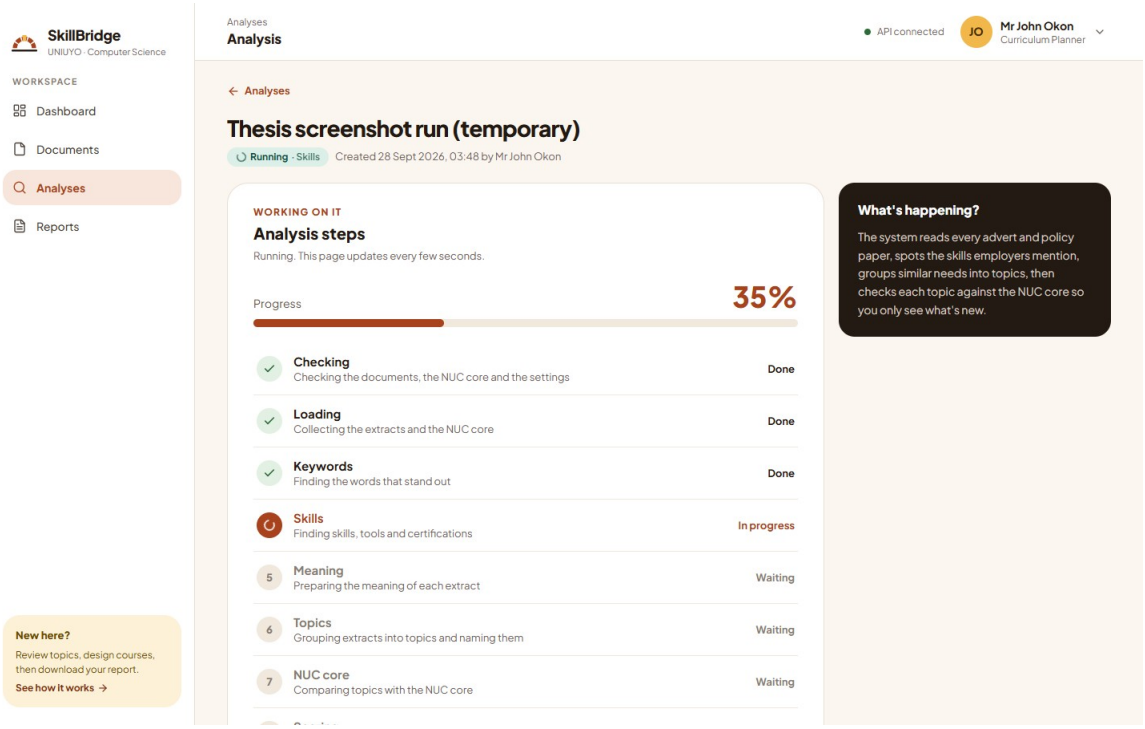


Figure 4.8: Analysis in progress

4.4.4 Evidence and Suggested Topics

The evidence page (Figure 4.9) summarises what the analysis read and presents the keywords, skills, topics and the comparison with the NUC core, each with a table beside the chart. The suggested topics page (Figure 4.10) lists the 20 top-ranked topics, each

with its course-style name, keywords, "Why this is recommended" description, score out of 100 and decision controls.

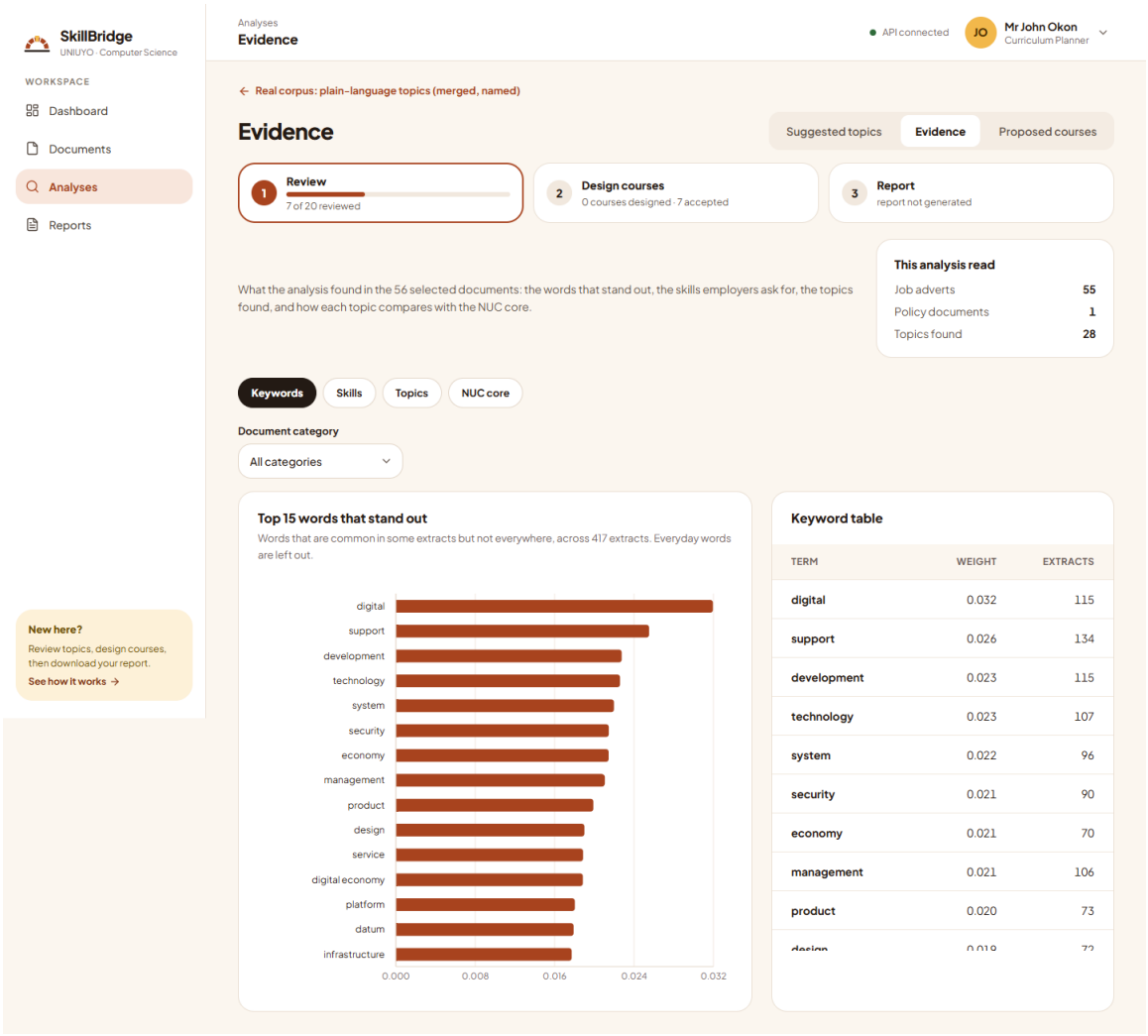


Figure 4.9: Evidence page

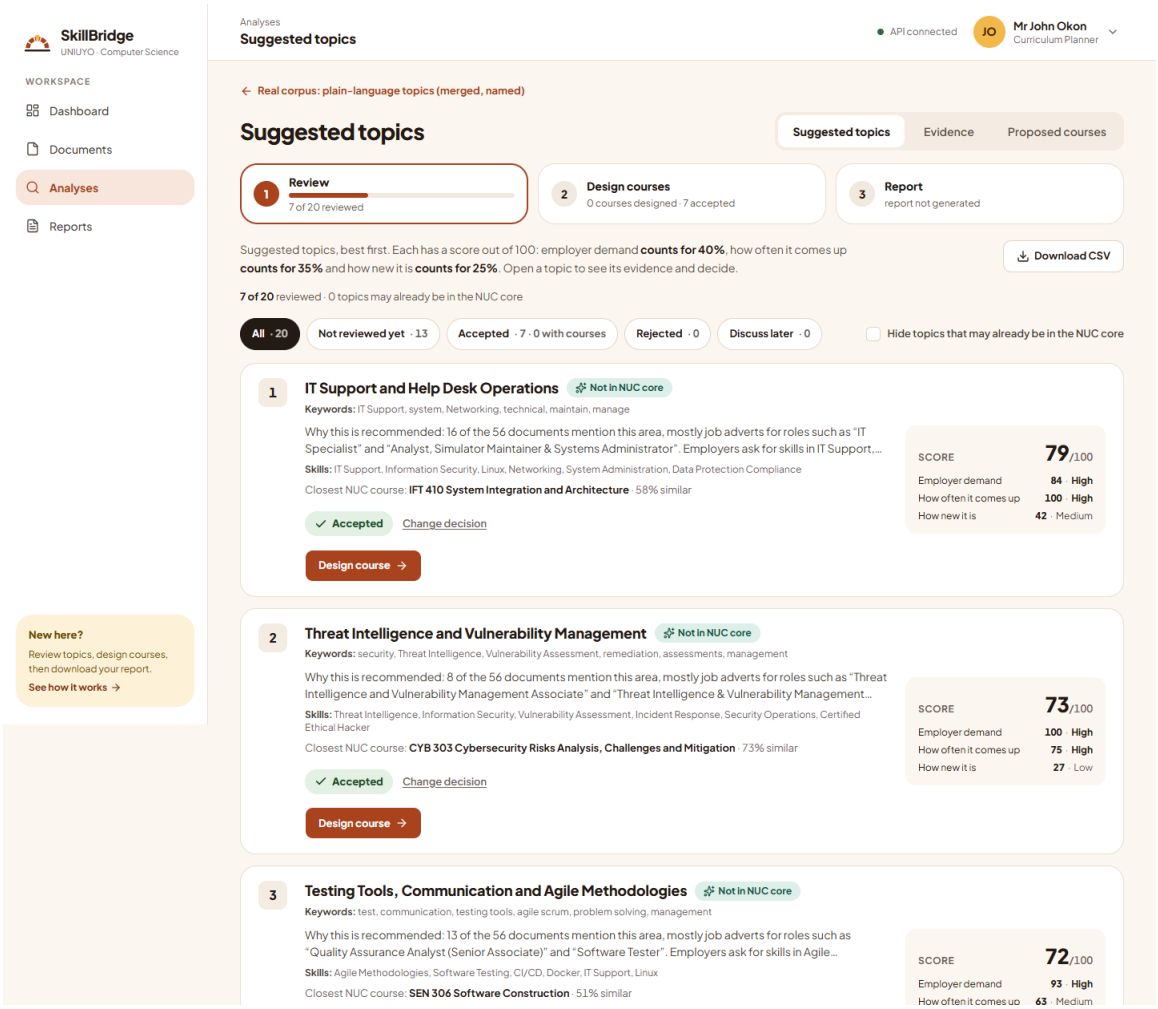


Figure 4.10: Suggested topics

4.4.5 Topic Detail

The topic detail page (Figure 4.11) shows everything behind one suggestion: the description, an example from a job advert with its source, the evidence extracts grouped by document, and the score with its three parts. Opening "Show calculation" (Figure 4.12) reveals the exact formula and values, and the comparison panel (Figure 4.13) shows the closest NUC course, its similarity and the closest extract from the CCMAS side by side with a typical extract from the topic.

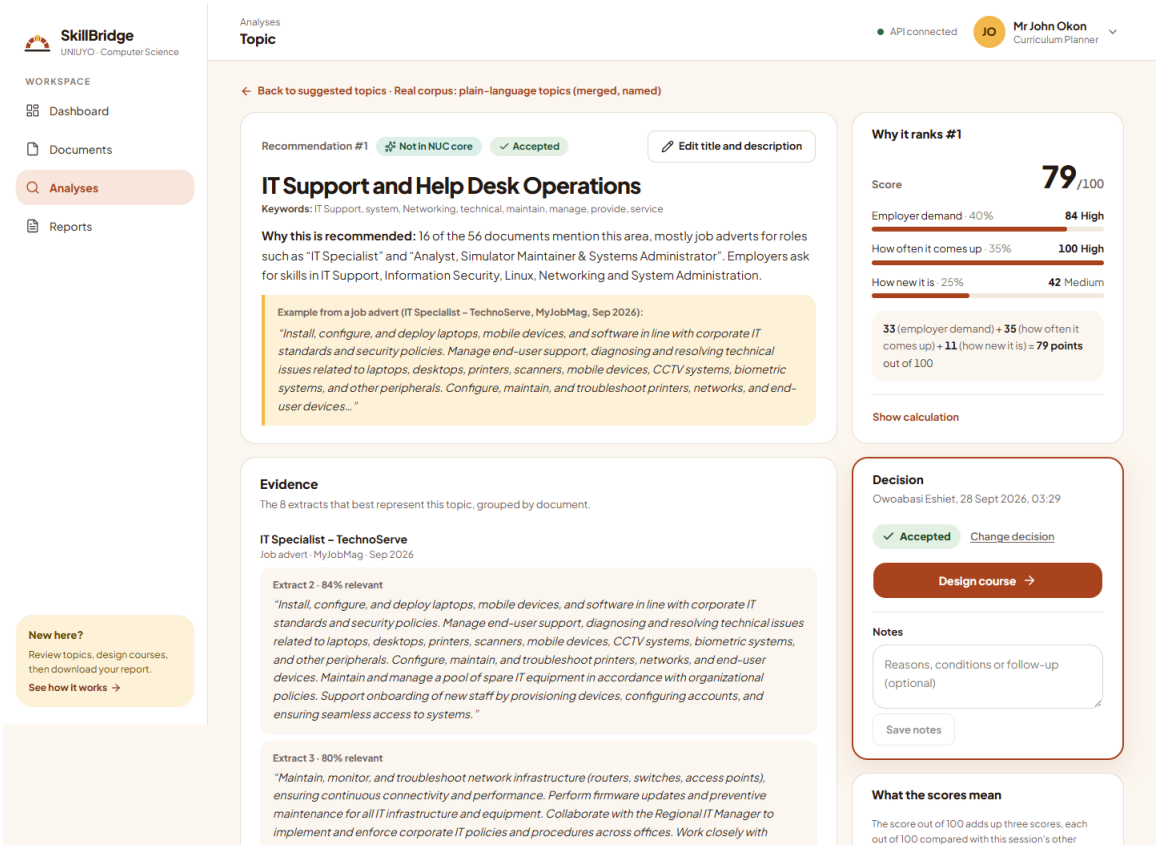


Figure 4.11: Topic detail for the top-ranked topic

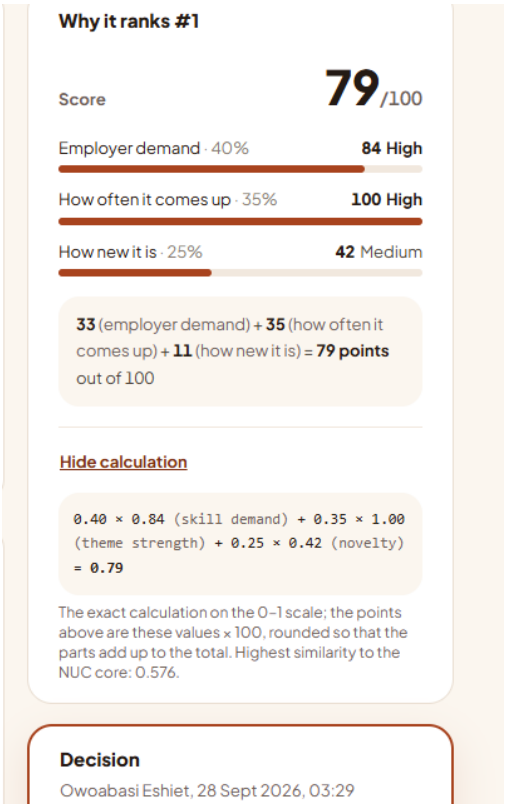


Figure 4.12: Score breakdown with "Show calculation" open

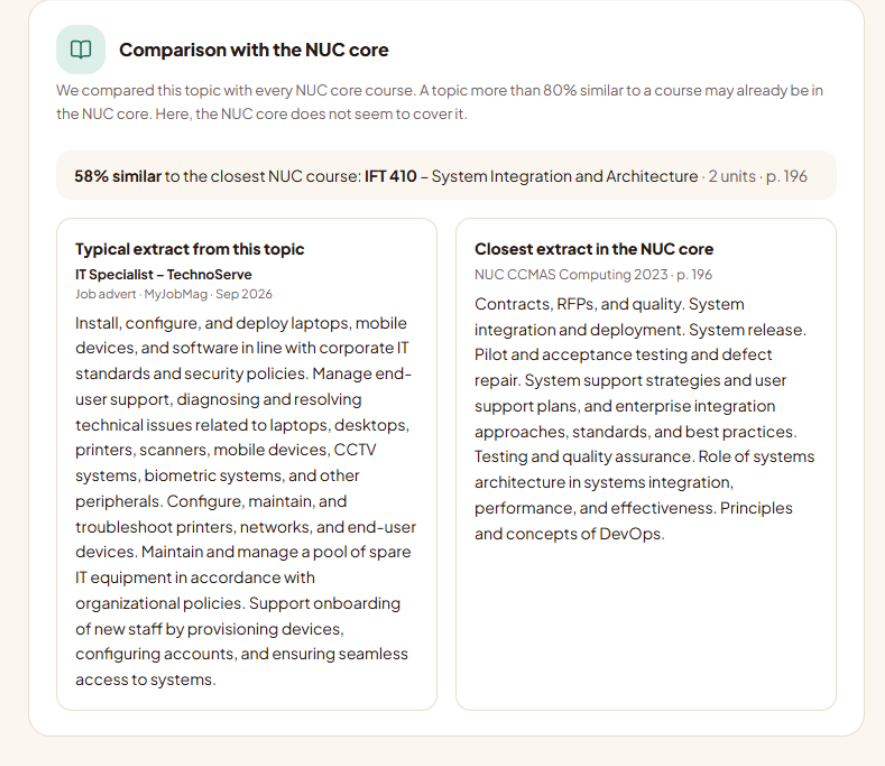


Figure 4.13: Comparison of a topic with the NUC core

4.4.6 Course Design, Proposed Courses and Reports

For an accepted topic, the planner designs a course (Figure 4.14) by entering its code, title, credit units, prerequisites and learning outcomes. The proposed courses page (Figure 4.15) lists the designed courses and their total credit units against the 30% allowance; the example shown is from an earlier analysis (session 9), the only one in which a course had been designed at the time of writing. The reports page (Figure 4.16) lists generated reports, and Figure 4.17 shows the first page of the report generated for the final analysis.

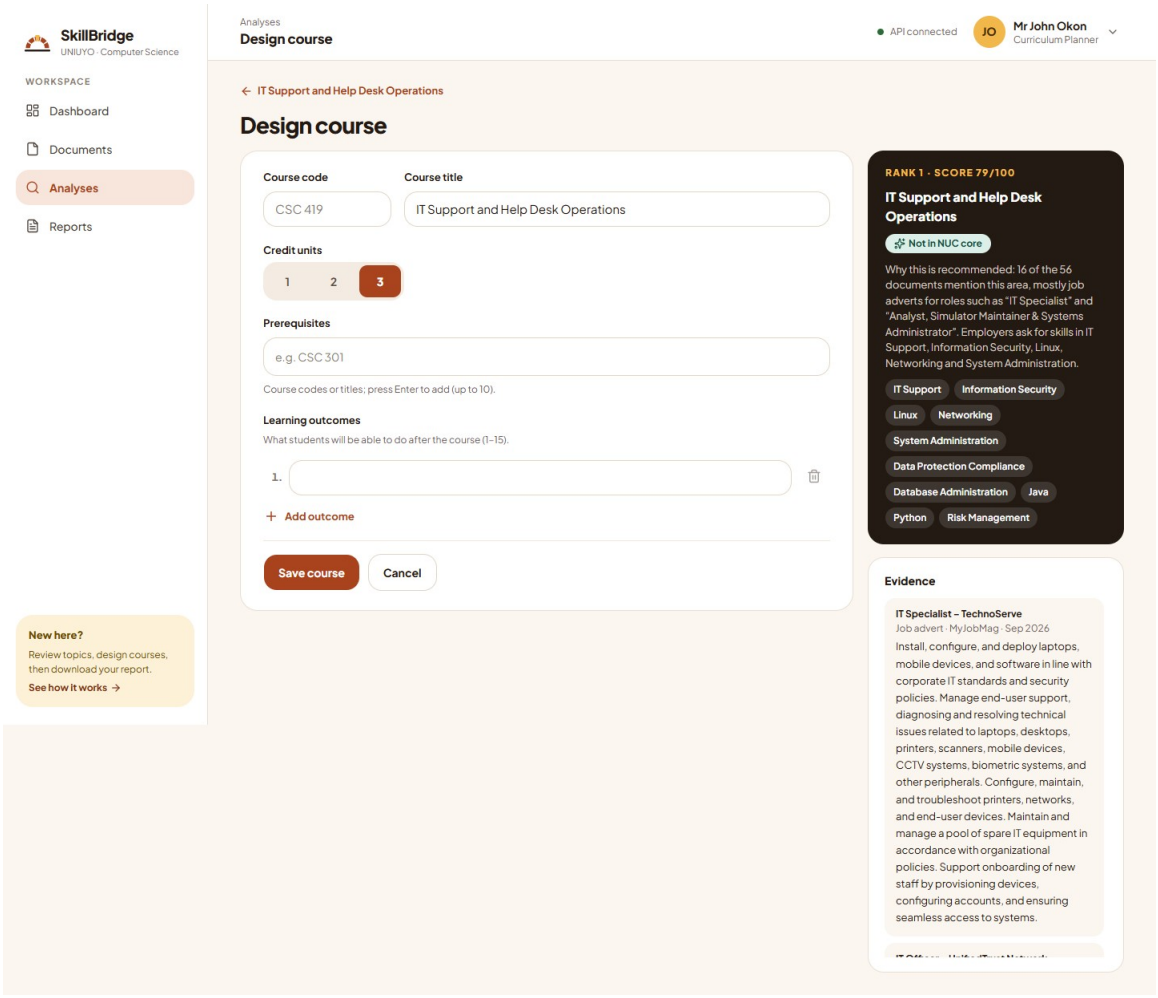


Figure 4.14: Design course form

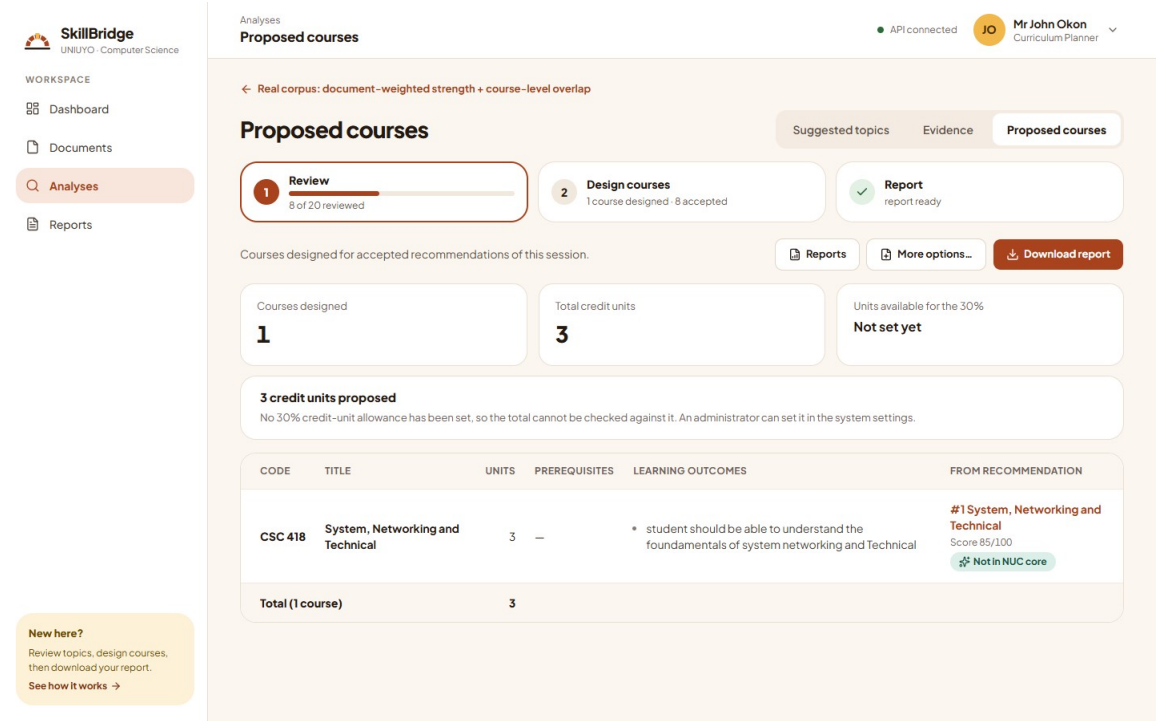


Figure 4.15: Proposed courses

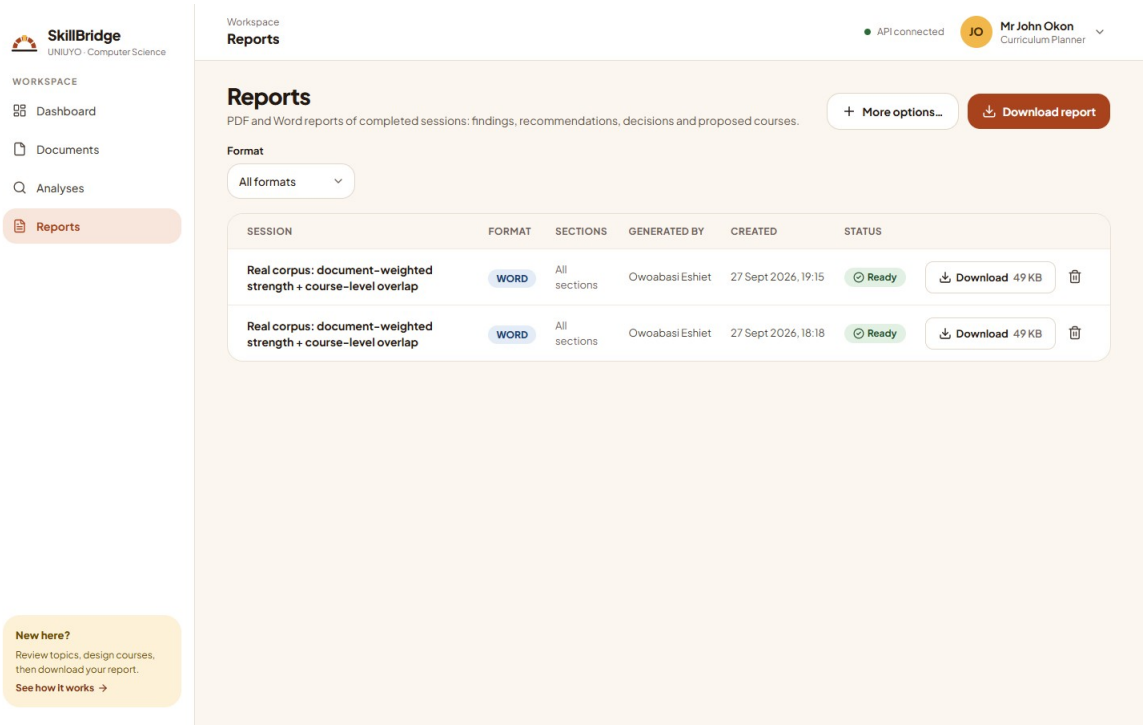
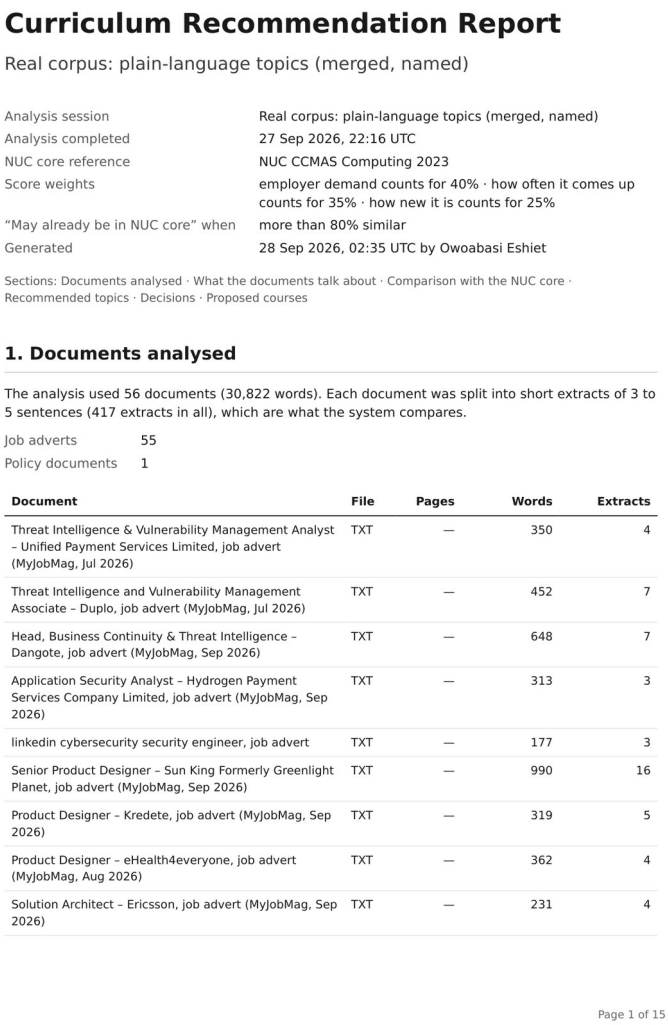


Figure 4.16: Reports



Curriculum Recommendation Report

Real corpus: plain-language topics (merged, named)

Analysis session	Real corpus: plain-language topics (merged, named)
Analysis completed	27 Sep 2026, 22:16 UTC
NUC core reference	NUC CCMAS Computing 2023
Score weights	employer demand counts for 40% · how often it comes up counts for 35% · how new it is counts for 25%
“May already be in NUC core” when	more than 80% similar
Generated	28 Sep 2026, 02:35 UTC by Owoabasi Eshiet

Sections: Documents analysed · What the documents talk about · Comparison with the NUC core · Recommended topics · Decisions · Proposed courses

1. Documents analysed

The analysis used 56 documents (30,822 words). Each document was split into short extracts of 3 to 5 sentences (417 extracts in all), which are what the system compares.

Job adverts	55
Policy documents	1

Document	File	Pages	Words	Extracts
Threat Intelligence & Vulnerability Management Analyst – Unified Payment Services Limited, job advert (MyJobMag, Jul 2026)	TXT	—	350	4
Threat Intelligence and Vulnerability Management Associate – Duplo, job advert (MyJobMag, Jul 2026)	TXT	—	452	7
Head, Business Continuity & Threat Intelligence – Dangote, job advert (MyJobMag, Sep 2026)	TXT	—	648	7
Application Security Analyst – Hydrogen Payment Services Company Limited, job advert (MyJobMag, Sep 2026)	TXT	—	313	3
linkedin cybersecurity security engineer, job advert	TXT	—	177	3
Senior Product Designer – Sun King Formerly Greenlight Planet, job advert (MyJobMag, Sep 2026)	TXT	—	990	16
Product Designer – Kredete, job advert (MyJobMag, Sep 2026)	TXT	—	319	5
Product Designer – eHealth4everyone, job advert (MyJobMag, Aug 2026)	TXT	—	362	4
Solution Architect – Ericsson, job advert (MyJobMag, Sep 2026)	TXT	—	231	4

Page 1 of 15

Figure 4.17: First page of a generated PDF report

4.4.7 Administration

Administrators see an additional Administration section. The NUC core page (Figure 4.18) shows the active CCMAS version and which courses are compared or left out. The users page (Figure 4.19) manages accounts and roles. The settings pages hold the scoring settings (Figure 4.20) and the catalogue of course-style topic names (Figure 4.21). The activity log (Figure 4.22) records who did what and when. Users who open a page their role does not allow see the No access page (Figure 4.23).

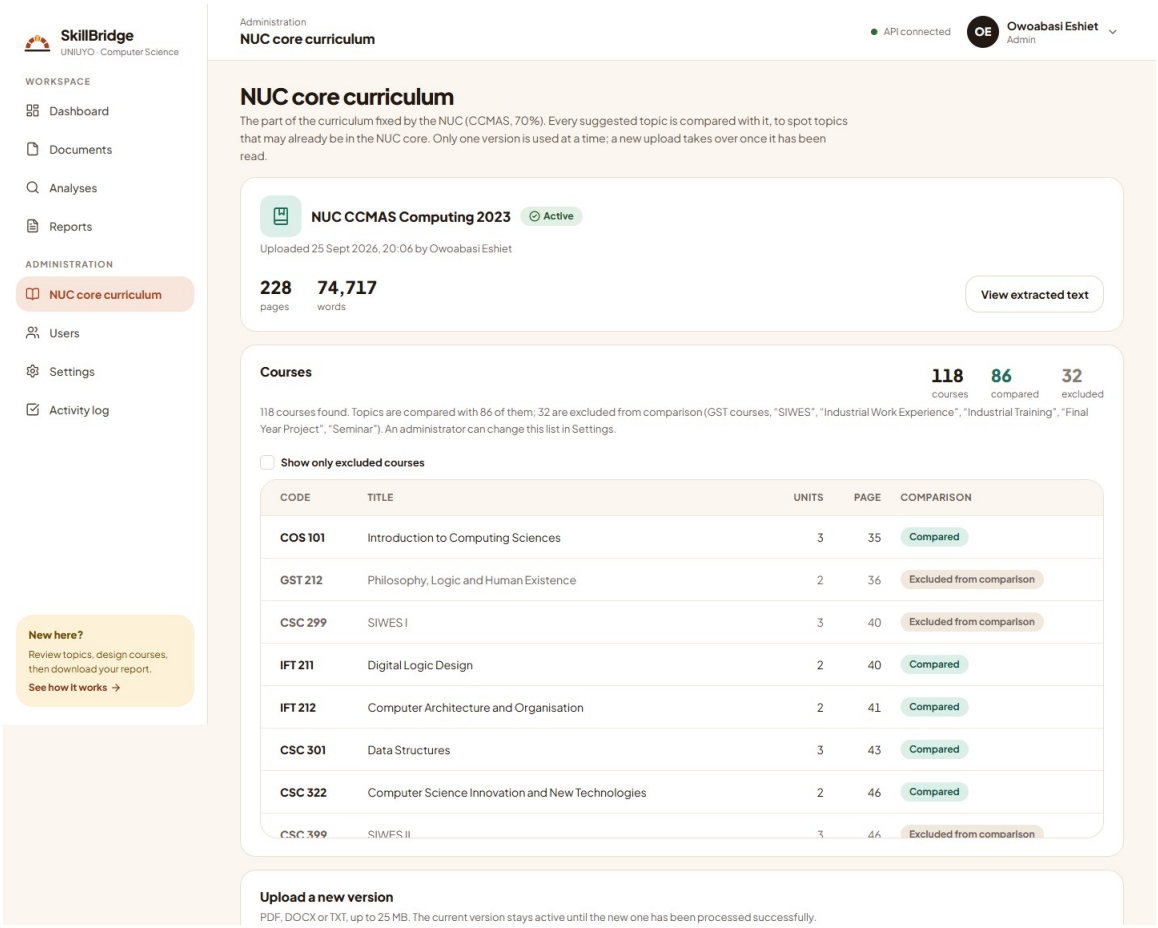


Figure 4.18: NUC core curriculum

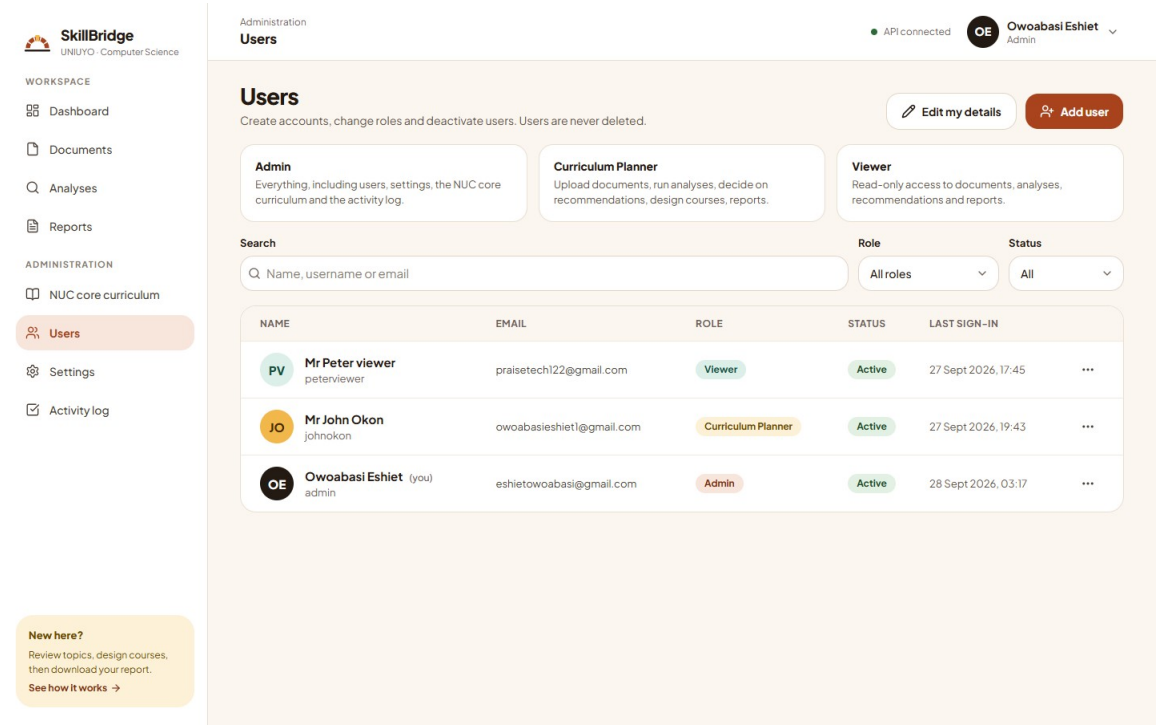


Figure 4.19: Users and roles

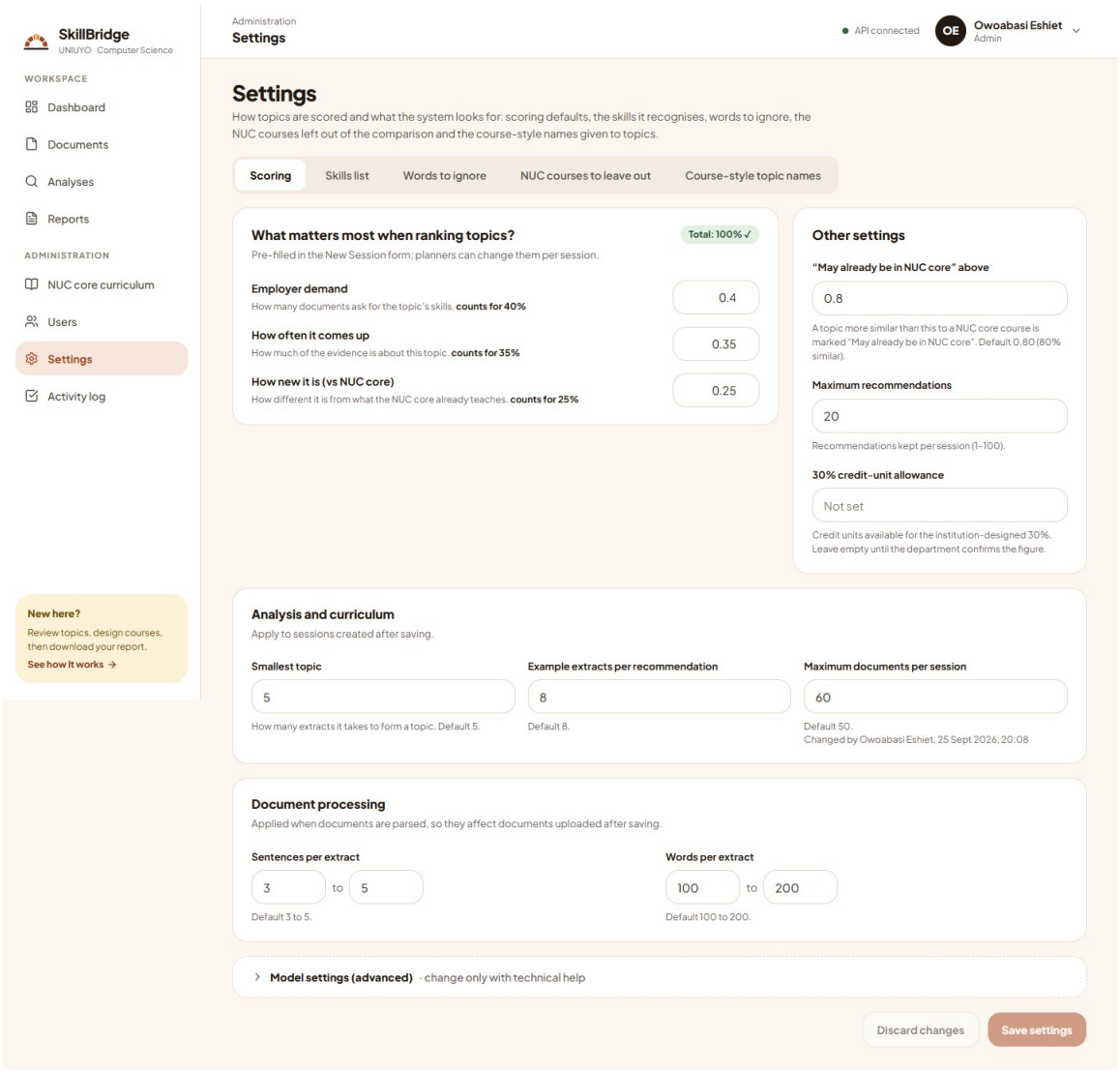


Figure 4.20: Scoring settings

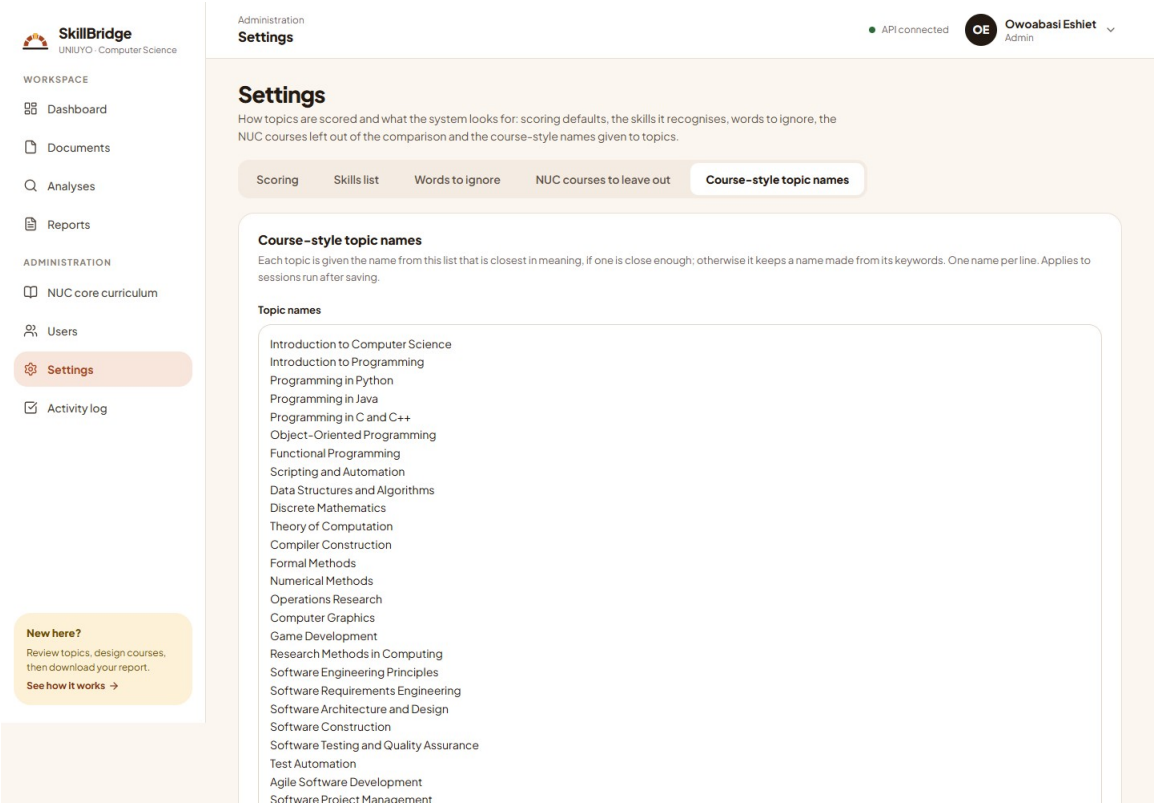


Figure 4.21: Course-style topic names catalogue

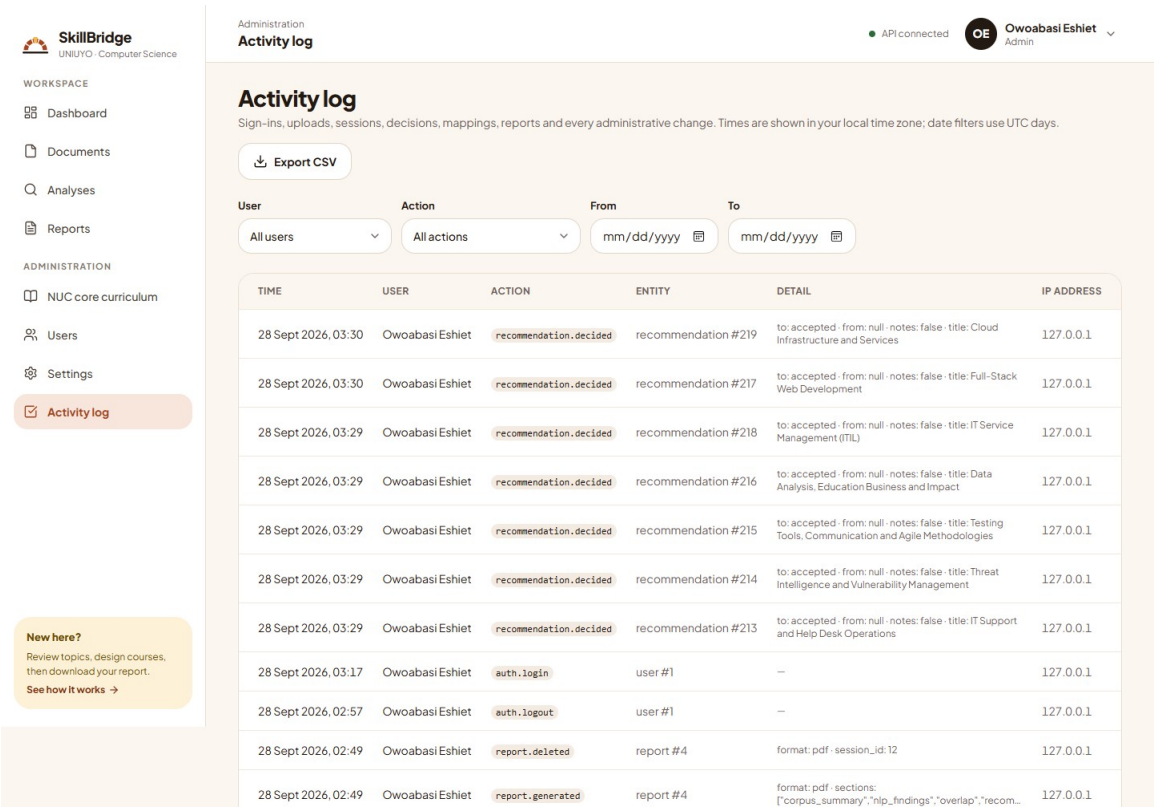


Figure 4.22: Activity log

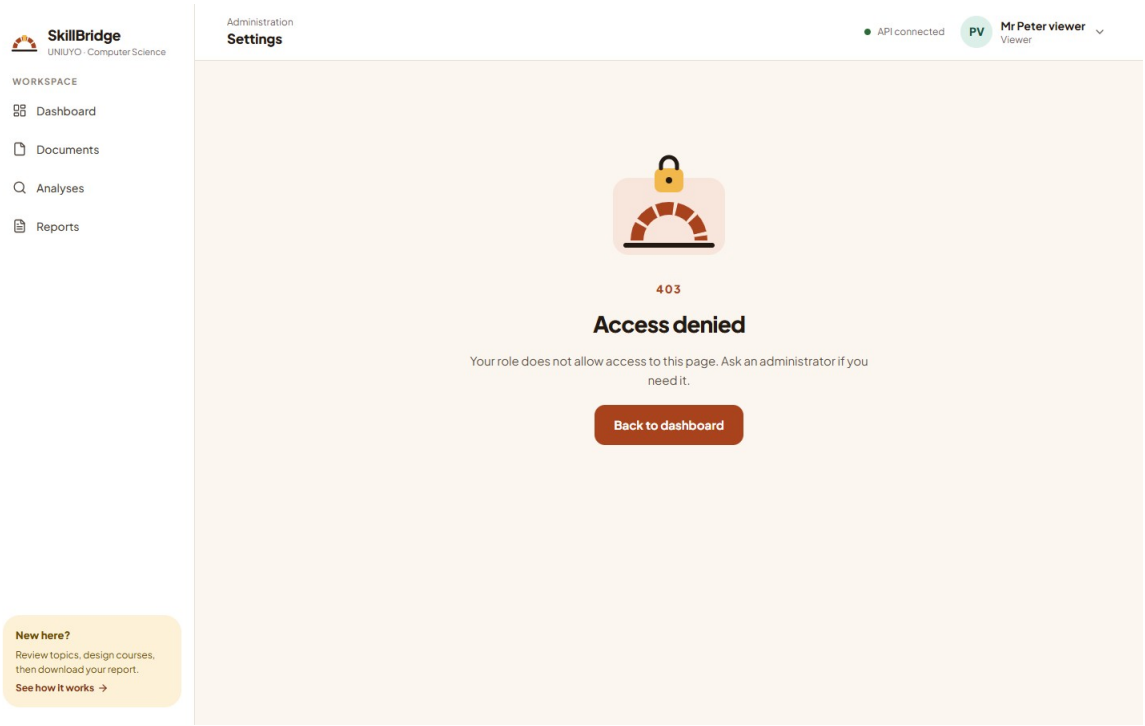


Figure 4.23: No access page shown to a Viewer

4.5 Testing

4.5.1 Testing Strategy

Testing means checking that the system does what it should and does not do what it should not. The system was tested at several levels, most of them automatically, so that the same checks could be repeated after every change:

- **Unit tests** check one small part at a time, for example that a file with the wrong content is rejected, that extracts have the right length, or that the score is calculated correctly.
- **Integration tests** check that the parts work together. Every API route was tested against a separate test database built in exactly the same way as the real one, including one complete analysis from start to finish. To keep the tests fast, most of them use a simple stand-in for the Sentence-BERT model, but one test runs the whole pipeline with the real model.
- **Access-control tests** check that each role can use only what it is allowed to; for example, that a Viewer is refused (HTTP 403) whenever they try to change data.
- **Front-end tests** check that each screen shows the right information and buttons, using the Vitest and Testing Library tools with a simulated server.
- **Code-quality checks** use tools (ruff, black, ESLint, Prettier and the TypeScript compiler) that find mistakes and inconsistent formatting without running the code.

- **Manual testing** involved opening every page as an Administrator, a Curriculum Planner and a Viewer. A script also confirmed that Viewers see no buttons that change data, and that no technical terms appear on screen outside "Show calculation" and the advanced settings.

4.5.2 Test Results

Table 4.3 summarises the results of the automated tests.

Table 4.3: Automated test results

Measure	Back end	Front end
Tools	pytest 9.1.1 with pytest-cov	Vitest 5.0.1 with Testing Library
Tests	405 collected: 392 passed, 13 skipped, 0 failed	133 passed, 0 failed (17 files)
Coverage	97.1% (statements and branches)	84.3% statements, 76.6% branches
Static checks	ruff: no issues; black: all files formatted	TypeScript: no errors; ESLint: no problems; Prettier: clean; build succeeds
Duration	495 s	139 s

The 13 skipped back-end tests require libmagic (12 tests) or the Pango library used by WeasyPrint (1 test), which are not available on native Windows; both are installed in the Docker images, where these features work. The back-end coverage of 97.1% exceeds the target of 85% set in the requirements.

Table 4.4 lists examples of the behaviours verified by the automated tests.

Table 4.4: Examples of verified behaviours

Test area	Input or action	Expected result	Result
Access control	Viewer calls any write endpoint	HTTP 403 Forbidden	Pass
Access control	Anonymous user calls a protected endpoint	HTTP 401 Unauthorised	Pass
Upload validation	File with a PDF extension but other content	Rejected as invalid	Pass
Upload validation	Same file uploaded twice	Second upload rejected as duplicate	Pass
Extract grouping	Long document	Extracts of 3–5 sentences, at most 200 words	Pass
Overlap	Topic with similarity exactly 0.80	Not flagged (flag only above 0.80)	Pass
Scoring	Weights that do not sum to 1	Rejected with a validation error	Pass
Full pipeline	Analysis on test documents with the real SBERT model	Completes with ranked topics	Pass
Report download	Download a generated Word or PDF report	File returned with the correct type and a readable name	Pass

Test area	Input or action	Expected result	Result
File storage	Storage folder given as a relative path in the settings	Resolved to one absolute folder for the API and the worker	Pass
New analysis	Planner opens the documents step	Upload documents button shown; upload window opens	Pass
New analysis	Uploaded document still being processed	Shown as "still being processed" until it can be selected	Pass

4.6 Results

4.6.1 Corpus Processing

The final analysis (session 13) processed the 56 documents described in Section 3.6. Table 4.5 summarises the outcome.

Table 4.5: Summary of the final analysis

Measure	Value
Documents / extracts analysed	56 / 417
Extracts containing at least one skill	213 of 417 (51.1%)
Distinct skills found	110
Topics found (after merging two groups of near-duplicates)	28
Extracts not assigned to any topic (outliers)	67 (16.1%)
Topics given a course-style name	21 of 28
Suggested topics kept	20
NUC courses compared / left out	86 / 32
Closest-course similarity (all 28 topics)	0.447 to 0.735
Topics flagged "May already be in NUC core" (> 0.80)	0
Total run time	178.2 s

4.6.2 Skills in Demand

Figure 4.24 shows the ten skills mentioned in the most documents. IT support was the most widespread, appearing in 22 of the 56 documents, followed by Agile methodologies, CI/CD and SQL (15 each) and information security (14). Cloud (Microsoft Azure), Linux, Python, DevOps and incident response each appeared in 10 or 11 documents.

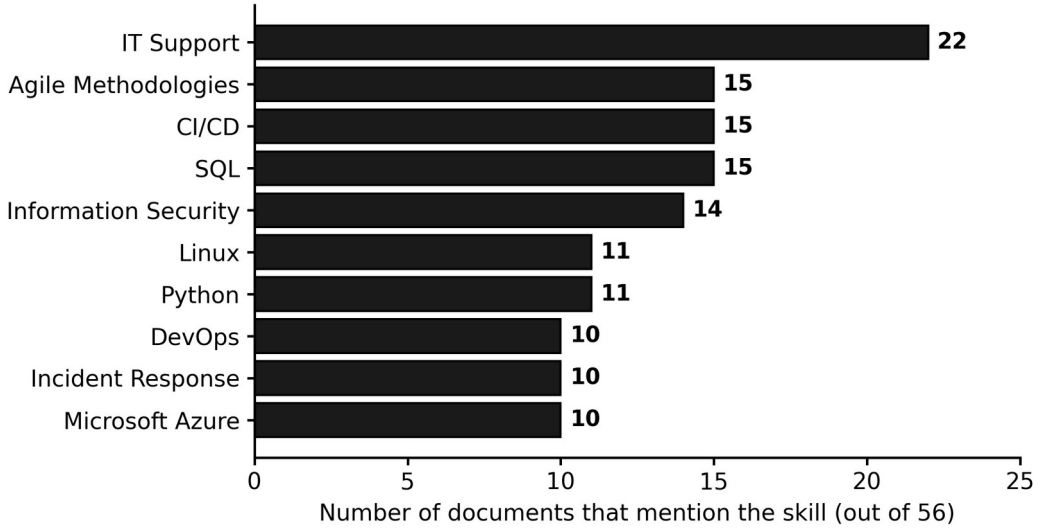


Figure 4.24: The ten skills mentioned in the most documents

4.6.3 Suggested Topics

Table 4.6 lists the 20 suggested topics of the final analysis with their scores, the closest NUC course and the decision recorded. Figure 4.25 shows how each score is made up of its three parts.

Table 4.6: Suggested topics of the final analysis (session 13)

Rank	Suggested topic	Score	Demand / Often / New	Closest NUC course (similarity)	Decision
1	IT Support and Help Desk Operations	79	84 / 100 / 42	IFT 410 System Integration and Architecture (58%)	Accepted
2	Threat Intelligence and Vulnerability Management	73	100 / 75 / 27	CYB 303 Cybersecurity Risks Analysis, Challenges and Mitigation (73%)	Accepted
3	Testing Tools, Communication and Agile Methodologies	72	93 / 63 / 49	SEN 306 Software Construction (51%)	Accepted
4	Data Analysis, Education Business and Impact	64	81 / 61 / 41	CYB 304 Information and Big Data Security (59%)	Accepted
5	Full-Stack Web Development	61	84 / 45 / 48	SEN 306 Software Construction (52%)	Accepted
6	IT Service Management (ITIL)	59	69 / 52 / 51	IFT 410 System Integration and Architecture (49%)	Accepted
7	Cloud Infrastructure and Services	57	87 / 33 / 42	CYB 404 Cloud Computing Security (58%)	Accepted
8	DevOps and Continuous Delivery	56	79 / 36 / 48	IFT 410 System Integration and Architecture (52%)	Not reviewed
9	Computer Science, System and Linux	56	69 / 47 / 47	IFT 410 System Integration and Architecture (53%)	Not reviewed
10	Data Engineering and Data Pipelines	54	78 / 37 / 37	DTS 302 Big Data Computing (63%)	Not reviewed

Rank	Suggested topic	Score	Demand / Often / New	Closest NUC course (similarity)	Decision
11	Mobile Application Development	53	58 / 49 / 50	SEN 410 Software Architecture and Design (50%)	Not reviewed
12	Agile Software Development	53	55 / 48 / 55	IFT 410 System Integration and Architecture (45%)	Not reviewed
13	Project, English and Problem	52	56 / 44 / 55	SEN 306 Software Construction (45%)	Not reviewed
14	Cloud, Container and Security DevOps	50	69 / 33 / 45	CYB 404 Cloud Computing Security (55%)	Not reviewed
15	Linux System Administration	48	61 / 35 / 45	SEN 306 Software Construction (55%)	Not reviewed
16	Data Engineering, Pl Pgsql and PostgreSQL	46	65 / 26 / 46	DTS 302 Big Data Computing (54%)	Not reviewed
17	Software Testing and Quality Assurance	31	24 / 32 / 42	CYB 401 Systems Vulnerability Assessment and Testing (58%)	Not reviewed
18	Digital Marketing	28	13 / 33 / 44	ENT 312 Venture Creation (56%)	Not reviewed
19	Digital Economy and Policy	27	31 / 14 / 40	CYB 203 Cybercrime, Law and Countermeasures (60%)	Not reviewed
20	Cyber Law and Data Protection	27	47 / 2 / 32	CYB 203 Cybercrime, Law and Countermeasures (68%)	Not reviewed

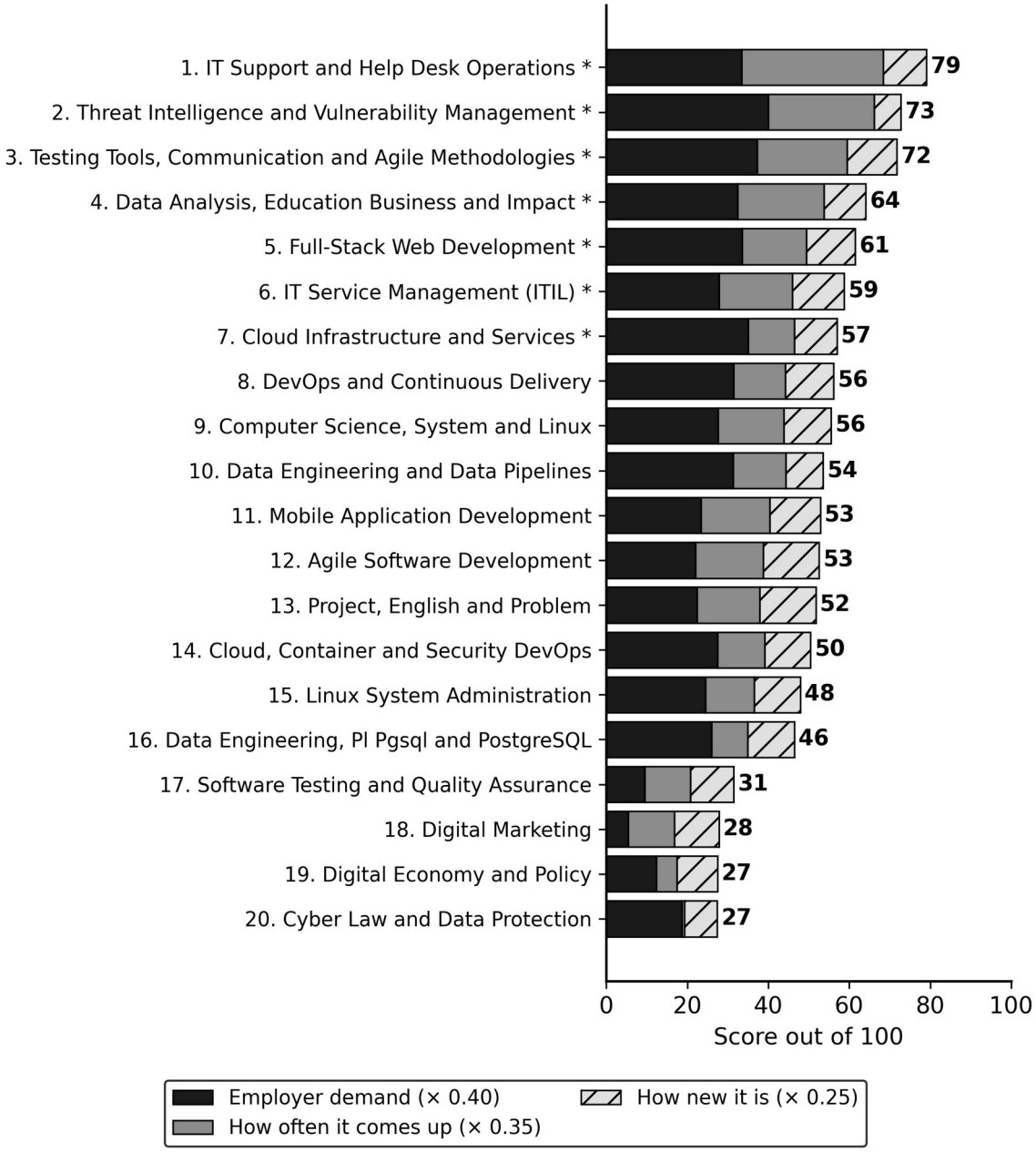


Figure 4.25: Composition of the scores of the 20 suggested topics

The top-ranked topic, IT Support and Help Desk Operations (79), combines strong employer demand with the highest prevalence: 16 of the 56 documents mention it, mostly adverts for roles such as IT specialist and systems administrator. Threat Intelligence and Vulnerability Management (73) has the highest employer demand but the lowest novelty among the top topics, because its closest NUC course, CYB 303 Cybersecurity Risks Analysis, Challenges and Mitigation, is also the closest match of any topic (73%); the system therefore correctly signals that it is partly, though not fully, covered. Topics from the policy document (Digital Economy and Policy; Cyber Law and Data Protection) ranked lowest, because a single document, weighted equally with each advert, contributes little to prevalence and names few technical skills.

Seven topics (ranks 1 to 7) were accepted during review of the final analysis; no course had yet been designed for them at the time of writing. In an earlier analysis (session 9), an accepted topic was designed as the course CSC 418 System, Networking and Technical (3 units), shown in Figure 4.15, demonstrating the complete path from evidence to proposed course.

4.6.4 Overlap with the NUC Core

No topic exceeded the 80% similarity threshold; the closest-course similarities ranged from 0.447 to 0.735, so all suggested topics were labelled "Not in NUC core". The highest similarities were for security-related topics compared with the Cybersecurity courses of the CCMAS (73% for threat intelligence, 68% for cyber law), and for data engineering compared with DTS 302 Big Data Computing (63%). This suggests that the practical, tool-oriented topics emphasised by employers, such as help-desk operations, IT service management, DevOps and cloud infrastructure, are not covered directly by the national core. However, because the threshold has not yet been calibrated against human judgement (Section 4.7.2), this finding should be treated as provisional.

4.6.5 Topic Quality Compared with LDA

To evaluate topic quality, the 28 topics produced by the system were compared with an LDA baseline trained on the same 417 extracts with the same number of topics (28), 20 passes and three random seeds. Coherence was computed with Gensim using the top 10 words of each topic. The results are shown in Table 4.7 and Figure 4.26.

Table 4.7: Topic coherence of BERTopic and LDA

Model	**C_v (higher is better)**	**UMass (closer to 0 is better)**
BERTopic (this system, 28 topics)	0.723	−2.927
LDA baseline (28 topics, mean ± SD of 3 seeds)	0.488 ± 0.012	−3.454 ± 0.233

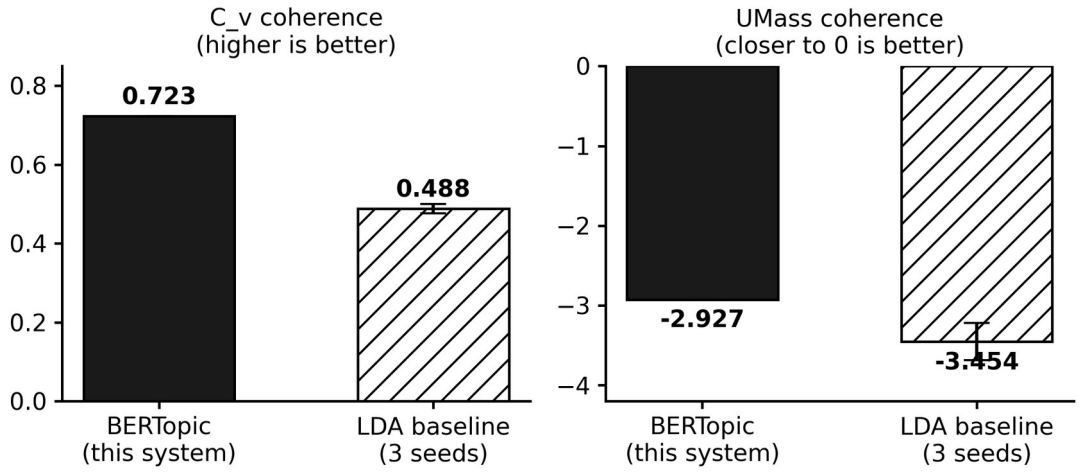


Figure 4.26: Topic coherence of BERTopic and the LDA baseline

The system's topics are markedly more coherent than those of LDA on both measures, with a C_v of 0.723 against 0.488. An earlier comparison on session 10, before topic merging (31 topics), gave a similar margin (0.726 against 0.521 ± 0.041), indicating that the advantage is consistent.

4.6.6 Repeatability

Two further analyses were run with the same 56 documents and settings as session 13. Both produced exactly the same 28 topics, 67 outliers, 20 suggested topics, ranks and scores. This confirms that, with fixed random seeds, the system's results are fully repeatable, an important property when a curriculum committee must be able to reproduce the evidence behind its decisions.

4.6.7 Processing Speed

Table 4.8 and Figure 4.27 show the time taken by each stage of the analysis.

Table 4.8: Time per stage of the analysis (seconds)

Stage	Session 13 (first run)	Re-run (warm worker)
Validating	0.28	0.26
Loading data	45.01	8.27
Keywords (TF-IDF)	0.28	0.10
Skills (NER)	9.21	6.20
Embeddings	0.04	0.08
Topics (BERTopic)	123.21	8.97
NUC overlap	0.06	0.03
Scoring	0.02	0.01
Total	178.2	24.0

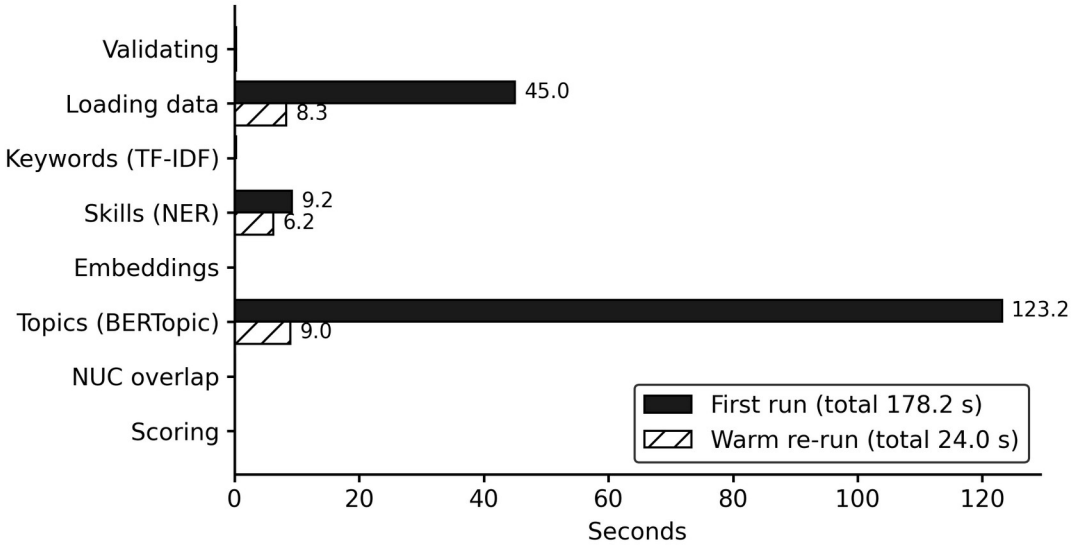


Figure 4.27: Time per stage, first run and warm run

The first analysis after the worker starts takes about three minutes. Most of this time is spent finding topics, because the mathematical libraries used by UMAP and HDBSCAN prepare (compile) their code the first time they are used. After that one-off preparation, the worker is "warm" and the same analysis takes only 24.0 seconds. Embeddings take almost no time during analysis because they are computed once at upload. On a synthetic benchmark of 20 documents (60,143 words, 896 extracts), an analysis took 45.1 seconds on the first run and 10.5 seconds warm, excluding one-off model loading (34.2 seconds). Both warm results are well within the target of 60 seconds for about 20 documents.

4.7 Evaluation with Human Participants

This part of the evaluation requires human annotators and participants and was in progress at the time of writing. The materials are complete: an annotator guide, 60 extracts stratified by document type for skill annotation, 140 topic–course pairs (the three most similar and two random NUC courses for each of the 28 topics), a task sheet with five tasks, and the SUS questionnaire.

4.7.1 Skill Extraction Accuracy

Two annotators independently mark the skills in each of the 60 extracts, using the list of 172 canonical skills; disagreements are resolved to form a reference set. The system's extracted skills are compared with the reference set.

[TO FILL AFTER EVALUATION] Insert Table 4.9 (Skill extraction results): micro precision, recall and F1 overall and by skill type (skill, tool, language, certification), Cohen's kappa between the two annotators with its Landis and Koch interpretation, and the

most frequently missed and wrongly extracted skills. Targets: precision > 0.80, recall > 0.75, F1 > 0.75.

4.7.2 Overlap Detection and Threshold Calibration

Annotators label each of the 140 topic–course pairs as "covered" (the course already teaches the topic) or "not covered". The system's similarity scores are compared with these labels.

[TO FILL AFTER EVALUATION] Insert Table 4.10 (Overlap detection results): AUC-ROC; precision, recall and F1 at the 0.80 threshold; the threshold with the best F1 and by Youden's J from the sweep 0.40–0.95; and a sentence stating whether the default threshold of 0.80 should be changed.

4.7.3 Usability

Between five and eight lecturers, Heads of Department or curriculum officers each complete five tasks on the final analysis (find the analysis, explain a score, decide on three topics, design a course and download the report) while the researcher observes, and then complete the ten-item SUS questionnaire.

[TO FILL AFTER EVALUATION] Insert Table 4.11 (Usability results): number and roles of participants; task completion rate per task; SUS score per participant; mean SUS, standard deviation and 95% confidence interval; adjective rating (Bangor et al., 2009); and the main comments. Target: mean SUS ≥ 70.

4.8 Discussion of Results

The results show that the system meets its objectives. Objective 1 was achieved: the system assembled and processed a corpus of 56 real documents into 417 extracts with embeddings, with validation that rejects invalid and duplicate files. Objective 2 was achieved: 110 distinct skills were extracted, and the topic model produced 28 topics that were substantially more coherent than an LDA baseline (C_v 0.723 against 0.488). Objective 3 was achieved: every topic was compared with the 86 subject courses of the CCMAS, and topics were ranked with a transparent three-part score that users can inspect down to the exact calculation. Objective 4 was achieved: the web application supports the full workflow from document upload to report, with server-enforced roles and an activity log, and passed all automated tests that could run on the development machine, with 97.1% back-end coverage. Objective 5 was partly achieved: topic quality, repeatability, speed and software correctness were evaluated and met their targets, while the evaluations requiring human participants are in progress.

Several observations arise from the results:

- **Employer demand is practical and operational.** The top-ranked topics (IT support, threat intelligence, testing and Agile practice, data analysis, full-stack development, IT service management and cloud infrastructure) are operational, tool-based areas that Nigerian employers advertise for. This agrees with earlier findings that IT skills are among the weakest areas of Nigerian graduates (Pitan & Adedeji, 2012) and with the African evidence of gaps in cloud and security skills (Nyale et al., 2026).
- **The NUC core is academic rather than operational.** The low similarities to the CCMAS (at most 0.735) suggest that the core curriculum describes these areas at a conceptual level, if at all, leaving a clear opportunity for the 30% component. Where similarity was highest, as for threat intelligence and CYB 303, the system correctly lowered the topic's novelty, showing that the overlap check works as intended.
- **Some topics remain generic.** A few topics kept keyword titles, and one ("Project, English and Problem") reflects the general content of job adverts, such as client communication and problem solving, rather than a technical subject. The human review step exists precisely for such cases, and the planner can rename, reject or defer any topic.
- **The corpus shapes the results.** With 55 job adverts and one policy document, the results reflect employer demand far more than national policy. Adding institutional and academic documents and more policy documents would give a more balanced view, and the equal weighting of documents ensures that no single long document dominates.
- **Performance is adequate for the intended use.** Even on a two-core laptop without a GPU, a full analysis completes in three minutes on a cold start and in under half a minute thereafter, which is acceptable for a task carried out a few times per curriculum review.

4.9 Summary

This chapter has described the implementation of the system, presented its interfaces, and reported its testing and results. All automated tests that could run on the development machine passed; the system processed the real corpus into 20 ranked suggested topics with full evidence, produced topics that were more coherent than an LDA baseline, gave identical results on repeated runs and met its speed requirement. The evaluation of skill extraction, overlap detection and usability with human

participants is in progress. The next chapter summarises the study and presents its conclusions and recommendations.

CHAPTER FIVE
SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

The Core Curriculum and Minimum Academic Standards (CCMAS) of the National Universities Commission leave about 30% of each programme to be designed by each university, but provide no method for deciding what that 30% should contain. In practice the decision rests largely on the opinions of committee members, with little recorded evidence, a real risk of duplicating the national core, and no trail linking identified needs to the courses eventually designed. This study set out to develop an NLP-driven curriculum recommendation system that brings evidence from job adverts and policy documents into that decision.

Chapter One introduced the problem, the aim and five objectives, and the scope and limitations of the study. Chapter Two reviewed the Nigerian curriculum framework and the evidence of a skills gap among Nigerian graduates, explained the NLP techniques used (text preprocessing, TF-IDF, rule-based named entity recognition, Sentence-BERT embeddings, BERTopic topic modelling and coherence measures), and reviewed related works that analyse job adverts to inform curricula. It identified a gap: no work was found that text-mines Nigerian job adverts for the CCMAS 30%, and few systems support the human decisions that follow the analysis.

Chapter Three described the incremental, phase-based development methodology, analysed the existing manual process and specified the requirements of the proposed system. It presented a three-tier architecture with a background worker; a pipeline that validates, parses, cleans, segments and embeds documents; an analysis that extracts skills, discovers, merges and names topics, compares each topic with the 86 subject courses of the CCMAS for Computing, and ranks topics with a transparent score combining employer demand (40%), how often a topic comes up (35%) and how new it is (25%); and the use case, activity, sequence and database designs.

Chapter Four described the implementation of SkillBridge using React, TypeScript, Flask, PostgreSQL with pgvector, Redis, spaCy, Sentence-BERT and BERTopic, and presented its interfaces. Applied to 55 Nigerian job adverts and the National Digital Economy Policy and Strategy, the system produced 28 topics and 20 ranked suggested topics, led by IT support and help desk operations, threat intelligence and vulnerability management, and testing and Agile practice. Its topics were more coherent than those of an LDA baseline (C_v 0.723 against 0.488), repeated runs gave identical results, a warm analysis completed in 24 seconds, and all 522 automated tests that could run on the development machine passed, with 97.1% back-end coverage. The evaluation of

skill extraction, overlap detection and usability with human participants was in progress.

5.2 Conclusion

This study has shown that natural language processing can turn the unstructured text of job adverts and policy documents into structured, evidence-backed suggestions for the university-designed 30% of the CCMAS, and that such suggestions can be delivered in a form that lecturers and administrators can use.

The system meets its objectives. It builds and processes a corpus of real Nigerian documents; extracts skills and discovers coherent topics; checks each topic by meaning against every subject course of the NUC core and ranks topics with a score whose parts and weights are visible and adjustable; and supports the complete human workflow of reviewing suggestions, recording decisions, designing courses within the credit allowance and producing a report, with every action recorded. Its results are repeatable, its speed is adequate on ordinary hardware, and it is thoroughly tested.

The results also carry a substantive message for curriculum planners in Nigerian Computer Science departments. The skill areas most emphasised by employers in the corpus, such as IT support, IT service management, cloud infrastructure, DevOps and security operations, are practical and tool-oriented, and none of them was found to be closely matched by a course in the national core. These areas are strong candidates for the 30% component, subject to the judgement of the curriculum committee.

Most importantly, the system does not replace that judgement. It changes the basis of the committee's discussion from recollection and opinion to documented evidence, and it leaves a record of what was decided and why. In this way it offers a practical means of using the flexibility that the CCMAS provides in a systematic and accountable manner.

5.3 Contributions to Knowledge

The study makes the following contributions:

1. **A method for evidence-based design of the CCMAS 30%.** To the best of the author's knowledge, based on the sources reviewed, this is the first system to apply transformer-based NLP to Nigerian job adverts and a national policy document for the purpose of designing the university-determined component of the CCMAS.
2. **Semantic overlap checking against a national core curriculum.** The study demonstrates course-level comparison of discovered topics with the courses of the

CCMAS for Computing, including a principled exclusion of generic courses such as SIWES, projects and general studies that otherwise distort the comparison.

3. **A transparent, adjustable ranking model.** The study proposes and implements a three-part score with document-weighted prevalence, specificity-weighted skill demand and novelty, whose calculation is fully visible to users.
4. **A human-in-the-loop curriculum workflow.** The system links each suggestion to a recorded decision, a designed course within the 30% credit allowance and a generated report, providing an audit trail suitable for Senate and accreditation.
5. **A reproducible evaluation design and corpus-collection method** for curriculum analytics in a Nigerian context, including a polite job-advert collector, annotation materials and a usability protocol.

5.4 Recommendations

Based on the findings, the following recommendations are made:

1. **To the Department of Computer Science, University of Uyo,** and similar departments: use the system as an input to the next review of the 30% component, giving the committee the ranked topics, their evidence and the report, and recording the committee's decisions in the system.
2. **To curriculum committees:** consider the operational, tool-oriented areas highlighted by the analysis (IT support and service management, cloud and DevOps, security operations, testing and full-stack development) as candidates for elective or specialisation courses in the 30% component, and review the list at every curriculum review using newly collected adverts.
3. **To the National Universities Commission:** consider providing guidance, and possibly shared tools, to help universities design the 30% component on the basis of labour-market and policy evidence, and make the CCMAS available in a structured, machine-readable form to support such tools.
4. **To employers and professional bodies:** publish job adverts with clear, specific skill requirements, since these adverts are a valuable signal to universities about the skills needed.
5. **To the system's administrators:** broaden the corpus before relying on the results, adding institutional documents, academic sources, more policy documents and adverts from more sources and over a longer period; and calibrate the overlap threshold using the labelled topic–course pairs.

5.5 Suggestions for Further Work

The following extensions are suggested for future work:

- **Completing and extending the human evaluation,** including a larger usability study with lecturers from several universities and a study of whether committees using the system reach different or better decisions.
- **Automated, continuous collection** of job adverts within the application, with trends over time, so that emerging skills can be detected as they appear.
- **Fine-tuned skill extraction,** training a model on annotated Nigerian job adverts to complement the rule-based patterns and recognise skills not yet in the list.
- **Support for other disciplines and programmes,** since the pipeline is not specific to Computer Science and could be applied to other CCMAS disciplines.
- **Learning-outcome generation and course outlines,** suggesting draft learning outcomes and content for accepted topics, for the committee to edit.
- **Optical character recognition** for scanned PDF documents, which are common among older institutional documents.
- **Production deployment** with HTTPS, rate limiting of sign-in attempts and hosting, so that several departments can use the system securely.

REFERENCES

Ahadi, A., Kitto, K., Rizoiu, M.-A., & Musial, K. (2022). Skills taught vs skills sought: Using skills analytics to identify the gaps between curriculum and job markets. In *Proceedings of the 15th International Conference on Educational Data Mining* (p. 542). https://doi.org/10.5281/zenodo.6853121

Aljohani, N. R., Aslam, A., Khadidos, A. O., & Hassan, S.-U. (2022). Bridging the skill gap between the acquired university curriculum and the requirements of the job market: A data-driven analysis of scientific literature. *Journal of Innovation & Knowledge, 7*(3), 100190. https://doi.org/10.1016/j.jik.2022.100190

Ayofe, A. N., & Ajetola, A. R. (2009). Exploration of the gap between computer science curriculum and industrial I.T skills requirements. *International Journal of Computer Science and Information Security, 4*(1 & 2). https://arxiv.org/abs/0908.4353

Bangor, A., Kortum, P., & Miller, J. (2009). Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies, 4*(3), 114–123.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. https://agilemanifesto.org/

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research, 3*, 993–1022.

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In J. Pei, V. S. Tseng, L. Cao, H. Motoda, & G. Xu (Eds.), *Advances in knowledge discovery and data mining* (pp. 160–172). Springer. https://doi.org/10.1007/978-3-642-37456-2_14

Chime, I. (2022). Digital economy policy: The case example of Nigeria. *International Journal of Innovative Finance and Economics Research, 10*(1), 176–189.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

De Mauro, A., Greco, M., Grimaldi, M., & Ritala, P. (2018). Human resources for big data professions: A systematic classification of job roles and required skill sets. *Information Processing & Management, 54*(5), 807–817. https://doi.org/10.1016/j.ipm.2017.05.004

Debortoli, S., Müller, O., & vom Brocke, J. (2014). Comparing business intelligence and big data skills: A text mining study using job advertisements. *Business & Information Systems Engineering*, *6*(5), 289–300.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Federal Ministry of Communications and Digital Economy. (2019). *National digital economy policy and strategy (2020–2030): For a digital Nigeria*. Federal Republic of Nigeria. https://nitda.gov.ng/wp-content/uploads/2020/06/National-Digital-Economy-Policy-and-Strategy.pdf

Grootendorst, M. (2022). *BERTopic: Neural topic modeling with a class-based TF-IDF procedure* (arXiv:2203.05794). https://doi.org/10.48550/arXiv.2203.05794

Gurcan, F., & Cagiltay, N. E. (2019). Big data software engineering: Analysis of knowledge domains and skill sets using LDA-based topic modeling. *IEEE Access*, *7*, 82541–82552. https://doi.org/10.1109/ACCESS.2019.2924075

Gurcan, F., Soylu, A., & Khan, A. Q. (2025). Towards a sustainable workforce in big data analytics: Skill requirements analysis from online job postings using neural topic modeling. *Sustainability*, *17*(20), 9293. https://doi.org/10.3390/su17209293

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength natural language processing in Python*. Zenodo. https://doi.org/10.5281/zenodo.1212303

James, G., et al. (2026). Delivering the Core Curriculum and Minimum Academic Standards (CCMAS) in a virtual platform in Nigeria: A systematic review. *International Journal of Education and Management Engineering*, *16*(1), 1–20. https://doi.org/10.5815/ijeme.2026.01.01

Joint Task Force on Computer Science Curricula. (2024). *Computer science curricula 2023*. ACM Press, IEEE Computer Society Press, & AAAI Press. https://doi.org/10.1145/3664191

Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd ed. draft). https://web.stanford.edu/~jurafsky/slp3/

Kauttonen, J., Khan, U. A., Aunimo, L., Nyqvist, A., & Klemetti, A. (2024). Topic mining for theses and job ads in ICT sector: Can higher education institutes

respond to job market demands? *Frontiers in Education, 9*, 1322774. https://doi.org/10.3389/feduc.2024.1322774

Kverne, C. L., Monteverdi, F., Polyzou, A., Lisetti, C., & Bhimani, J. (2025). *Course-job fit: Understanding the contextual relationship between computing courses and employment opportunities* [Conference paper]. ASEE Annual Conference. https://peer.asee.org/course-job-fit-understanding-the-contextual-relationship-between-computing-courses-and-employment-opportunities.pdf

Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

Lavi, D., Medentsiy, V., & Graus, D. (2021). *conSultantBERT: Fine-tuned Siamese Sentence-BERT for matching jobs and job seekers* (arXiv:2109.06501). https://arxiv.org/abs/2109.06501

Li, Y., Huang, T., & Li, X. (2025). Training path of big data management and application talents based on BERTopic-TOPSIS model. *PLOS ONE, 20*(12), e0334127. https://doi.org/10.1371/journal.pone.0334127

Li, Z., & Jia, X. (2024). Analysis of logistics curriculum and recruitment requirements based on text mining: A case study of China. *SAGE Open, 14*(2). https://doi.org/10.1177/21582440241239839

McCowan, T., Walker, M., Fongwa, S., Oanda, I., Sifuna, D., Adedeji, S., Oyebade, S., Ananga, E. D., Adzahlie-Mensah, V., & Tamanja, E. (2016). *Universities, employability and inclusive development: Repositioning higher education in Ghana, Kenya, Nigeria and South Africa*. British Council.

McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *Journal of Open Source Software, 2*(11), 205. https://doi.org/10.21105/joss.00205

McInnes, L., Healy, J., & Melville, J. (2018). *UMAP: Uniform manifold approximation and projection for dimension reduction* (arXiv:1802.03426). https://doi.org/10.48550/arXiv.1802.03426

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space* (arXiv:1301.3781). https://doi.org/10.48550/arXiv.1301.3781

National Bureau of Statistics. (2024). *Nigeria labour force survey: Q2 2024*. https://www.nigerianstat.gov.ng/pdfuploads/NLFS_Q2_2024.pdf

National Universities Commission. (2022). *Core curriculum and minimum academic standards for the Nigerian university system: Computing*. https://www.nuc.edu.ng/wp-content/uploads/2026/03/Computing-CCMAS-2023-FINAL.pdf

National Universities Commission. (2023, August 31). *FG insists CCMAS implementation to begin September 2023*. https://www.nuc.edu.ng/fg-insists-ccmas-implementation-to-begin-september-2023/

Nyale, D., Karume, S., Kipkebut, A., Mukudi, F., & Sharafat, A. (2026). An intelligent curriculum alignment model for digital skills integration in higher education: A data-driven approach to bridge academia–industry gaps. *Discover Education, 5*, 151. https://doi.org/10.1007/s44217-026-01180-y

Ogunode, N. J., Abdullahi, A., & Sani, K. (2024). Core Curriculum and Minimum Academic Standards (CCMAS) implementation in Nigerian universities: Roles of artificial intelligence. *American Journal of Technology Advancement, 1*(2), 1–9.

Ojan, M.-P., Lara-Navarra, P., & Sanz-Martos, S. (2026). Bridging communication studies and employability: ESCO-based curriculum mapping and job-vacancy skill signals. *Education Sciences, 16*(4), 606. https://doi.org/10.3390/educsci16040606

Okolie, U. C., Igwe, P. A., Nwosu, H. E., Eneje, B. C., & Mlanga, S. (2020). Enhancing graduate employability: Why do higher education institutions have problems with teaching generic skills? *Policy Futures in Education, 18*(2), 294–313. https://doi.org/10.1177/1478210319864824

Omoniwa, O. B., & Adedapo, A. A. (2017). Assessing Nigerian graduate employability. *Global Journal of Business Disciplines, 1*(2), 58–75.

Onifade, O. J., Gambo, O. O., Ogunleye, A. T., & Ajayi, P. O. (2026). Assessing the relevance of the undergraduate computer science curriculum of Nigerian universities: Insights from industry. *Educational Dimension, 14*, 117–132. https://doi.org/10.55056/ed.864

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Pejic-Bach, M., Bertoncel, T., Meško, M., & Krstić, Ž. (2020). Text mining of Industry 4.0 job advertisements. *International Journal of Information Management, 50*, 416–431. https://doi.org/10.1016/j.ijinfomgt.2019.07.014

Pitan, O. S., & Adedeji, S. O. (2012). Skills mismatch among university graduates in the Nigeria labor market. *US-China Education Review A, 1*, 90–98.

Pitan, O. S., & Muller, C. (2023). Assessment of strategies for preparing graduates for the disruptive workplace: Evidence from Nigeria and South Africa. *Journal of Teaching and Learning for Graduate Employability, 14*(1), 15–30. https://doi.org/10.21153/jtlge2023vol14no1art1576

Rasheed, A. A. (n.d.). *Preface* [to the Core Curriculum and Minimum Academic Standards]. National Universities Commission. https://nuc-ccmas.ng/preface/

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on*

Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (pp. 3982–3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). *Recommender systems handbook* (3rd ed.). Springer. https://doi.org/10.1007/978-1-0716-2197-4

Röder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. In *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining* (pp. 399–408). ACM. https://doi.org/10.1145/2684822.2685324

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Samwi, E., Matto, G., & Mbise, M. (2026). Association rule mining for uncovering co-occurring skill sets in multi-portal ICT job advertisements. *Journal of Information Systems and Informatics, 8*(4), 4074–4101. https://doi.org/10.63158/journalisi.v8i4.1671

Senger, E., Zhang, M., van der Goot, R., & Plank, B. (2024). Deep learning-based computational job market analysis: A survey on skill extraction and classification from job postings. In *Proceedings of the First Workshop on Natural Language Processing for Human Resources* (pp. 1–15). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.nlp4hr-1.1

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Soni, R., Suleiman, B., & Singh, S. (2025). *Improving graduate outcomes by identifying skills gaps and recommending courses based on career interests* (arXiv:2511.09819). https://arxiv.org/abs/2511.09819

Turaev, S., Aldabet, S., John, M., Musthafa, N., Awad, M., Zaki, N., & Shuaib, K. (2026). *Measuring curriculum–labor market alignment at the scale of a program portfolio* (arXiv:2608.12356). https://arxiv.org/abs/2608.12356

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30*. https://papers.nips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Vogt, J., Voigt, T., Nowak, A., & Pawlowski, J. M. (2023). Development of a job advertisement analysis for assessing data science competencies. *Data Science Journal, 22*, 33. https://doi.org/10.5334/dsj-2023-033

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained

transformers. In *Advances in Neural Information Processing Systems 33* (pp. 5776–5788). Curran Associates.

Yaprak, B., Ercan, S., Coşan, B., & Ecevit, M. Z. (2025). Curriculum–skill gap in the AI era: Assessing alignment in communication-related programs. *Journalism and Media, 6*(4), 171. https://doi.org/10.3390/journalmedia6040171

Zhang, M., Jensen, K., Sonniks, S., & Plank, B. (2022). SkillSpan: Hard and soft skill extraction from English job postings. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4962–4984). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.naacl-main.366

APPENDIX A
SYSTEM USABILITY SCALE (SUS) QUESTIONNAIRE AND TASKS

Participants rate each statement from 1 (strongly disagree) to 5 (strongly agree) after completing the tasks. The SUS score is computed as follows: for odd-numbered items subtract 1 from the rating; for even-numbered items subtract the rating from 5; add the ten results and multiply by 2.5, giving a score from 0 to 100 (Brooke, 1996).

Table A.1: SUS items

No.	Statement	1	2	3	4	5
1	I think that I would like to use this system frequently.					
2	I found the system unnecessarily complex.					
3	I thought the system was easy to use.					
4	I think that I would need the support of a technical person to be able to use this system.					
5	I found the various functions in this system were well integrated.					
6	I thought there was too much inconsistency in this system.					
7	I would imagine that most people would learn to use this system very quickly.					
8	I found the system very cumbersome to use.					
9	I felt very confident using the system.					
10	I needed to learn a lot of things before I could get going with this system.					

Table A.2: Usability tasks (on the final analysis, session 13)

Task	Description
1	Find the final analysis and open its suggested topics.
2	Open the top-ranked topic and explain in your own words why it scored as it did.
3	Decide on three topics (accept, reject or discuss later), adding a note to one of them.
4	Design a course for an accepted topic (code, title, units and at least two learning outcomes).
5	Download the report of the analysis.

APPENDIX B
API ENDPOINTS

All endpoints are under /api. "All roles" means any signed-in user; roles are enforced on the server.

Table B.1: API endpoints

Method	Path	Purpose	Roles
GET	/api/admin/audit-logs	The activity (audit) trail, newest first, with filters.	Admin
GET	/api/admin/audit-logs/export	The filtered audit trail as CSV (at most 100,000 rows, newest first).	Admin
GET	/api/admin/settings	Every setting with its current value, default and description.	Admin
PUT	/api/admin/settings	Change any subset of the settings. New values apply to sessions created afterwards	Admin
GET	/api/admin/skill-patterns	Skill patterns by canonical name. Filters: label, is_active, search.	Admin
POST	/api/admin/skill-patterns	Add a pattern; it is used by sessions run afterwards.	Admin
PATCH	/api/admin/skill-patterns/ <int:pattern_id>	Edit a pattern or (de)activate it.	Admin
GET	/api/admin/stop-words	Domain stop words, alphabetically. Filters: is_active, search.	Admin
POST	/api/admin/stop-words	Add a stop word; sessions run afterwards leave it out of keywords and theme words.	Admin
PATCH	/api/admin/stop-words/ <int:word_id>	Activate or deactivate a stop word.	Admin
GET	/api/admin/users	List users. Filters: role, is_active (true/false), search.	Admin
POST	/api/admin/users	Create a user with an initial password.	Admin
PATCH	/api/admin/users/<int:user_id>	Change a user's name, email, role or active flag, or reset their password.	Admin
POST	/api/auth/change-password	Change the logged-in user's password. Other sessions of this user are ended.	All roles
GET	/api/auth/csrf	Return a CSRF token; send it as X-CSRFToken on every state-changing request.	Public
POST	/api/auth/login	Log in with username or email + password; sets the session cookie.	Public
POST	/api/auth/logout	End the current session. Returns a fresh CSRF token for the anonymous session.	All roles
GET	/api/auth/me	The logged-in user.	All roles

Method	Path	Purpose	Roles
GET	/api/dashboard/summary	Counts of documents, sessions, suggested topics awaiting review and mapped courses.	All roles
GET	/api/documents	List documents with filters, plus per-category counts of active documents.	All roles
POST	/api/documents	Upload one or more files (multipart files) with one categories value each.	Admin, Planner
DELETE	/api/documents/<int:document_id>	Permanently delete a document that no analysis session has used.	Admin, Planner
GET	/api/documents/<int:document_id>	Document metadata, a short text preview and the sessions that use it.	All roles
PATCH	/api/documents/<int:document_id>	Change the details used wherever the document is referenced: title, source, link, date.	Admin, Planner
POST	/api/documents/<int:document_id>/archive	Hide a document from the library and from new sessions; past sessions keep it.	Admin, Planner
GET	/api/documents/<int:document_id>/file	Download the original file (never served from a public path).	All roles
GET	/api/documents/<int:document_id>/passages	All extracted passages of a document, in order (paginated).	All roles
GET	/api/health	Report API liveness and the reachability of Postgres and Redis.	Public
DELETE	/api/mappings/<int:mapping_id>	Delete a course design (mapping).	Admin, Planner
PUT	/api/mappings/<int:mapping_id>	Replace the course details of a mapping.	Admin, Planner
GET	/api/nuc-core	The active version (null if none) and the most recent upload (may still be processing).	All roles
POST	/api/nuc-core	Upload a new version (multipart: file and version_label).	Admin
GET	/api/nuc-core/courses	The courses found in the active version, and whether each is compared with themes.	All roles
GET	/api/nuc-core/versions	Every uploaded version, newest first.	All roles
GET	/api/recommendations/<int:recommendation_id>	Full detail: scores, evidence passages, the closest NUC passage and the mapping.	All roles
PATCH	/api/recommendations/<int:recommendation_id>	Change the title and/or description (the machine title stays in auto_title).	Admin, Planner
PATCH	/api/recommendations/<int:recommendation_id>/decision	Accept, reject or mark for later discussion (null clears the decision).	Admin, Planner
GET	/api/recommendations/<int:recommendation_id>/mapping	Get the course design of a suggested topic.	All roles
POST	/api/recommendations/<int:recommendation_id>/mapping	Map an accepted suggested topic to a proposed course (one course per suggested topic).	Admin, Planner

Method	Path	Purpose	Roles
GET	/api/reports	Reports, newest first. Filters: session_id, status, format.	All roles
DELETE	/api/reports/<int:report_id>	Delete a report and its file (not while it is being generated).	Admin, Planner
GET	/api/reports/<int:report_id>	Get the status and details of a report.	All roles
GET	/api/reports/<int:report_id>/download	Download a generated report file.	All roles
GET	/api/sessions	Sessions, newest first. Filters: status, search (name).	All roles
POST	/api/sessions	Create a session; with "run": true it is queued for analysis straight away.	Admin, Planner
DELETE	/api/sessions/<int:session_id>	Delete a session, its results and its report files (documents are kept).	Admin, Planner
GET	/api/sessions/<int:session_id>	Status, stage, progress, documents, parameters and NUC core version of a session.	All roles
GET	/api/sessions/<int:session_id>/curriculum	Every proposed course of a session, the total credit units and the allowance.	All roles
GET	/api/sessions/<int:session_id>/entities	Skills, tools, languages and certifications: mentions and document frequency.	All roles
GET	/api/sessions/<int:session_id>/keywords	Top TF-IDF terms overall and per category. ?category= limits to one category.	All roles
GET	/api/sessions/<int:session_id>/progress	Where a session stands in the review workflow: review → design courses → report.	All roles
GET	/api/sessions/<int:session_id>/recommendations	All suggested topics of a session in rank order, with review progress.	All roles
GET	/api/sessions/<int:session_id>/recommendations/export	Every suggested topic as CSV, with scores both out of 100 and as exact 0-1 decimals.	All roles
POST	/api/sessions/<int:session_id>/reports	Queue a report of a completed session (202 Accepted; poll the report for its status).	Admin, Planner
POST	/api/sessions/<int:session_id>/retry	Run a failed session again from the start (202 Accepted).	Admin, Planner
POST	/api/sessions/<int:session_id>/run	Queue a pending session for analysis (202 Accepted; poll the session for progress).	Admin, Planner
GET	/api/sessions/<int:session_id>/similarity	Overlap of every theme with the NUC core, with the closest NUC passage.	All roles
GET	/api/sessions/<int:session_id>/topics	Every discovered theme with keywords, size, strength and sample passages.	All roles

Method	Path	Purpose	Roles
GET	/api/sessions/defaults	Default parameters for a new session, the document limit and the NUC core status.	All roles

APPENDIX C
SOURCE CODE AND DATA

The complete source code of the system, including the evaluation scripts, database migrations and automated tests, is held in the project repository (github.com/eshietowoabasi/NLP-REC-SYSTEM). The machine-readable results used in Chapter Four (the suggested topics of session 13, the list of documents analysed, topic-coherence results, stage timings and test results) are stored in the docs/thesis/results folder of the repository. Collected job-advert texts are not reproduced, in order to respect the rights of the advertisers.